Production Agent Engineering in 2026

A production systems field manual for advanced engineers

Technical editor: Ankit Kumar Pandey

29 July 2026

Contents

Publication information

Edition: 1.7.0 - release-integrity revision
Technical editor: Ankit Kumar Pandey
Publication date: 29 July 2026

Maintained-edition policy: framework-specific examples are version-pinned and must be revalidated before reuse against a later SDK. Corrections should increment the semantic edition identifier and record the affected sections in the changelog.

This book is a systems field manual for senior engineers building production agents. It is not a claim that every implementation shown is universal or permanently current. Organising models such as the seven planes and the agency budget are manuscript conventions, introduced to make design review tractable. They are not industry-standard taxonomies.

The reader-facing PDF does not claim external peer review. The code mappings were checked against the cited versioned documentation and source releases. Independent review by an SDK specialist, a security engineer, and an experimentation or statistics reviewer remains recommended before commercial publication.

Scope and evidence labels

The field moves faster than ordinary software infrastructure. The guide uses five evidence labels:

- **Established** - ordinary software-engineering or statistical knowledge with durable empirical support, such as idempotency, least privilege, transaction semantics, confidence intervals, or proper scoring rules.
- **Strong production evidence** - repeatedly documented by multiple mature engineering organisations or platforms, but not necessarily formalised as a standard.
- **Emerging evidence** - supported by recent benchmarks, experiments, or early production reports whose external validity remains uncertain.
- **Engineering hypothesis** - a reasoned design position that must be tested against the local workload.
- **Open question** - a problem for which current systems do not have a reliable general solution.

A chapter-level evidence note describes the dominant basis of that chapter; it does **not** certify every local claim at the same strength. Vendor documentation is used to describe vendor capabilities, not to prove universal superiority. Benchmark results measure a complete system - model, harness, tools, environment, budget, and grader - rather than an intrinsic model property.

Code authenticity labels

Every substantial code block uses one of these labels:

- **Tested example - SDK version X.Y.Z:** executed in dependency-enabled CI against the stated package, runtime and feature flags. A test command and environment manifest must accompany the block.
- **Source-contract checked - version or commit pinned; not runtime-executed:** syntax-checked and compared with exact versioned source definitions, but not executed with the provider dependency and credentials.
- **Illustrative API mapping - not executable:** API-shaped design guidance whose omitted application services prevent direct execution.
- **Framework-neutral pseudocode:** architecture logic independent of a provider SDK.

Pattern contract

Every major named architecture pattern is specified with: use when, avoid when, mechanism, invariants, guardrails, failure modes, observability, evaluation, framework mapping, and competing alternatives. Compact reliability catalogues may use a table, but must cover the same concerns.

Companion reproducibility package

This edition is distributed with and embeds the following companion archive:

- **Filename:** `Production_Agent_Engineering_2026_Edition_1.7_Reproducibility_Package.zip`

- **Integrity:** member hashes are recorded inside the archive; the release-level archive hash and final PDF hash are recorded in the separately distributed build receipt. The receipt is intentionally external because embedding the final PDF hash inside the PDF would create a circular dependency.
- **PDF access:** open the document's attachment pane or select the visible paperclip annotation placed beside this heading on the rendered companion-package page.

The archive contains the complete synthetic evaluation data, executable analysis, expected results, a content-addressed Python environment manifest, exact PDF build and post-processing scripts, a deterministic source-to-PDF normalised-text verifier, TOC/bookmark verification, a machine-readable extraction of every manuscript code block and authenticity label, the canonical Markdown source, deterministic code-block extractor, complete deferred-approval listing, source-contract manifest and verifier, PDF text-layer checks, and member checksums. Publication checks fail when the canonical manuscript, PDF, or any required verification input is absent; no manuscript or PDF binding check is silently skipped.

Executive summary

A production agent that depends on remote models, tools, or durable state must be engineered with distributed-systems discipline. A fully local, single-process agent is not literally distributed, but it still contains a probabilistic decision component. The model is only one subsystem. The surrounding harness defines authority, context, state, tools, approvals, retries, verification, observability, and evaluation. When those concerns are left to natural-language instructions, the system has converted ordinary engineering invariants into suggestions.

The central design rule is:

> **Use models for semantic judgement under ambiguity. Use code-owned orchestration for invariants, authority, permitted transitions, side effects, accounting, and verification. The graph definition and legal transition relation can be deterministic; the realised path may still vary when routing depends on model or external outputs.**

That rule does not confine the model to classification. A capable model may interpret a messy request, construct a plan, choose among allowed tools, diagnose failures, and decide which evidence is relevant. The system should compile those choices into an execution surface whose legal transitions and consequences are defined by code.

A common production architecture uses the following loop:

Framework-neutral pseudocode.

```
receive task
  -> validate task contract
  -> construct trusted context
  -> ask model for a typed proposal
  -> validate proposal and authority
  -> execute one bounded action
  -> record state and evidence
  -> verify the external result
  -> continue, escalate, or stop
```

The most important consequences are:

1. **Agency is a budget, not a virtue.** Start with the least autonomous architecture that can solve the task. A code-owned workflow containing probabilistic model nodes is usually safer and easier to evaluate than a free-running loop.

2. **Context is a control plane.** The context builder decides what the model can know, how trusted each item is, and which capabilities are disclosed. Context assembly must be versioned, observable, and tested.
3. **State is not memory.** State is required for correctness and recovery. Memory is optional information that may improve usefulness. Never store an invariant only in model-facing memory.
4. **Access to tools conveys authority.** Effective authority is determined by the tool, credentials, enforcing gateway policy, and reachable resources together. A description or schema is not an access-control mechanism. Every side-effecting tool needs typed arguments, scoped identity, deterministic policy checks, idempotency, and audit records.
5. **Completion requires independent evidence.** The model saying "done" is not a completion condition. A command being accepted is not a completion condition. The intended external state must be read back or independently verified.
6. **Guardrails are lifecycle controls.** Effective controls exist before inference, during context construction, before and after tool execution, before state transitions, before external side effects, before final output, and after execution through monitoring.
7. **Durability changes the programming model.** Long-running work needs explicit states, replay-safe orchestration, retries by error class, cancellation, deadlines, compensation, version migration, and duplicate-safe side effects.
8. **Evaluation is a release discipline.** One successful run proves little. Measure task distributions, repeated-run reliability, verified completion, safe completion, trajectory quality, cost, latency, and grader quality. A worked evaluation in Part 15 shows the complete calculation.
9. **Confidence is not authority.** Model-reported confidence is often miscalibrated. Escalation and stopping thresholds must be fitted from empirical risk, loss, and information value rather than accepted from prose.
10. **Frameworks are implementation choices, not architecture.** OpenAI Agents SDK, Claude Agent SDK, Google ADK, LangGraph, Dapr, Temporal, and similar systems expose useful primitives, but none defines your business invariants, authority model, durable state, or acceptance criteria [R1-R25].

The result should look less like a chatbot with plugins and more like a workflow runtime with a semantic planner inside.

Part 1 - Engineering stance, vocabulary, and system boundary

Evidence status: Established for the software-engineering principles; strong production evidence for the agent-specific decomposition.

1.1 Agent, model, harness, workflow, and tool

An **agent** is a system that repeatedly observes task state, chooses an action, receives the result, and decides what to do next. The action may be a tool call, a handoff, a request for approval, a model call, or termination.

A **model** is the probabilistic component that maps context to a response or action proposal. Treat it as a fallible decision service. The model may be extremely capable, but its output is not a transaction, permission check, or proof.

A **harness** is the runtime around the model. It constructs context, invokes the model, validates outputs, dispatches tools, persists state, applies policy, records traces, and decides whether the loop may continue. Anthropic explicitly uses this term for the scaffold around

an agent, while OpenAI and Google expose similar concerns through their SDK primitives [R1-R13].

A **workflow** is an explicit control graph whose legal transition relation is owned by code. Model calls can be nodes in a workflow. The graph definition and permitted transitions remain code-owned; the realised path may still be probabilistic when routing depends on model output or external operations.

A **tool** is a capability interface. It may expose a function, API, command, browser, database query, file operation, or another agent. Access to it conveys authority only in combination with credentials, gateway policy, and reachable resources. A tool schema describes how to call the capability; it does not decide whether the call is authorised.

1.2 The correct system boundary

The model boundary must not become the system boundary. The model should not own:

- credential selection;
- permission evaluation;
- legal state transitions;
- retry accounting;
- durable task identity;
- concurrency control;
- billing limits;
- irreversible commit decisions;
- audit records;
- the final determination that the external objective was satisfied.

The model may propose values for these operations, but deterministic components must validate and commit them.

A practical seven-plane decomposition is:

Framework-neutral pseudocode.

```
+-------------------------------------------------------------+
| Control plane: task lifecycle, routing, budgets, scheduling |
+-------------------------------------------------------------+
| Context plane: prompt assembly, retrieval, trust labels     |
+-------------------------------------------------------------+
| Decision plane: model calls, planning, semantic judgement   |
+-------------------------------------------------------------+
| Execution plane: tools, sandboxes, external systems         |
+-------------------------------------------------------------+
| State plane: event history, checkpoints, memory, artifacts  |
+-------------------------------------------------------------+
| Policy plane: identity, authorization, guardrails, approval |
+-------------------------------------------------------------+
| Verification/observability plane: graders, traces, evidence |
+-------------------------------------------------------------+
```

This is an engineering model, not an industry standard. Its purpose is to prevent a common failure: hiding identity, context, execution, and policy inside one opaque “agent” object.

1.3 Capability is not authority

Capability means the system can technically perform an action. **Authority** means the action is permitted for this task, user, resource, time, and data provenance.

A browser agent may be capable of clicking “Send”, but it may lack authority to send to an external recipient. A coding agent may be capable of running `terraform apply`, but it may lack authority to change production. A data agent may be capable of querying a warehouse, but it may lack authority to export customer-level rows.

The authority check belongs outside the model because a compromised or confused model cannot be trusted to police its own permissions.

Pattern: capability-authority split

Use when. The planner needs broad semantic choice, but real actions must be constrained by user, tenant, resource, data provenance, time, risk, or approval state.

Avoid when. A single deterministic service operation already expresses the entire authorised workflow. Adding a model-visible capability catalogue and policy gateway to a trivial CRUD path only creates latency and another failure surface.

Mechanism. The model sees capability descriptors and emits a typed action proposal. A deterministic policy decision point resolves identity, resource scope, risk class, provenance, approval status, and current task state. Only an execution broker can invoke the real tool, and it receives a credential scoped to the approved action rather than the model process’s ambient credentials.

Framework-neutral pseudocode.

```
proposal = model.propose_action(context)
validated = ActionProposal.model_validate(proposal)
decision = policy.evaluate(
    actor=task.actor,
    action=validated.action,
    resource=validated.resource,
    provenance=context.provenance,
    task_state=state.snapshot(),
)
if decision.effect == "deny":
    return record_denial(decision)
if decision.effect == "require_approval":
    return suspend_for_approval(decision)
return broker.execute(validated, decision.scoped_credential)
```

Invariants. No side effect occurs solely because the model requested it. The execution broker cannot mint broader authority than the policy decision granted. Resource identity is canonicalised before authorisation. Approval applies to an immutable proposal hash, not to a later mutable prompt.

Guardrails. Filter capability discovery by actor and task before disclosure; validate arguments against current authoritative state; deny ambiguous destinations; apply egress and data-classification policy; require read-back verification for writes; expire scoped credentials quickly.

Failure modes. A policy layer that merely asks another model “is this safe?” moves the same problem sideways. Broad broker credentials defeat the split. Aliases can bypass policy if different names reach the same backend. A race between approval and execution can invalidate the approved state.

Observability. Record the proposed action, canonical resource, actor identity, policy inputs, decision reason, policy version, approval hash, scoped credential identifier, execution result, and postcondition result. Alert on denied-action retries, alias mismatches, and scope expansion.

Evaluation. Generate adversarial proposals that vary user, tenant, resource, data classification, destination, timing, stale state, and persuasive justification. Property-test that equivalent resource aliases produce identical decisions. Inject approval replay and state-change races.

Framework mapping. OpenAI tool guardrails and approval-capable tools, Claude Agent SDK permission callbacks and `PreToolUse` hooks, and Google ADK before-tool callbacks can intercept execution [R3, R4, R8, R12, R15]. None of them defines the organisation's resource canonicalisation, identity delegation, policy language, credential minting, or compare-and-set semantics; those remain application responsibilities.

Competing alternatives. A fixed code-owned workflow is preferable when the action sequence is known. A read-only agent can remove most authority complexity. For very high-risk work, separate proposer, approver, and executor services rather than relying on one broker.

1.4 Invariants, policies, and preferences

An **invariant** must always hold: an invoice cannot be paid twice; a task cannot transition from `SUCCEEDED` back to `RUNNING`; a tool may not access a resource outside its tenant.

A **policy** is an enforced rule that may depend on context: external email requires approval above a sensitivity threshold.

A **preference** is soft guidance: prefer concise status updates.

Do not encode invariants as prompt prose. Prompts are suitable for preferences and for helping the model reason about policy, but enforcement belongs in code.

1.5 Core principles

1. Make every irreversible boundary explicit.
2. Prefer one bounded action per loop iteration.
3. Persist state before acknowledging progress.
4. Treat external content as untrusted data, not instructions.
5. Give each tool the minimum identity and resource scope it needs.
6. Separate proposed state from committed state.
7. Record evidence for every completion claim.
8. Test the complete system, not the model in isolation.
9. Version prompts, tools, policy, state schemas, and graders together.
10. Design failure and cancellation paths before adding autonomy.

Part 2 - Agency gradient and architecture selection

Evidence status: Strong production evidence; local thresholds are engineering hypotheses and must be evaluated.

2.1 Agency is a design variable

Teams often ask whether a system "is an agent". The useful question is how much discretion the model has over sequence, tool choice, scope, stopping, and side effects.

Use an **agency gradient**:

Level	Model discretion	Typical architecture	Default risk
0	None	deterministic code	low
1	Produces content or classification	single model call with schema	low
2	Chooses one tool from a bounded set	tool-calling micro-agent	medium
3	Chooses sequence inside a fixed workflow	workflow with model nodes	medium
4	Builds and executes a bounded plan	plan-compile-execute	medium-high
5	Open-ended loop across broad tools	autonomous harness	high
6	Multiple autonomous agents coordinate	distributed agent system	very high

Higher levels are not inherently better. They increase the search space, context demand, failure surface, and cost of evaluation.

2.2 Classify the task before selecting the architecture

Evaluate at least these dimensions:

- **Input ambiguity.** Is the request structured, or must meaning be inferred?
- **Path multiplicity.** Is there one known procedure or many valid strategies?
- **Environmental uncertainty.** Can tools return unexpected states?
- **Side-effect severity.** What is the cost of a wrong action?
- **Reversibility.** Can the action be undone reliably?
- **Duration.** Can the task outlive a process, deployment, or context window?
- **Verification quality.** Can success be checked deterministically?
- **Latency tolerance.** Is iterative reasoning acceptable?
- **Volume.** Will the architecture run thousands or millions of times?
- **Adversarial exposure.** Will it read untrusted web pages, emails, code, or documents?

A task with ambiguous input but deterministic execution should not become a free-running agent. Use a model to interpret the input, validate the structured result, and pass it to normal code.

2.3 Architecture decision rules

Use deterministic code when the task has a known algorithm, fixed state transitions, a compliance requirement, or a single correct operation.

Use a single structured model call when one semantic judgement is needed and the result can be validated.

Use a workflow with model nodes when the overall process is known but some steps require interpretation, ranking, drafting, or diagnosis.

Use a bounded agent loop when the environment cannot be predicted and the model must inspect results before choosing the next step.

Use multiple agents only when independent specialisation, parallel search, or trust separation produces a measured benefit that exceeds coordination cost.

Framework-neutral pseudocode.

```
def choose_architecture(task):
    if task.has_known_algorithm and not task.requires_semantic_judgement:
        return "deterministic"
    if task.semantic_decisions == 1 and task.output_is_validatable:
        return "single_structured_call"
    if task.process_is_known:
        return "workflow_with_model_nodes"
    if task.environment_is_interactive and task.can_be_bounded:
        return "bounded_agent_loop"
    if task.decomposes_into_independent_specialists and task.benefit_is_measured
    :
        return "multi_agent"
    return "human_led_workflow"
```

2.4 The agency budget

An **agency budget** limits how much exploratory behaviour a run may consume. It is broader than a token limit.

A useful budget contains:

- maximum model calls;
- maximum tool calls by risk class;
- maximum wall-clock time;
- maximum external writes;
- maximum spend;
- maximum retries per error class;
- maximum number of distinct resources touched;
- maximum delegation depth;
- maximum unverified state transitions.

When a budget is exhausted, the system should not merely stop with “failed”. It should produce a structured partial-completion record containing completed work, verified evidence, unresolved items, and the reason for stopping.

2.5 Anti-patterns

The prompt workflow

A long prompt describes a 12-step business process and asks the model to follow it. There is no explicit state machine, so retries, skips, and partial failures are invisible.

Replacement: encode the process in code and invoke models only at semantic steps.

The universal tool belt

Every tool is exposed to every task. The model spends context on irrelevant schemas and has excessive authority.

Replacement: disclose tools progressively by task stage, identity, and risk.

The manager-agent org chart

A manager model delegates to multiple agents because the architecture sounds organisationally natural.

Replacement: add an agent only when the work is separable and an evaluation shows higher quality, lower latency, lower cost, or stronger trust separation.

Autonomous retries

The model retries until something works, without classifying the error.

Replacement: deterministic retry policies distinguish transient, permanent, conflict, policy, and validation failures.

2.6 Architecture review questions

Before approving an agentic design, ask:

1. Which decisions actually require semantic judgement?
2. Why cannot those decisions be isolated as model nodes in a workflow?
3. What does the model control that code could control more safely?
4. What is the maximum blast radius of one wrong tool call?
5. Can success be verified without trusting the same model that acted?
6. What happens after process death, timeout, cancellation, or deployment?
7. Which measured result justifies each additional agent or loop?

Part 3 - Deterministic task architecture

Evidence note. The transaction, validation, and workflow foundations are established; their precise composition around current model behaviour is an engineering design that must be evaluated locally.

3.1 Deterministic and probabilistic boundaries

A component is **deterministic** when the same state and input produce the same allowed transition or output. A language model is probabilistic even when temperature is zero: provider changes, numerical execution, context variation, and model updates can alter behaviour.

The goal is not to eliminate probabilistic reasoning. It is to place it where variation is acceptable and observable.

Use models for:

- interpreting natural language;
- extracting candidate entities from messy input;
- ranking alternatives;
- proposing plans;
- diagnosing unfamiliar failures;
- generating text or code under a contract;
- deciding which evidence is semantically relevant.

Use code for:

- schema validation;
- access control;
- state transitions;
- arithmetic and accounting;
- retries and timeouts;
- concurrency control;
- transaction commit;
- idempotency;
- audit logging;
- deterministic acceptance checks.

3.2 Pattern 1: code-owned workflow with probabilistic nodes

Use when. The business process and legal transitions are known, but one or more bounded steps require interpretation, ranking, diagnosis, or generation.

Avoid when. The task is genuinely open-ended and cannot be represented without constantly changing the workflow graph. Also avoid it when a single deterministic parser or classifier already meets quality requirements.

Framework-neutral pseudocode.

```
VALIDATE_REQUEST
  -> INTERPRET_INTENT (model)
  -> VALIDATE_INTERPRETATION
  -> FETCH_AUTHORITATIVE_STATE
  -> PROPOSE_OPERATION (model)
  -> CHECK_POLICY_AND_PRECONDITIONS
  -> EXECUTE_TRANSACTION
  -> VERIFY_POSTCONDITIONS
  -> COMPLETE
```

Mechanism. Every node has typed input, typed output, timeout, retry class, state owner, and completion condition. Model nodes return proposals or semantic judgements; they never commit side effects. Deterministic nodes own legal transitions and persistence.

Framework-neutral pseudocode.

```
intent = IntentSchema.model_validate(interpret_model(request_text))
account = account_store.read(intent.account_id)
operation = OperationSchema.model_validate(
    proposal_model(intent=intent, account=account.redacted_view())
)
policy.require_allowed(actor, operation, account)
result = transaction_service.execute(operation, idempotency_key=task.id)
verification.require_matches(result, account_store.read(intent.account_id))
```

Invariants. The graph defines all legal transitions. Model output cannot skip validation, approval, transaction, or verification. State is persisted before acknowledging node completion. Retries are duplicate-safe.

Guardrails. Constrain model-node schemas; disclose only tools required by the node; cap hidden internal iterations; validate current state immediately before side effects; require a deterministic completion gate after writes.

Failure modes. Hidden loops inside a model node can reintroduce uncontrolled agency. Syntax-only validation can accept semantically dangerous values. The graph can become brittle if every novel state terminates instead of entering a bounded diagnosis or escalation branch.

Observability. Emit node-entered, model-invoked, schema-rejected, policy-decided, action-committed, postcondition-checked, and node-completed events. Track time and cost by node, not only by task.

Evaluation. Test model nodes independently, then execute trajectory tests for malformed outputs, stale state, conflicts, process death, duplicate delivery, approval suspension, policy denial, and verifier failure.

Framework mapping. Google ADK's explicit workflow and graph agents map directly to explicit node order [R15, R16]. OpenAI Agents SDK and Claude Agent SDK can implement model nodes and tool boundaries, but the surrounding workflow graph is application code or an external workflow engine [R1-R8]. Temporal, Durable Task, Dapr Workflow, or LangGraph can own durable transitions [R20-R25].

Competing alternatives. Use plan-compile-execute when the sequence varies substantially but the instruction set is fixed. Use propose-check-commit for one atomic mutation. Use a bounded free loop only when the state space is too open for a useful graph and risk is controlled.

3.3 Pattern 2: plan-compile-execute

Use when. The task allows many valid sequences, but every executable step belongs to a controlled instruction set and can be statically checked before execution.

Avoid when. The environment changes so quickly that a long compiled plan will be stale before execution, or when each next action depends on observations that cannot be predicted. In those cases compile short plan fragments or use a bounded loop.

The model produces a **declarative plan**: desired operations and dependencies without direct execution. A compiler validates the plan, resolves references, inserts mandatory checks, estimates scope and cost, and emits an executable graph.

Framework-neutral pseudocode.

```
{
  "goal": "publish_release_notes",
  "steps": [
    {"id": "s1", "op": "read_git_range", "args": {"from": "v2.4", "to": "HEAD
    "}},
    {"id": "s2", "op": "draft_notes", "depends_on": ["s1"]},
    {"id": "s3", "op": "request_approval", "depends_on": ["s2"]},
    {"id": "s4", "op": "publish", "depends_on": ["s3"]}
  ]
}
```

Mechanism. The compiler accepts only named operations, validates types and dependencies, expands resource selectors, inserts approval and verification nodes, rejects unbounded loops, and produces a versioned graph.

Framework-neutral pseudocode.

```
plan = PlanSchema.model_validate(model.generate_plan(task))
graph = compiler.compile(plan, policy=task.policy)
assert graph.is_acyclic()
assert graph.has_required_gate("publish", "approval")
assert graph.has_verifier_after_every_write()
workflow.start(graph)
```

Invariants. The model defines intent and ordering only inside the instruction set. The compiler owns legality. The compiled graph is immutable for approval and audit. Any runtime re-plan produces a new graph version.

Guardrails. Deny arbitrary shell or SQL strings unless they enter a separate sandboxed compiler; bound fan-out and resource expansion; perform static data-flow checks; estimate worst-case cost; attach approval to the compiled plan hash; verify each mutation.

Failure modes. A compiler that accepts arbitrary commands is a string passthrough. Broad selectors such as `all_repositories` hide excessive scope. Static plans can become stale. A compiler may validate syntax while missing dangerous semantic composition across steps.

Observability. Persist source plan, compiler diagnostics, expanded resources, inserted gates, cost estimate, graph hash, graph version, and runtime deviations. Measure rejection reasons and re-plan frequency.

Evaluation. Property-test the compiler with random and adversarial plans. Verify illegal sequences are unrepresentable. Run mutation tests that remove mandatory approval or verification nodes and confirm compilation fails. Evaluate stale-plan detection under concurrent state changes.

Framework mapping. Google ADK graph workflows and custom agents can host the compiled graph [R15, R16]. LangGraph can execute a compiled state graph with checkpoints and interrupts [R64]. OpenAI and Claude SDKs can implement the planner and individual nodes but do not provide a business-specific plan compiler. Temporal and Durable Task provide durable graph execution but not model-plan validation [R20-R23].

Competing alternatives. A code-owned workflow with probabilistic model nodes is simpler for stable sequences. Propose-check-commit is safer for a single mutation. Receding-horizon planning compiles only the next few steps when the environment is dynamic.

3.4 Pattern 3: propose-check-commit

Use when. A model proposes one state mutation and the system can atomically validate the expected state before committing it.

Avoid when. The action spans multiple systems without a reliable compensation path, the external system exposes no revision identity, or the mutation cannot be represented and reviewed independently of execution.

The proposal contains target resource, expected version or hash, intended change, justification, evidence references, risk classification, and requested authority.

Framework-neutral pseudocode.

```
proposal = PatchSchema.model_validate(model.propose_patch(snapshot))
current = store.read(proposal.resource_id)
if current.version != proposal.expected_version:
    raise Conflict("state changed; re-plan")
policy.require_allowed(actor, proposal)
new_state = apply_patch(current, proposal.patch)
invariants.check(new_state)
store.compare_and_set(current.version, new_state)
verification.read_back_and_compare(proposal.resource_id, new_state)
```

A **compare-and-set** commits only when the resource still has the expected version. This prevents a proposal based on stale state from overwriting concurrent changes.

Mechanism. The model produces a canonical mutation object. Deterministic code reads current state, checks the expected revision, validates policy and invariants, commits atomically, and reads back the result.

Invariants. The model cannot commit against unobserved state. Approval and commit refer to the same proposal hash. Protected fields cannot change unless explicitly authorised. Duplicate commits are idempotent.

Guardrails. Canonicalise patches; show human-readable and machine-readable diffs; reject broad or implicit selectors; require stronger approval for protected fields; bind the commit to a short freshness window; verify through an authoritative read path.

Failure modes. Application-level hashes can miss hidden state. Semantic effects may depend on triggers, defaults, or downstream systems. A successful compare-and-set can still create an invalid cross-resource state if invariants are incomplete.

Observability. Record before/after revisions, proposal hash, changed fields, policy decision, approval identity, conflict count, commit identifier, read-back result, and compensation status.

Evaluation. Inject concurrent updates, duplicate delivery, stale approvals, protected-field changes, hidden defaults, trigger side effects, and failed read-back. Verify no lost updates and no unapproved mutation.

Framework mapping. Any agent SDK can generate the proposal. The compare-and-set, transaction, policy, and verification layers belong to the application's storage and execution services. Tool hooks can block execution but cannot invent missing database concurrency semantics [R3, R8, R15].

Competing alternatives. Use a deterministic form or command handler when the mutation can be derived without a model. Use a saga for multi-system work. Use plan-compile-execute when the mutation is only one step in a larger variable sequence.

3.5 Preconditions, postconditions, and transactions

A **precondition** must hold before an action. A **postcondition** must hold after it. For a CRM update:

- precondition: record exists, actor has write scope, current owner matches expected owner;
- action: update status;
- postcondition: authoritative read shows new status and unchanged protected fields.

Side effects should be transactional where possible. Where a transaction cannot span systems, use a saga: a sequence of local commits with compensating actions. Compensation is not rollback magic. It is a new action that attempts to restore an acceptable business state.

3.6 Error taxonomy and deterministic retry

Do not let the model decide whether every error deserves a retry.

Error class	Example	Default response
Transient	timeout, rate limit	bounded exponential backoff
Conflict	version changed	refresh state and re-plan
Validation	schema or invariant failure	return structured correction request
Policy	missing authority	deny or request approval
Permanent	resource deleted	stop with partial result
Unknown	unclassified exception	quarantine, capture trace, limited diagnostic branch

Every retry must reuse an idempotency key or otherwise guarantee duplicate safety.

3.7 Deterministic fallback

Fallback should be specified before deployment:

- use a simpler deterministic path;
- request user clarification;
- escalate to a stronger model;
- request human approval;
- produce a partial result;
- terminate safely.

"Try harder" is not a fallback policy.

Part 4 - Harness and control-plane architecture

Evidence note. Multiple current SDKs expose comparable loop, hook, permission, session, and trace primitives [R1-R16]. Their business-level correctness and durability remain application-specific.

Failure vignette. A browser client disconnects after a side-effecting tool returns but before the UI receives the event. The client reconnects, starts a fresh loop, and the model repeats the write because task identity, commit status, and replay semantics lived in the client rather than one authoritative harness. The visible failure is a duplicate action; the architectural failure is split ownership of lifecycle state.

4.1 What the harness owns

A production harness should make the following responsibilities explicit:

1. task admission and contract validation;
2. context construction and provenance labelling;
3. model selection and invocation;
4. output parsing and schema validation;
5. policy evaluation;
6. tool dispatch and sandbox management;
7. durable state persistence;
8. retry, timeout, cancellation, and budget enforcement;
9. approval suspension and resumption;
10. verification and completion;
11. tracing, metrics, and audit logging;
12. version compatibility.

If a framework supplies these features, the application must still define their semantics.

4.2 Pattern: bounded agent loop

Use when. The next useful action depends on observations and cannot be fully planned in advance, while each action can be individually constrained and verified.

Avoid when. A code-owned workflow or compiled plan can represent the task. Avoid a free loop for irreversible high-risk operations unless an external workflow, approval, and transaction layer surrounds it.

A useful base loop is gather, propose, validate, act, observe, verify.

Framework-neutral pseudocode.

```
while state.status == "RUNNING":
    budgets.require_remaining(state.usage)
    context = context_builder.build(task, state)
    proposal = model_router.call(context, contract=ActionProposal)
    action = validators.validate(proposal, state)
    decision = policy.evaluate(task, state, action, context.provenance)

    if decision.requires_approval:
        state = suspend(state, decision)
        break
    if decision.denied:
        state = record_denial_and_replan_or_stop(state, decision)
        continue

    outcome = tool_broker.execute(action, decision.credential)
```

```
    state = event_store.append_and_reduce(state, outcome)
    verification = verifier.check(task, state, outcome)
    state = completion_policy.advance(state, verification)
```

Mechanism. Each iteration constructs a stage-specific context, requests one typed proposal, validates policy and state, executes at most one consequential action, persists the outcome, and evaluates progress. The loop terminates through a completion gate, escalation, cancellation, deadline, or budget tripwire.

Invariants. Every action is authorised against current state. State recording is atomic with or precedes acknowledgement. One iteration cannot silently execute a chain of unobserved side effects. Completion depends on evidence rather than model declaration.

Guardrails. Limit steps, cost, wall-clock time, repeated action signatures, and authority. Separate read and write phases. Require approval for risk-class transitions. Quarantine unknown errors. Prevent the same model call from both acting and approving.

Failure modes. The loop can oscillate, retry permanent errors, consume budget without evidence gain, or compress away a critical constraint. Hidden multi-action tools defeat the one-action boundary. A verifier that uses the same narrative can rubber-stamp failure.

Observability. Emit one trace span per iteration with context manifest, disclosed tools, proposal, validation result, policy decision, tool outcome, state delta, evidence delta, and stop decision. Track progress-per-call, repeated-action rate, and verification failures.

Evaluation. Include loop-inducing tasks, impossible tasks, ambiguous tasks, stale observations, repeated transient and permanent errors, context truncation, budget exhaustion, and approval suspension. Measure successful stop, safe stop, and needless-loop rate.

Framework mapping. OpenAI's runner can execute tool turns and expose tracing and guardrails [R1-R5]. Claude Agent SDK provides a persistent loop, permissions, hooks, interrupts, checkpointing, and telemetry [R8, R12]. Google ADK provides model agents, callbacks, sessions, and custom control flow [R15, R16]. These SDKs do not by themselves guarantee durable business state, idempotent side effects, transactional persistence, or application-specific stop rules. Use a durable workflow engine when process survival matters.

Competing alternatives. A code-owned workflow with probabilistic model nodes is easier to reason about. Plan-compile-execute is better when the action language is known. A single tool-calling request is sufficient for one bounded decision.

4.3 Harness state

The harness needs a state object independent of transcript text. A minimal form:

Framework-neutral pseudocode.

```
{
  "task_id": "tsk_...",
  "status": "RUNNING",
  "objective": {...},
  "current_stage": "VERIFY_PATCH",
  "workflow_version": "2026-07-28.3",
  "model_policy_version": "router-17",
  "prompt_bundle_version": "repo-agent-42",
  "tool_catalog_version": "tools-31",
  "attempts": {"model": 7, "tool": 11, "retry": 1},
  "budgets": {"cost_remaining": 2.80, "tool_calls_remaining": 9},
  "resources": {"workspace_id": "ws_...", "repo_revision": "abc123"},
  "approvals": [],
  "evidence": [],
```

```
  "last_error": null
}
```

Transcript messages can be regenerated or compressed. The state object is authoritative.

4.4 Hooks and interceptors

A **hook** is deterministic code invoked at a lifecycle boundary. Useful boundaries include:

- before context assembly;
- before model invocation;
- after model output but before persistence;
- before tool selection is exposed;
- before tool execution;
- after tool execution;
- before state transition;
- before completion;
- after run termination.

Hooks are suitable for policy checks, redaction, argument rewriting, caching, tracing, and approval. OpenAI guardrails, Anthropic hooks and permission policies, Google ADK plugins, and Dapr Agent hooks illustrate this design direction [R3-R13, R25].

Do not use hooks to hide core workflow logic. If every step depends on invisible interceptors, the execution graph becomes difficult to reason about.

4.5 Initialiser-worker-verifier harness

Long-running work benefits from role separation:

- **Initialiser:** validates environment, creates task manifest, records baseline, and decomposes acceptance criteria.
- **Worker:** advances one bounded unit and leaves durable artifacts.
- **Verifier:** independently checks the resulting state.

Anthropic's long-running agent work reports that structured progress files, feature lists, setup scripts, and clean handoff artifacts reduce the tendency to lose state or declare premature completion [R10, R11].

The roles may use the same model. Separation is architectural: they have different prompts, tools, authority, and acceptance criteria.

4.6 Workspace and sandbox design

A sandbox is an isolated execution environment for code, files, browsers, or tools. It should define:

- filesystem scope;
- network egress policy;
- CPU, memory, and time limits;
- secret injection rules;
- process limits;
- artifact export rules;
- snapshot and reset behaviour;
- audit visibility.

The sandbox must be disposable. Persist only explicit artifacts and state, not accidental process memory.

4.7 Harness failure modes

- **Transcript-as-state:** context truncation silently removes required facts.
- **Hidden authority:** tools inherit the host process's broad credentials.
- **Unbounded loop:** the agent consumes budget without increasing verified progress.
- **Self-certification:** the actor marks success without independent evidence.
- **Non-atomic persistence:** a tool succeeds but state recording fails, causing duplicate execution.
- **Version drift:** an in-flight run resumes under incompatible prompts, tools, or workflow code.
- **Observability gaps:** tool arguments or policy decisions are not recorded.

4.8 Harness acceptance tests

A harness is not production-ready until tests demonstrate:

1. recovery after process termination at every lifecycle boundary;
2. duplicate delivery without duplicate side effects;
3. cancellation while waiting for a model, tool, approval, or timer;
4. policy re-evaluation after resumption;
5. budget enforcement under loops and retries;
6. deterministic reconstruction of task state;
7. safe behaviour when traces, memory, or a tool backend are unavailable;
8. migration of in-flight tasks across a version change.

Part 5 - Task contracts, prompt engineering, and specification pipelines

Evidence status: Strong production evidence for contracts and versioning; prompt-specific details are workload-dependent and require ablation.

5.1 Prompt engineering starts after architecture

A prompt cannot compensate for missing state, authority, or verification. Once those are externalised, prompt engineering becomes a tractable specification problem: tell the model what judgement it owns, what evidence it receives, which actions it may propose, and how it should represent uncertainty.

5.2 The task contract

A **task contract** is the machine-readable agreement between the caller and the agent system. It should contain:

- objective;
- actor and delegated authority;
- resource scope;
- constraints and non-goals;
- inputs and their trust level;
- required evidence;
- permitted action classes;
- approval rules;
- completion conditions;
- partial-completion format;

- escalation conditions;
- cost, latency, and time limits;
- output schema.

Framework-neutral pseudocode.

```
objective: "Verify Jira ticket QA-1842 against staging"
actor:
  user_id: "u-17"
  authority: "qa-verification"
resources:
  allowed_hosts: ["staging.example.internal"]
  allowed_ticket: "QA-1842"
non_goals:
  - "Do not change application data except test fixtures"
  - "Do not comment on unrelated tickets"
evidence_required:
  - "screenshots for visible defects"
  - "request/response IDs for API failures"
completion:
  - "every acceptance criterion classified pass/fail/blocked"
  - "all findings linked to evidence"
escalation:
  - "credentials fail twice"
  - "test requires production access"
budgets:
  max_minutes: 30
  max_tool_calls: 80
```

The prompt is derived from the contract; the contract is not reconstructed from the prompt.

5.3 Authority, goals, constraints, and non-goals

Prompts often mix these concepts. Keep them separate:

- **Authority:** what the model may propose or access.
- **Goal:** the intended outcome.
- **Constraints:** conditions that must hold during execution.
- **Non-goals:** attractive but out-of-scope work.
- **Completion conditions:** evidence required to stop successfully.
- **Escalation conditions:** states where autonomous progress must stop.

Explicit non-goals reduce scope expansion. Explicit completion conditions reduce premature success.

5.4 Prompt assembly pipeline

Build prompts from versioned sections rather than one hand-edited string:

Framework-neutral pseudocode.

```
1. platform policy and immutable safety rules
2. application role and authority summary
3. task contract
4. current workflow stage
5. authoritative state snapshot
6. retrieved evidence with provenance labels
7. disclosed tool schemas
8. output schema
9. concise examples for ambiguous behaviour
```

Framework-neutral pseudocode.

```
def build_prompt(bundle, task, state, evidence, tools):
    return render([
        bundle.platform_policy,
        bundle.role_policy(task.actor),
        render_task_contract(task),
        render_stage(state.current_stage),
        render_state_snapshot(state),
        render_evidence(evidence, include_provenance=True),
        render_tools(tools),
        bundle.output_contract,
        bundle.examples.for_stage(state.current_stage),
    ])
```

Every section should have a source and version. Record the final prompt hash in the trace.

5.5 Instruction precedence and provenance

A model may receive instructions from system policy, the user, retrieved documents, tool output, and prior messages. The system must distinguish **instructions** from **data**.

Use explicit trust labels:

Framework-neutral pseudocode.

```
TRUSTED_POLICY: application-controlled and immutable for the run
TRUSTED_TASK: authenticated user request within delegated authority
AUTHORITATIVE_STATE: current data from the system of record
UNTRUSTED_CONTENT: web pages, email bodies, tickets, documents, code comments
MODEL_GENERATED: prior plans, summaries, and hypotheses
```

Do not assume delimiters alone prevent prompt injection. Labels improve model behaviour and auditability, but deterministic execution policy remains necessary.

5.6 Static and dynamic prompt sections

Static sections change rarely: role, safety policy, output contract, general tool-use rules.

Dynamic sections change per turn: current objective, state, evidence, tool availability, budgets, unresolved questions.

Keep dynamic sections short and authoritative. Do not append the entire transcript when a compact state representation is available.

5.7 Examples as policy tests

Examples can define behaviour more reliably than abstract prose, but they also create accidental policy. Maintain examples as test cases:

- input situation;
- expected proposal;
- prohibited proposal;
- reason;
- applicable model families;
- regression owner.

An example that includes broad tool use may teach the model to overuse that tool. Remove examples that are not covered by evaluations.

5.8 Prompt versioning and compatibility

Version the complete prompt bundle, not only the main system string. A bundle includes:

- templates;
- examples;
- tool descriptions;
- output schemas;
- context selection rules;
- model-specific adapters.

In-flight tasks should record their bundle version. A resumed task either continues with the compatible version or passes through an explicit migration.

5.9 Prompt ablation

A **prompt ablation** removes or changes one component to measure its causal value. Useful ablations include:

- remove examples;
- remove explicit non-goals;
- replace verbose tool descriptions with concise contracts;
- move a rule from prompt to deterministic guard;
- vary evidence ordering;
- vary state representation;
- remove self-review instructions;
- add an independent verifier rather than more reflection.

Measure outcome, safe completion, cost, latency, and trajectory. Do not accept a prompt change because sample transcripts look better.

5.10 Completion and escalation contracts

The final response should be schema-constrained:

Framework-neutral pseudocode.

```
{
  "status": "SUCCEEDED | PARTIAL | BLOCKED | FAILED",
  "summary": "...",
  "verified_outcomes": [
    {"claim": "...", "evidence_ids": ["ev-12"], "verifier": "..."}
  ],
  "unresolved": ["..."],
  "risks": ["..."],
  "next_required_actor": "none | user | operator | security"
}
```

The harness, not the model, maps this report to the task's terminal state.

5.11 Prompt anti-patterns

- "Be careful" without defining prohibited effects.
- "Use your best judgement" where authority should be explicit.
- "Do not hallucinate" instead of requiring sources and verification.
- "Reflect until correct" without an external acceptance test.
- burying completion conditions in a long prose paragraph;
- including stale state from earlier turns;

- giving the model tools it cannot safely use;
- treating the system prompt as a security boundary.

Part 6 - Context engineering

Evidence note. Progressive disclosure and artifact-backed context have strong production support; ranking, compression, and optimal context policies remain workload-dependent.

Failure vignette. A repository agent receives a startup `git status`, an old design note, and a fresh file read. The stale startup snapshot appears earlier and with stronger wording, so the model plans against a branch state that no longer exists. No tool failed and no policy was violated; context construction silently supplied contradictory world models without freshness or authority labels.

6.1 Context is a computed view

The context window is not memory and should not be treated as a database. It is a computed, temporary view assembled for one model decision. A context builder chooses which facts, instructions, tools, artifacts, and history are relevant now.

A context item should have:

- content or a stable reference;
- source;
- creation time;
- freshness time or expiry;
- trust level;
- sensitivity label;
- task relevance score;
- owner;
- version;
- whether the model may quote, transform, or act on it.

This metadata lets policy reason about information flow instead of treating all tokens as equivalent.

6.2 Context budget

A context budget allocates scarce model attention across:

- policy;
- task objective;
- current state;
- domain evidence;
- tool schemas;
- examples;
- recent observations;
- unresolved hypotheses.

Long context can reduce quality when irrelevant, contradictory, or stale items obscure the current decision. The objective is not maximum recall. It is the minimum sufficient, trustworthy context for the next action.

6.3 Progressive disclosure

Progressive disclosure exposes detailed information only when the current stage requires it. Instead of placing 500 tool definitions in the initial prompt, expose a small capability catalogue and let the system load a relevant subset.

Framework-neutral pseudocode.

```
stage: TRIAGE
visible tools: search_ticket, read_ticket, classify_task

stage: INVESTIGATE_UI
visible tools: open_browser, screenshot, inspect_network

stage: REPORT
visible tools: attach_evidence, write_report, request_review
```

Anthropic has documented the cost of loading large tool catalogues and advocates on-demand loading and code execution for some workloads [R14]. OpenAI and Google similarly support filtered or dynamically selected tools [R4, R7, R12].

Pattern: capability search then disclosure

Use when. The available tool or knowledge surface is too large, expensive, or sensitive to expose in every model call.

Avoid when. The allowed tool set is already small and stable. A search layer over five tools adds ambiguity and can reduce selection reliability.

Mechanism. The model receives filtered capability families, proposes a semantic capability query, deterministic code resolves the authorised subset, and full schemas are disclosed only for the current stage.

1. The model receives concise capability families, not every schema.
2. It proposes a query such as `database.read.analytics`.
3. Deterministic code filters by actor, task, tenant, risk, and current stage.
4. The resolver returns a bounded set of full schemas.
5. The disclosed set and catalogue version are recorded.

Invariants. The search result cannot reveal or grant capabilities outside pre-filtered authority. Disclosure does not imply permission to execute. Tool identity remains stable across aliases and versions.

Guardrails. Filter before semantic search; hide sensitive capability existence; cap result count; separate read and write families; require exact schema selection before execution; reject ambiguous tool-name resolution.

Failure modes. Capability search can leak sensitive tool names, retrieve an over-broad write tool, or repeatedly miss the necessary tool. Embedding-based search can collapse distinct security semantics into one similarity result.

Observability. Record query, pre-filtered catalogue size, returned tools, scores, rejected tools, catalogue version, and later selection outcome. Monitor no-tool and wrong-tool rates by task class.

Evaluation. Create tasks whose required tools are semantically similar to forbidden tools. Measure recall of the required tool, precision of disclosed tools, leakage, token savings, and downstream success.

Framework mapping. OpenAI and Google support dynamically selected or filtered tools, while Claude Agent SDK and MCP clients can restrict tool exposure and permissions [R4,

R8, R13-R16]. The organisation must still implement security-aware indexing, catalogue governance, and stable capability identity.

Competing alternatives. Static stage-specific tool sets are safer when workflows are known. A deterministic router can select the tool family. Code execution over a narrow API may be more token-efficient than exposing hundreds of fine-grained tool schemas [R14].

6.4 Retrieval is not authority

Retrieved content can be relevant and still be untrusted. A document may be outdated, malicious, or written for a different environment. The context builder should separate:

- **authoritative facts:** current values from a system of record;
- **supporting evidence:** sources that inform judgement;
- **untrusted content:** text that may contain instructions or attacks;
- **model hypotheses:** generated interpretations that require verification.

A common mistake is to convert a retrieved page into plain text and then place it beside system instructions without provenance. That flattens data and authority into one sequence.

6.5 Context selection algorithm

A practical selector uses hard filters before semantic ranking:

Framework-neutral pseudocode.

```
def select_context(task, state, candidates, token_budget):
    allowed = [c for c in candidates if policy.may_disclose(task.actor, c)]
    fresh = [c for c in allowed if not c.is_expired(state.now)]
    required = [c for c in fresh if c.is_required_for(state.current_stage)]
    optional = [c for c in fresh if c not in required]
    ranked = semantic_rank(optional, task.objective, state.current_stage)
    selected = pack(required, ranked, token_budget)
    return deduplicate_and_label(selected)
```

Hard filters protect security and correctness. Semantic ranking optimises relevance only after those constraints.

6.6 Compression and summaries

A summary is a lossy transform. It should record:

- source IDs;
- summariser model and prompt version;
- time;
- intended use;
- omitted categories;
- confidence or known ambiguity.

Do not repeatedly summarise summaries. Compression chains amplify errors and erase provenance. Prefer retrieving the original artifact when a high-impact decision depends on a detail.

6.7 Artifact-first context

For long-running work, persist intermediate outputs as typed artifacts:

- task manifest;

- plan;
- decision log;
- evidence index;
- patch set;
- test report;
- unresolved questions;
- progress checkpoint.

The next model call receives a concise index and loads artifacts as needed. This is more stable than asking the model to infer progress from a long transcript.

6.8 Context cache

Caching can reduce cost, but cache keys must include all semantics that affect the result:

- model and version;
- prompt bundle hash;
- tool catalogue version;
- source revisions;
- tenant and access scope;
- policy version;
- output schema;
- temperature or reasoning settings.

Never share a cache entry across security domains merely because the natural-language query is the same.

6.9 Context poisoning and memory poisoning

An attacker may insert instructions into a document, tool description, ticket, or memory entry so that future runs act on it. Defences include:

- provenance labels;
- retrieval-time safety checks;
- separate stores for authoritative state and model memory;
- approval before promoting untrusted content into durable memory;
- expiry and review policies;
- data-flow restrictions on sensitive destinations;
- audit of which memory items influenced a tool call.

Memory is particularly dangerous because a one-time injection can become persistent. Microsoft now treats agent memory safety as a separate lifecycle concern [R31].

6.10 Context evaluation

Evaluate the context builder independently:

- precision: how much selected context was relevant;
- recall: whether required evidence was included;
- freshness: whether stale values were excluded;
- provenance accuracy;
- security: whether forbidden content crossed domains;
- compression fidelity;
- token cost;
- downstream outcome change.

Run ablations that compare full transcript, state summary, artifact index, semantic retrieval, and progressive disclosure.

Part 7 - State, memory, and artifacts

Evidence status: Established for state management; strong production evidence for artifact handoffs in long-running agents.

7.1 State versus memory

State is information required to continue correctly: task status, current stage, resource versions, approvals, retry counts, budgets, locks, and evidence IDs.

Memory is information that may improve future model performance: preferences, prior approaches, user style, learned heuristics, or summaries.

State belongs in a deterministic store with schemas and transactional semantics. Memory may use retrieval or model-generated summaries, but it cannot be trusted for invariants.

> Use state for correctness. Use memory for usefulness. Use artifacts for handoff and evidence.

7.2 Event-sourced task state

An **event log** is an append-only sequence of facts that happened. Current state is derived by reducing those events.

Framework-neutral pseudocode.

```
TaskCreated
ContractValidated
WorkspaceProvisioned
ModelProposalRecorded
ToolCallAuthorised
ToolCallStarted
ToolCallSucceeded
EvidenceAttached
ApprovalRequested
ApprovalGranted
VerificationPassed
TaskSucceeded
```

Benefits:

- complete audit history;
- deterministic reconstruction;
- easier debugging;
- support for replay and migration;
- clear distinction between proposal and commit.

An event must be immutable. Corrections are new events, not edits to history.

7.3 Long-running task state machine

A practical state machine:

Framework-neutral pseudocode.

```
QUEUED
  -> RUNNING
  -> WAITING_FOR_INPUT
  -> WAITING_FOR_APPROVAL
  -> RETRY_SCHEDULED
  -> PARTIALLY_COMPLETED
  -> VERIFYING
  -> SUCCEEDED
  -> FAILED
  -> CANCELLED
  -> EXPIRED
```

Legal transitions are explicit:

Framework-neutral pseudocode.

```
ALLOWED = {
    "QUEUED": {"RUNNING", "CANCELLED", "EXPIRED"},
    "RUNNING": {"WAITING_FOR_INPUT", "WAITING_FOR_APPROVAL",
                "RETRY_SCHEDULED", "PARTIALLY_COMPLETED", "VERIFYING",
                "FAILED", "CANCELLED", "EXPIRED"},
    "VERIFYING": {"SUCCEEDED", "PARTIALLY_COMPLETED", "RETRY_SCHEDULED",
                  "FAILED", "CANCELLED"},
    "WAITING_FOR_APPROVAL": {"RUNNING", "FAILED", "CANCELLED", "EXPIRED"},
}
```

A model can recommend a transition, but only the state machine can commit it.

7.4 Explicit handles

A **handle** is an application-level identifier that represents stateful external context: browser session, shopping basket, workspace, database snapshot, or remote task. Store it explicitly in state and pass it through typed tool arguments.

This is preferable to invisible protocol sessions. The 2026 MCP release candidate similarly emphasises a stateless core with application state represented explicitly, although final publication status must be checked before treating the candidate as final [R17, R18].

7.5 Checkpoints

A checkpoint is a durable snapshot from which work can resume. It should include:

- event-log position;
- workflow and prompt versions;
- authoritative resource revisions;
- active handles;
- current budgets;
- pending approvals;
- artifacts and hashes;
- unresolved operations;
- last verified milestone.

Create checkpoints after externally meaningful progress, not after every token.

7.6 Artifact manifests

An artifact manifest prevents the model from treating filenames as truth.

Framework-neutral pseudocode.

```
{
  "artifact_id": "art-204",
  "type": "test_report",
  "uri": "workspace://reports/integration.json",
  "sha256": "...",
  "created_by_run": "run-88",
  "source_revision": "abc123",
  "schema_version": "2",
  "verification": {
    "status": "PASSED",
    "verifier": "pytest-parser-v4"
  }
}
```

7.7 Memory tiers

Use separate tiers:

1. **Session memory:** short-lived conversation facts.
2. **Task memory:** temporary hypotheses and notes for one task.
3. **User or organisation memory:** curated durable preferences and facts.
4. **Domain knowledge:** versioned documents and datasets.
5. **Operational state:** never called memory; stored in transactional systems.

Promotion from task memory to durable memory should require policy and often human review.

7.8 Schema and workflow migration

Long-running tasks may span deployments. Record the versions that created each event and artifact. Migration strategies include:

- keep old workers available for old tasks;
- branch workflow logic using version markers;
- transform state at a controlled checkpoint;
- cancel and restart from a verified artifact;
- prohibit migration for high-risk tasks.

Temporal and Durable Task documentation explain why replay-based workflows require deterministic code and explicit versioning [R20-R23].

7.9 State failure modes

- task status stored only in a chat message;
- same field written by model and workflow without ownership rules;
- retry counters reset after process restart;
- stale resource IDs reused across tenants;
- memory entries promoted without provenance;
- checkpoints that omit tool-side effects;
- schema migration that changes the meaning of old events;
- shared browser sessions leaking identity or data.

7.10 State tests

Property-test the reducer and transition rules. Reconstruct state from random event prefixes. Inject duplicate events, reordered delivery, missing artifacts, expired handles, and old schema versions. Verify that the system either recovers deterministically or stops safely.

Part 8 - Tool design, authority, and execution gateways

Evidence status: Established. Agent-specific tool descriptions are an interface-design concern; authorisation remains ordinary security engineering.

Failure vignette. A tool is described as “read customer configuration”, but its backend credential can also update configuration and the gateway forwards arbitrary paths. The model is prompt-injected into calling an undocumented endpoint. The schema looked read-only; the effective credential and broker were not. Tool descriptions did not create an authority boundary.

8.1 Tool execution through an enforcing gateway is an authority boundary

Tools convert model output into effects. The strongest model cannot make a badly designed tool safe.

A production tool needs:

- a narrow purpose;
- typed input and output schemas;
- precise side-effect classification;
- scoped identity;
- resource constraints;
- deterministic validation;
- stable error taxonomy;
- idempotency semantics;
- timeout and cancellation support;
- audit events;
- verification strategy;
- versioning and deprecation policy.

8.2 Design semantic tools, not UI macros

Prefer a tool that expresses business intent:

Framework-neutral pseudocode.

```
create_refund(order_id, amount, reason, idempotency_key)
```

rather than low-level UI operations:

Framework-neutral pseudocode.

```
click(x, y)
type(text)
```

Semantic tools reduce ambiguity and improve validation. Computer-use tools remain necessary when no API exists, but they require stronger verification and tighter authority.

8.3 Separate read and write tools

Do not overload a single `database` tool with arbitrary SQL. Expose:

- read-only query tools with row and column policies;
- specific mutation tools for approved operations;
- administrative tools on separate identities;
- export tools with data-loss prevention checks.

The model should not receive write capabilities during a read-only stage.

8.4 Tool contract

A concise tool contract includes:

Framework-neutral pseudocode.

```
name: update_ticket_status
purpose: "Change one Jira ticket status"
side_effect: write
risk_class: medium
input_schema:
  ticket_id: {type: string, pattern: "^[A-Z]+-[0-9]+$"}
  expected_status: {type: string}
  new_status: {enum: ["In Progress", "Blocked", "Done"]}
  reason: {type: string, maxLength: 500}
  idempotency_key: {type: string}
authority:
  required_scope: jira.ticket.transition
  resource_rule: "ticket_id must equal task.ticket_id"
preconditions:
  - "current status equals expected_status"
postconditions:
  - "read-back status equals new_status"
errors:
  - CONFLICT
  - FORBIDDEN
  - NOT_FOUND
  - TRANSIENT
```

8.5 Tool gateway

A **tool gateway** is the single broker through which agent tool calls pass. It becomes a security boundary only when it independently enforces authentication, authorisation, confinement, policy, and audit. It can enforce:

- schema validation;
- identity binding;
- tenant and resource scope;
- data classification and destination policy;
- rate limits and budgets;
- approvals;
- idempotency;
- egress restrictions;
- request and response logging;
- tool version compatibility;
- kill switches.

Framework-neutral pseudocode.

```
model proposal
  -> schema validator
  -> policy decision point
  -> approval gate
  -> credential broker
  -> tool adapter
  -> postcondition verifier
  -> audit log
```

The model never receives raw long-lived credentials.

8.6 Idempotency

An operation is **idempotent** when repeating it with the same key has the same effect as executing it once. Network timeouts create ambiguity: the caller may not know whether the server committed the request. Idempotency keys resolve that ambiguity.

For non-idempotent external systems, build an adapter that stores request keys and observed results. If duplicate prevention is impossible, the workflow must require manual reconciliation before retry.

8.7 Error contracts

Return machine-readable errors:

Framework-neutral pseudocode.

```
{
  "code": "CONFLICT",
  "retryable": false,
  "message": "Ticket status changed",
  "current_state": {"status": "Done", "version": "991"},
  "suggested_recovery": "refresh_and_replan"
}
```

Avoid returning stack traces or vague strings such as “something went wrong” to the model. Error contracts allow deterministic retry and recovery policies.

8.8 Tool descriptions

A model-visible description should state:

- what the tool does;
- when it should be used;
- when it should not be used;
- side effects;
- required preconditions;
- important argument semantics;
- expected result;
- common errors.

Do not place secret policy or security assumptions only in the description. The description improves selection; the gateway enforces behaviour.

8.9 Resource selectors

Broad selectors are dangerous:

- `all_files=true`;
- `query="*"`;
- `send_to=any`;
- `repository="all"`.

Resolve resources deterministically from the task contract. Prefer immutable IDs over names. For batch operations, require an explicit manifest and a maximum cardinality.

8.10 Tool supply chain

MCP servers, plugins, scripts, and tool adapters are dependencies. Apply:

- signed or pinned versions;
- code review;
- dependency scanning;
- sandboxing;
- network allowlists;
- capability inventories;
- owner and deprecation metadata;
- behavioural conformance tests.

A tool can be malicious through its implementation or through a poisoned description that manipulates model choice.

8.11 Tool evaluation

Test:

- valid and invalid arguments;
- boundary values;
- cross-tenant access;
- stale versions;
- duplicate keys;
- timeout after commit;
- partial failure;
- cancellation;
- oversized output;
- malicious output containing instructions;
- unavailable verifier;
- policy changes between proposal and execution.

Part 9 - Interoperability: MCP, A2A, and protocol boundaries

Evidence status: Strong ecosystem evidence for the division of roles; protocol maturity and release status are time-sensitive.

9.1 Protocols solve connectivity, not architecture

The Model Context Protocol (MCP) standardises how clients discover and invoke tools, prompts, and resources. Agent2Agent (A2A) standardises communication and task collaboration between independent agents. Official A2A documentation presents MCP as agent-to-tool communication and A2A as agent-to-agent communication [R17-R19].

This split is useful, but neither protocol defines your business workflow, trust model, or completion criteria.

9.2 MCP use cases

Use MCP when an agent needs a standard interface to:

- files or knowledge resources;
- databases;
- developer tools;
- SaaS APIs;
- internal services;
- reusable prompts or capabilities.

Do not assume that connecting an MCP server makes its tools safe. Apply the same gateway, identity, policy, and verification controls as any other tool.

9.3 MCP freshness note - 29 July 2026

The official MCP blog published the **`2026-07-28` specification release candidate** on 21 May 2026, describing a stateless core, extensions, revised authorisation, tasks, and deprecation policy. Beta SDKs were announced on 29 June. At the time this guide was assembled, the official archive still prominently listed the release candidate and beta SDK post rather than a separately verifiable final-release announcement. Therefore this guide treats those changes as **release-candidate semantics**, not as a confirmed final release [R17, R18].

This is an example of research hygiene: planned release dates are not release evidence.

9.4 A2A use cases

Use A2A when independently deployed agents need:

- capability discovery;
- delegated tasks;
- asynchronous status;
- streaming updates;
- artifacts;
- opaque internal implementation;
- cross-framework communication.

A2A 1.0 is documented by the Linux Foundation project as a stable interoperability protocol [R19]. The protocol should still sit behind organisational identity and policy.

9.5 Pattern: protocol gateway

Use when. One organisation must connect multiple MCP servers, A2A agents, or legacy adapters while enforcing common identity, policy, audit, and compatibility rules.

Avoid when. The integration is inside one trust domain and a direct typed API is simpler. A gateway should not be introduced merely because a protocol is fashionable.

Framework-neutral pseudocode.

```
local agent
  -> organisation policy gateway
       -> MCP server A
       -> MCP server B
       -> A2A remote agent C
       -> legacy API adapter D
```

Mechanism. The gateway authenticates the caller, resolves delegated authority, filters discovery, normalises capabilities and errors, applies rate and data-flow policy, negotiates versions, forwards the request with scoped credentials, and records response provenance.

Invariants. Protocol connectivity never bypasses organisational policy. Remote capability names are mapped to canonical internal identities. Delegated authority cannot exceed the caller's authority ceiling. Every response identifies its source and negotiated version.

Guardrails. Allowlist servers and agent operators; validate signed metadata where available; restrict destinations and data classes; cap task duration and fan-out; isolate untrusted returned content; require approval for new external side effects.

Failure modes. The gateway can become a confused deputy, flatten distinct remote identities, silently downgrade protocol features, or centralise credentials into a high-value target. Translation may lose cancellation or partial-completion semantics.

Observability. Record caller, remote identity, protocol and version, capability mapping, delegated scope, request and task IDs, data classification, latency, retries, cancellation, and provenance. Alert on unknown servers, scope changes, and version downgrade.

Evaluation. Test mixed versions, unknown fields, capability removal, remote timeout, cancellation races, malicious metadata, identity substitution, partial completion, and replay. Verify policy equivalence between direct and gateway paths.

Framework mapping. OpenAI, Anthropic, and Google provide MCP clients or integrations; Google ADK also supports A2A [R4, R13, R15-R19]. None supplies an organisation-wide trust gateway, cross-protocol policy model, or canonical identity registry. Those are platform-engineering responsibilities.

Competing alternatives. Direct API adapters are simpler inside one service. A service mesh can provide transport identity and telemetry but not semantic tool authority. Separate gateways by risk domain if one central gateway would create excessive blast radius.

9.6 Agent delegation contract

When delegating to another agent, include:

- objective and non-goals;
- authority ceiling;
- data that may be disclosed;
- allowed side effects;
- deadline and budget;
- expected artifacts;
- evidence and verification requirements;

- cancellation semantics;
- error and partial-completion schema.

Do not delegate with a conversational message such as “handle this” and then trust the returned prose.

9.7 Trust and identity

An agent card or capability description is not proof of identity. Use authenticated transport, workload identity, scoped tokens, and signed metadata where available. The caller should know:

- which organisation operates the remote agent;
- which agent and version handled the task;
- what authority was delegated;
- what downstream tools were allowed;
- where data may be stored;
- how results are attested.

9.8 Versioning and compatibility

Protocols evolve. Pin protocol and SDK versions for production. Record negotiated versions in traces. Test:

- unknown fields;
- deprecated fields;
- capability removal;
- changed error semantics;
- long-running tasks resumed after upgrade;
- mixed-version clients and servers.

9.9 When not to use a protocol

Do not introduce MCP or A2A when a direct typed API inside one service is simpler. Protocols add discovery, transport, authentication, compatibility, and observability work. Use them when organisational or ecosystem boundaries justify that cost.

9.10 Concrete cross-framework mapping

A framework mapping is useful only when it states what was checked, against which immutable version, and what remains application-owned. None of the provider blocks below is labelled runtime-tested because the publication build could not resolve provider packages or use live credentials. The Claude and Google blocks are **source-contract checked** against pinned commits; the OpenAI block is a **versioned documentation mapping**. All three are syntax-checked during the manuscript build. The machine-readable `framework_source_contract_manifest.json` in the companion archive records package, version, commit, symbols, source files, blob hashes, and separate booleans for syntax, import, and runtime verification.

Compatibility and authenticity matrix

Stack	Pinned source	Code status	Checked contract	Application responsibility not supplied
OpenAI Agents SDK for Python	release 0.19.1 at commit `ddc39d0e54c9` and versioned HITL documentation [R1-R3]	Versioned documentation mapping; syntax-checked, not source-contract checked or runtime-executed	documented agent/run state, approval interruptions, and tool guardrails	durable business workflow, policy store, credential delegation, domain verifier
Claude Agent SDK for Python	0.2.128, commit `f8b9ec923982` [R74]	Source-contract checked; not runtime-executed	`HookMatcher`, `can_use_tool`, permission results, deferred tool state	durable approval wait, business transaction and final read-back
Google ADK for Python	2.5.0, commit `6bab08fc803d` [R73]	Source-contract checked; not runtime-executed	singular `google.adk.workflow`, edge-defined `Workflow`, callbacks on `LlmAgent`	organisation policy, idempotent side effects, long external approval semantics

The companion `verify_manuscript_examples.py` fails when a substantial code block lacks an authenticity label, contains invalid Python syntax, or contains a forbidden stale interface. `verify_source_contract_manifest.py` validates manifest structure and, by default, fetches every pinned source path and recomputes its Git blob SHA. A `--manifest-only` mode is explicitly labelled structural validation and is not accepted as source-content verification. A separate dependency-enabled CI job is still required before changing a provider block's label to **Tested example**.

OpenAI Agents SDK with application-owned durability

Illustrative API mapping - not executable as a complete service. Versioned documentation mapping for OpenAI Agents SDK 0.19.1 at release commit `ddc39d0e54c9`; this block is not source-contract checked.

```
from agents import Agent, Runner

qa_agent = Agent(
    name="qa_worker",
    instructions=prompt_bundle,
    tools=[read_ticket, inspect_ui, propose_finding],
    input_guardrails=[task_contract_guard],
    output_guardrails=[finding_schema_guard],
)

# The SDK owns model/tool turns. The durable workflow owns business state.
run = Runner.run_streamed(qa_agent, compiled_task_input)
async for event in run.stream_events():
    audit.append_sdk_event(task_id, event)

if run.interruptions:
```

```
    serialised_sdk_state = run.to_state().to_json()
    request = canonicalise_interruptions(run.interruptions)
    approval = approval_store.create(
        task_id=task_id,
        action_hash=sha256(request.canonical_bytes),
        policy_version=policy.version,
        expires_at=clock.now() + APPROVAL_TTL,
        sdk_state=serialised_sdk_state,
    )
    durable_workflow.suspend(task_id, approval.id)
```

This block intentionally omits application services and therefore is not presented as executable. On resume, the application reloads state, verifies task ownership, approval expiry, policy version and action hash, re-evaluates current policy, applies the decision to the relevant interruption, and invokes the runner again. The SDK does not define the organisation's transaction, credential, outbox or postcondition semantics.

Claude Agent SDK 0.2.128 with deferred approval and non-bypassable hooks

Listing 9.3 - Source-contract checked against Claude Agent SDK Python 0.2.128 at commit `f8b9ec923982`; Python-syntax checked; not package-imported or runtime-executed. The complete listing is `listing_9_3_claude_deferred_approval.py` in the companion archive.

```
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher, ResultMessage

async def pre_tool_gate(hook_input, tool_use_id, _context):
    request = canonicalise(hook_input["tool_name"], hook_input["tool_input"])
    decision = policy.evaluate(request, effective_authority())
    approval = approvals.find_fresh(action_hash=request.action_hash)

    if decision.effect == "deny":
        return hook_decision("deny", reason=decision.reason)
    if decision.effect == "approval_required" and approval is None:
        # Preserves a typed pending call in ResultMessage.deferred_tool_use.
        return hook_decision("defer", updated_input=request.arguments)

    revalidate_policy_authority_and_toctou(request, approval)
    return hook_decision("allow", updated_input=request.arguments)

options = ClaudeAgentOptions(
    tools=["ReadTicket", "InspectUI", "ProposeFinding"],
    hooks={
        "PreToolUse": [HookMatcher(matcher=None, hooks=[pre_tool_gate])],
        "PostToolUse": [HookMatcher(matcher=None, hooks=[capture_evidence])],
    },
    permission_mode="default",
    setting_sources=[],
)

# On terminal ResultMessage, persist deferred_tool_use plus session_id,
# canonical action hash, policy/authority versions and approval expiry.
```

The pinned SDK exposes `permissionDecision: "defer"` on `PreToolUse` and surfaces the pending call as `ResultMessage.deferred_tool_use`, containing its ID, name and arguments [R74, R83-R86]. This differs materially from `PermissionResultDeny(..., interrupt=True)`: denial terminates an attempted action, whereas defer preserves typed pending-action state for an application-owned approval round trip.

The application still owns the durable wait. Persist the SDK session ID, deferred tool-use ID, canonical arguments, immutable action hash, policy version, effective-authority digest, requester, expiry and task identity. On approval, resume the same SDK session and permit only a reissued action whose canonical identity matches the record after fresh policy and TOCTOU validation. A changed proposal must receive a new approval.

Verification status: this is a **source-contract-checked integration design**, not a runtime-proven exact-resumption guarantee. The pinned SDK demonstrates typed deferred state, result propagation and session-resume surfaces [R74, R83-R86]. This publication did not execute a credentialed cross-process round trip proving that the same deferred call executes exactly once after process exit. A dependency-enabled end-to-end test remains required before upgrading the label to "Tested example". The companion package includes an application-level deterministic test for canonical hashing, expiry, revalidation and duplicate suppression; it deliberately does not impersonate the provider integration.

`can_use_tool` remains useful as a replacement for an interactive permission prompt, but it is not a universal interceptor. Whole-tool allow rules and some permission modes can approve calls before it is consulted. A `PreToolUse` hook is therefore the control point for policy that must observe or gate every invocation. Hook matchers for one event are dispatched concurrently, so independent hooks must not depend on ordering [R74, R84].

Google ADK 2.5.0 graph workflow

Source-contract checked against Google ADK 2.5.0 at commit `6bab08fc803d`; not runtime-executed because application nodes and model credentials are placeholders.

```
from google.adk.agents import LlmAgent
from google.adk.workflow import DEFAULT_ROUTE, START, Workflow

proposal_agent = LlmAgent(
    name="proposal_agent",
    model="gemini-2.5-pro",
    instruction=prompt_bundle,
    tools=[read_ticket, inspect_ui, propose_finding],
    before_tool_callback=policy_and_authority_callback,
    after_tool_callback=evidence_callback,
)

qa_workflow = Workflow(
    name="qa_verification",
    edges=[
        (START, load_task_node, proposal_agent, deterministic_policy_node),
        (
            deterministic_policy_node,
            {
                "allowed": scoped_execution_node,
                "requires_approval": approval_wait_node,
                DEFAULT_ROUTE: reject_node,
            },
        ),
        (approval_wait_node, approval_revalidation_node, scoped_execution_node),
        (scoped_execution_node, authoritative_readback_node),
    ],
)
```

The package is `google.adk.workflow`, singular. `Workflow` takes edge declarations; its graph derives nodes from those edges and rejects an explicitly populated node list [R73]. Tool callbacks belong to `LlmAgent`. The workflow can express graph routing and replay-aware node

orchestration, but application code still owns approval durability across days, transaction identity, idempotency, delegated credentials, policy-version migration and domain-specific verification.

What no SDK supplies

Production responsibility	OpenAI	Claude	Google ADK	Required owner
Model/tool loop	yes	yes	yes	SDK/runtime
Typed tool surface	yes	yes	yes	SDK plus tool implementation
Observe every tool invocation	guardrails/hooks, configuration-dependent	`PreToolUse` hook	`LlmAgent` callbacks	application policy gateway
Durable approval bound to immutable action	no complete business primitive	no complete business primitive	no complete business primitive	workflow and approval service
Short-lived delegated credentials	no	no	no	identity platform
Transactional side-effect semantics	no	no	no	domain service
Authoritative read-back verifier	no	no	no	domain verifier
Release evidence and statistical gate	no	no	no	evaluation programme

9.11 Framework mapping decision procedure

1. Pin package version, repository commit, runtime and feature flags.
2. Mark each block as runtime-tested, source-contract checked, illustrative, or framework-neutral.
3. Identify which lifecycle events the SDK actually emits and which the application invents.
4. Check whether permission configuration can bypass callbacks or guardrails.
5. Separate serialisable SDK run state from durable business workflow state.
6. Put organisation policy, credential minting, transaction semantics and verification outside provider-specific prompt logic.
7. Add dependency-enabled CI before promoting a source-contract-checked block to a tested example.

Part 10 - Durable execution and long-running agents

Evidence note. Replay, idempotency, leases, cancellation, and compensation are established distributed-systems concepts. Their integration with stochastic model loops remains an active engineering area.

10.1 Durable execution

Durable execution means a task maintains state and progress across process crashes, deployments, network failures, and long waits. Systems such as Temporal, Azure Durable Task, and Dapr Workflows persist event history and reconstruct orchestration state [R20-R25].

The model call and tool call are activities inside the durable process; they are not the durable process itself.

10.2 Replay

Replay re-executes workflow code against recorded history to reconstruct local state. Replay requires deterministic orchestration: the same history must produce the same commands. External I/O and nondeterministic operations belong in activities whose results are recorded.

Do not invoke a model directly inside replayed deterministic code unless the result is captured as an event and not recomputed during replay.

10.3 At-least-once execution

Many queues and workflow engines provide **at-least-once delivery**: an activity may run more than once. Therefore every side effect needs idempotency or reconciliation.

Exactly-once business effects are achieved through design, not by assuming exactly-once transport.

10.4 Leases, heartbeats, and fencing tokens

A **lease** grants a worker temporary ownership of a task. A **heartbeat** proves the worker is still active. After lease expiry, another worker may continue.

A **fencing token** is a monotonically increasing ownership number included with writes. The external resource rejects writes from stale workers with lower tokens. This prevents an old worker that resumes late from corrupting current state.

Framework-neutral pseudocode.

```
lease = coordinator.acquire(task_id)
while working:
    coordinator.heartbeat(lease)
    external.write(data, fencing_token=lease.generation)
```

10.5 Durable state machine

A long-running agent should represent waits explicitly:

Framework-neutral pseudocode.

```
RUNNING
  -> WAITING_FOR_TOOL
  -> WAITING_FOR_TIMER
```

```
  -> WAITING_FOR_INPUT
  -> WAITING_FOR_APPROVAL
  -> RETRY_SCHEDULED
  -> VERIFYING
  -> terminal state
```

A wait is not a blocked process. The runtime persists state and releases compute.

10.6 Activity boundaries

Choose activity boundaries around side effects and expensive nondeterminism:

- model call;
- database transaction;
- browser session step;
- external API request;
- sandbox command;
- verification suite;
- notification.

Activities should return compact, typed results and store large outputs as artifacts.

10.7 Retry policy

A durable retry policy contains:

- eligible error codes;
- initial delay;
- backoff factor;
- maximum delay;
- maximum attempts;
- total retry deadline;
- jitter;
- idempotency key;
- escalation after exhaustion.

Model retries should distinguish transport failure from unsatisfactory reasoning. Repeating the same prompt after a semantic failure usually reproduces the same failure. Change context, model, strategy, or ask for clarification.

10.8 Compensation and sagas

A **saga** coordinates a sequence of local transactions across systems. If a later step fails, compensating actions attempt to restore an acceptable state.

Framework-neutral pseudocode.

```
reserve inventory
  -> charge payment
  -> create shipment

if create shipment fails:
  -> refund payment
  -> release inventory
```

An agent may propose compensation, but code should define allowed compensating operations and their order.

10.9 Cancellation

Cancellation is a first-class command. Define behaviour for:

- pending model call;
- running tool;
- external API without cancellation;
- waiting approval;
- partial side effects;
- verifier in progress.

Cancellation should produce a final state and reconciliation report, not simply kill a process.

10.10 Deadlines and expiry

Separate:

- **step timeout:** one activity took too long;
- **task deadline:** objective must finish by a time;
- **approval expiry:** an old approval may no longer be valid;
- **credential expiry:** tokens must be refreshed;
- **evidence freshness:** a result may require re-verification.

On resume, re-evaluate time-sensitive policy and state.

10.11 Backpressure and queues

Backpressure prevents producers from overwhelming workers or external systems. Apply queue limits by tenant, risk, and task class. High-cost agents need concurrency controls because model and browser work can exhaust budget quickly.

Use dead-letter queues for tasks that repeatedly fail due to unclassified errors. Dead-lettering is an operational state requiring investigation, not a silent terminal failure.

10.12 Versioning in-flight tasks

A workflow upgrade can change prompts, models, tool schemas, or transition logic. Strategies:

- worker version pinning;
- explicit version branches;
- checkpoint migration;
- restart from verified artifacts;
- compatibility adapters.

Never allow an old task to resume with a newly broadened tool catalogue without policy review.

10.13 Durable execution pseudocode

Framework-neutral pseudocode.

```
@workflow
def agent_task(task_id):
    state = load_or_create_state(task_id)
    while not state.terminal:
        event = await next_event_or_activity(state)
```

```
        state = reducer.apply(state, event)

        if state.needs_model_proposal:
            proposal = await activity.call_model(state.model_input(), retry=
    transport_only)
            state = reducer.apply(state, ProposalRecorded(proposal))

        if state.needs_tool_execution:
            decision = policy.evaluate(state)
            if decision.requires_approval:
                state = reducer.apply(state, ApprovalRequested(decision))
                continue
            result = await activity.execute_tool(
                state.action,
                idempotency_key=state.action_id,
                retry=classified_retry,
            )
            state = reducer.apply(state, ToolResultRecorded(result))

        if state.needs_verification:
            check = await activity.verify(state)
            state = reducer.apply(state, VerificationRecorded(check))

        persist_checkpoint(state)
    return state.final_report()
```

10.14 Failure-injection tests

Terminate workers:

- before sending a tool call;
- after the external commit but before recording success;
- during artifact upload;
- while waiting for approval;
- during verification;
- after a deployment changes workflow code.

Also inject duplicate queue deliveries, expired credentials, stale fencing tokens, unavailable memory, and partial external outages.

Part 11 - Verification, completion, and evidence

Evidence status: Established. The exact verifier mix is workload-specific.

11.1 Four levels of “done”

Agent systems routinely collapse four different claims:

1. **The model believes it succeeded.**
2. **A tool accepted a command.**
3. **The authoritative system reflects the intended state.**
4. **An independent check confirms that the state satisfies the task contract.**

Only the fourth is a reliable completion condition for consequential work.

11.2 Completion contract

Define completion before execution. A completion contract contains:

- observable outcomes;
- authoritative sources to query;
- acceptable tolerances;
- required evidence artifacts;
- independent verifier where needed;
- partial-completion representation;
- unresolved-risk thresholds;
- freshness window.

For a code change:

Framework-neutral pseudocode.

```
completion:
  - patch applies to revision abc123
  - targeted regression test passes
  - full affected test suite passes
  - linter and type checker pass
  - no unexpected public API change
  - diff is linked to issue acceptance criteria
```

11.3 Write then read back

After a mutation, query the authoritative system rather than trusting the write response.

Framework-neutral pseudocode.

```
write = crm.update(record_id, patch, idempotency_key)
read = crm.get(record_id)
assert read.version >= write.version
assert project(read) == expected_projection
```

The read-back should use an independent route where practical. For example, verify a browser submission through the server API, not only through the rendered confirmation screen.

11.4 Generate then execute

Generated artifacts need executable verification:

- code -> compile, test, lint, run;
- SQL -> parse, explain, execute in a bounded environment, compare rows;
- infrastructure plan -> policy scan, dry run, review diff;
- spreadsheet -> recalculate formulas and validate expected cells;
- document -> validate structure, links, citations, and rendering;
- browser action -> inspect authoritative server state.

11.5 Pattern: independent verifier

Use when. A completion claim has meaningful consequences and can be checked through tests, authoritative state, a different evidence path, or a separately constrained reviewer.

Avoid when. The output is purely subjective and no independent criterion exists, or the cost of verification exceeds the consequence of error. Do not pretend a second identical prompt is independent.

Mechanism. The actor produces artifacts and evidence identifiers. The verifier receives the task contract, authoritative state, and artifacts, but not the actor's persuasive narrative unless that narrative is itself evidence under review.

Framework-neutral pseudocode.

```
actor_result = actor.run(task)
verifier_input = {
    "contract": task.completion_contract,
    "artifacts": actor_result.artifact_ids,
    "authoritative_state": fetch_state(task),
}
verification = verifier.run(verifier_input)
```

Independence can come from deterministic tests, a different tool path, a separate model with restricted context, rule-based validation, human review, or external system confirmation.

Invariants. The actor cannot write the verifier's result. The verifier uses fresh authoritative state. Success requires all mandatory criteria, and unresolved criteria remain explicit rather than being averaged away.

Guardrails. Separate credentials and write permissions; hide actor confidence and unsupported conclusions; require evidence hashes; use deterministic checks first; escalate verifier disagreement on high-risk tasks.

Failure modes. The verifier can share the actor's blind spot, consume the same poisoned context, or accept self-authored evidence. A permissive aggregate score can conceal failure of one mandatory condition.

Observability. Record verifier version, inputs, evidence accessed, criterion-level decisions, disagreement, latency, and whether the result changed the task disposition. Track false acceptance and false rejection through audits.

Evaluation. Seed known actor failures, tampered evidence, stale state, missing artifacts, and persuasive false narratives. Calibrate model-based verifier decisions against blinded human or deterministic labels.

Framework mapping. All major agent SDKs can instantiate a verifier agent or tool [R1, R8, R15]. None guarantees independence. The application must separate context, credentials, evidence paths, and acceptance logic.

Competing alternatives. Deterministic postcondition tests are superior where available. Human review is appropriate for rare high-consequence ambiguity. N-version verification using two models helps only when their errors are measurably less correlated.

11.6 Evidence bundles

An evidence bundle links each claim to immutable or versioned artifacts:

Framework-neutral pseudocode.

```
{
  "claim_id": "c-17",
  "claim": "Acceptance criterion 3 passes",
  "evidence": [
    {"id": "ev-2", "type": "screenshot", "hash": "..."},
    {"id": "ev-3", "type": "http_trace", "request_id": "..."}
  ],
  "verified_by": "qa-verifier-v8",
  "verified_at": "2026-07-28T10:30:00Z",
  "fresh_until": "2026-07-29T10:30:00Z"
}
```

Evidence should be sufficient for a reviewer to reproduce or inspect the conclusion.

11.7 Partial completion

A task is partial when some objectives are verified and others are blocked, failed, or unresolved. Partial completion is not a vague apology. It should state:

- verified outcomes;
- attempted but unverified outcomes;
- blockers;
- side effects already committed;
- rollback or compensation status;
- next required actor;
- remaining risk.

11.8 Completion gate

Framework-neutral pseudocode.

```
def completion_gate(task, state):
    required = task.completion_contract.required_claims
    verified = {c.claim_id for c in state.verifications if c.passed and c.
   is_fresh}
    if required <= verified and not state.open_high_risk_findings:
        return "SUCCEEDED"
    if verified and state.blocked:
        return "PARTIALLY_COMPLETED"
    if state.retryable:
        return "RETRY_SCHEDULED"
    return "FAILED"
```

The model's final answer is a report, not the state transition authority.

11.9 Premature-completion tests

Seed environments where:

- a success toast appears but the server rejected the write;
- a test command exits zero without running tests;
- a file exists but contains stale data;
- a pull request is open but checks failed;
- a message was drafted but not sent;
- an API returns `202 Accepted` but the asynchronous job later fails.

Measure whether the agent seeks authoritative confirmation.

Part 12 - Guardrail architecture across the lifecycle

Evidence status: Established for deterministic controls; emerging for model-based detectors and critics.

Failure vignette. A probabilistic classifier screens outbound messages. During a dependency outage the caller treats timeout as “no violation” and continues. The system has a guardrail catalogue, but no composed policy for dependency failure, conflict resolution, or fail-closed behaviour at the irreversible boundary.

12.1 Guardrails are not one filter

A guardrail is any control that prevents, transforms, pauses, contains, or detects unsafe or invalid behaviour. Production guardrails are distributed across the lifecycle.

12.2 Lifecycle positions

1. Before inference

- authenticate caller;
- validate task contract;
- classify risk;
- remove unsupported requests;
- select allowed workflow;
- set initial budgets.

2. During context construction

- enforce data access;
- label provenance;
- remove secrets;
- filter stale or poisoned memory;
- isolate untrusted content;
- limit tool disclosure.

3. Before tool selection

- expose only relevant capabilities;
- hide prohibited tools;
- restrict resource selectors;
- separate read and write stages.

4. Before tool execution

- validate arguments;
- evaluate identity and authority;
- check state preconditions;
- require approval;
- apply rate and cost limits;
- enforce egress and data-flow policy.

5. After tool execution

- validate output schema;
- remove malicious instructions from tool output;

- classify sensitivity;
- detect partial failure;
- attach provenance;
- verify side effects.

6. Before state transition

- verify legal transition;
- check required events and evidence;
- prevent stale-worker writes;
- persist atomically.

7. Before external side effect

- present exact action for confirmation;
- bind approval to arguments and expiry;
- re-check current state;
- generate idempotency key.

8. Before final output

- enforce disclosure policy;
- distinguish verified facts from hypotheses;
- include unresolved risks;
- prevent unsupported completion claims.

9. After execution

- monitor anomalies;
- sample traces;
- detect drift;
- support kill switches;
- review incidents and update evals.

12.3 Control types

Control	Strength	Typical role
Deterministic validation	high for explicit rules	schemas, enums, resource scope
Policy engine	high for encoded policy	authorisation, data flow, approval
Statistical classifier	probabilistic	injection or sensitive-data detection
Model judge	probabilistic and attackable	semantic policy or quality review
Human approval	strong but costly	high-impact ambiguity
Sandbox	containment	code, browser, files, network
Budget tripwire	deterministic	loops, cost, rate, delegation
Post-execution monitor	detective	anomalies and incident response

Layer controls so that failure of one does not expose unrestricted authority.

12.4 Guardrail responses

A guardrail trigger need not always terminate. Possible responses:

- allow;
- allow with redaction;
- rewrite or normalise arguments;
- downgrade authority;
- substitute a safer tool;
- request clarification;
- request approval;
- retry with isolated context;
- quarantine the task;
- cancel and compensate;
- block and alert.

The response must be deterministic for a given policy decision.

12.5 Fail-open and fail-closed

A guardrail **fails closed** when its own failure blocks the operation. Use this for high-risk writes, credential issuance, data export, and production changes.

A guardrail **fails open** when its failure allows the operation. This may be acceptable for low-risk, read-only assistance where availability is more important than the control.

Document the choice. Accidental fail-open behaviour is a serious defect.

12.6 Policy decision schema

Framework-neutral pseudocode.

```
{
  "effect": "ALLOW | DENY | REQUIRE_APPROVAL | TRANSFORM | QUARANTINE",
  "reason_code": "DATA_EGRESS_REQUIRES_APPROVAL",
  "policy_version": "2026-07-28.7",
  "bound_action_hash": "...",
  "expires_at": "2026-07-28T12:00:00Z",
```

```
  "required_approver_role": "data_owner",
  "transform": null
}
```

Approvals must be bound to the exact action hash. If arguments change, approval is invalid.

12.7 Guardrail patterns

Argument-level authorisation

Validate each resource and destination, not merely the tool name.

State-transition guard

Prevent transitions without required prior events and evidence.

Data-flow guard

Track source sensitivity and destination. Block private data from flowing to public tools or channels.

Resource-path restriction

Resolve and canonicalise file paths; reject traversal and symlink escapes.

Loop and budget tripwire

Stop when tool-call count, cost, or repeated-state threshold is exceeded.

Transactional write guard

Require expected version, idempotency key, and postcondition verifier.

Evidence gate

Block completion or publication when required sources are missing.

Kill switch

Disable a tool, model, tenant, or workflow version centrally without redeploying every agent.

12.8 Guardrail evaluation

Create a matrix of:

- lifecycle position;
- threat or failure;
- control;
- expected decision;
- fail-open/closed behaviour;
- latency and cost;
- false-positive tolerance;
- owner;
- regression tests.

Test bypasses through aliases, nested delegation, encoded arguments, stale approval, malicious tool output, and resumed tasks.

12.9 Worked guardrail-policy execution path

A customer ticket contains untrusted HTML instructing the agent to export account data. The user asked only for UI verification. The correct path is:

Framework-neutral pseudocode.

```
untrusted input
  -> provenance label: external_ticket_content
  -> model proposes ExportCustomerData(account=42)
  -> canonicalise resource, tenant, fields and destination
  -> deterministic schema and task-contract validation
  -> classifier assigns exfiltration-risk score 0.94
  -> policy maps score >=0.80 to DENY, not to a model suggestion
  -> argument-level authorisation rejects export outside task scope
  -> no approval request and no credential minting
  -> denial event, trace, and security signal recorded
```

For an allowed but approval-gated mutation, the path continues:

Framework-neutral pseudocode.

```
policy ALLOW_WITH_APPROVAL
  -> create immutable canonical action bytes
  -> bind approval to SHA-256(action), principal, tenant, policy version, expiry
  -> suspend durable workflow
  -> on resume, validate issuer, audience, signature, expiry, revocation and
   nonce
  -> reload authoritative resource state
  -> rerun policy to close the time-of-check/time-of-use gap
  -> require the reconstructed action hash to equal the approved hash
  -> mint one-operation, short-lived workload credential
  -> execute through confined gateway with idempotency key
  -> read authoritative state back from the source system
  -> compare postcondition, commit audit event, revoke credential
```

Classifier-to-action policy

Score or condition	Deterministic action	Dependency outage behaviour
schema invalid or provenance missing	reject	fail closed
exfiltration score >= 0.80	deny and alert	fail closed
0.40-0.80 on reversible read	allow only in sandbox; log	fail closed if classifier unavailable
0.40-0.80 on write	approval required	fail closed
<0.40 and policy-authorised read	allow with scoped credential	fall back to conservative static policy

Policy conflicts resolve by deny-overrides unless a documented, versioned exception names the exact resource and action. A policy-version change invalidates pending approvals unless migration code proves equivalence. Measure bypass rate with adversarial variants, false-positive cost in reviewer minutes and delayed tasks, and outage behaviour through dependency fault injection.

Common misdiagnoses

- A model refusing an instruction is not authorisation.
- A signed approval is not valid if it is not bound to the exact canonical action.
- A tool description is not confinement.
- A successful API response is not proof of the intended postcondition.
- Signed remote metadata proves who signed the claim, not that the claimed execution was correct.

Part 13 - Threat modelling and agent security

Evidence status: Established for least privilege and isolation; strong evidence that prompt injection remains unsolved; emerging evidence for information-flow controls.

13.1 Security objective

The security objective is not “make the model ignore bad instructions”. It is:

> A compromised, confused, or manipulated model must not be able to exceed the authority and information-flow rules enforced by the surrounding system.

OWASP lists prompt injection and excessive agency among the leading risks for LLM applications, while current Microsoft and OpenAI guidance emphasises defence in depth, least privilege, confirmations, monitoring, and system-level controls [R26-R34].

13.2 Assets

Identify assets such as:

- credentials and tokens;
- private data;
- source code;
- production systems;
- payment or communication authority;
- model and prompt configuration;
- memory stores;
- audit logs;
- tool registry;
- user trust.

13.3 Trust boundaries

Typical boundaries:

Framework-neutral pseudocode.

```
user/device
  -> agent API
  -> context and memory stores
  -> model provider
  -> tool gateway
  -> sandbox
  -> internal systems
  -> external internet/services
  -> remote agents
```

For each boundary, document authentication, encryption, identity propagation, logging, and data retention.

13.4 Identity, authority, delegation and attestation

Keep four concepts separate:

- **identity:** which workload, user, service or remote agent is acting;
- **authority:** which resources and operations that principal may use;
- **delegation:** how a caller passes a strictly attenuated subset of authority to another component;
- **attestation:** a signed statement about software, environment or execution context. Attestation is evidence, not proof that the business action was correct.

Validate token issuer and audience, signature, subject, tenant, expiry, not-before time, revocation state and request binding. Use workload identity rather than long-lived shared API keys. Every delegation hop must narrow or preserve scope - never widen it - and record the delegation chain. Bind high-risk approvals to an immutable canonical action hash, principal, resource version, policy version, expiry and one-time nonce. Mint credentials only after final policy and TOCTOU revalidation; make them short-lived, operation-scoped and unusable for a different destination. Use idempotency keys and replay caches where duplicate delivery could repeat a side effect.

13.5 Threats

Direct prompt injection

A user attempts to override application instructions.

Indirect prompt injection

Malicious instructions are embedded in content the agent reads: webpages, emails, tickets, documents, code, images, or tool output. Large-scale 2026 red-team evidence shows that frontier systems remain vulnerable, although attack rates vary by model and scenario [R32].

Confused deputy

The model uses its authority to act for content that was never authorised by the user.

Tool poisoning

A tool description or response manipulates the model into choosing dangerous actions.

Memory poisoning

Malicious or incorrect content is persisted and influences later tasks.

Data exfiltration

Sensitive input is transmitted to an unauthorised destination through a tool, model call, log, or remote agent.

Excessive agency

The system grants more functions, permissions, autonomy, or scope than the task requires [R26-R28].

Supply-chain compromise

An MCP server, plugin, model, dependency, or prompt package is compromised.

Code-execution escape

Generated code escapes a sandbox, accesses secrets, or uses unrestricted network egress.

Cross-tenant leakage

Caches, memory, browser sessions, logs, or artifacts mix security domains.

13.6 The lethal combination

A particularly dangerous design combines:

1. access to untrusted content;
2. access to sensitive data;
3. ability to communicate externally.

A successful injection can then instruct the agent to copy sensitive data to an attacker-controlled destination. Break this combination through separate identities, data-flow policy, read/write separation, and approval.

13.7 Security architecture

Minimum controls:

- task-scoped workload identity;
- short-lived credentials;
- no ambient host credentials;
- read/write tool separation;
- policy gateway for every side effect;
- network egress allowlist;
- sandbox isolation;
- provenance labels;
- secret redaction from prompts and logs;
- approval for irreversible or external actions;
- memory promotion controls;
- tamper-evident audit logs;
- central kill switch;
- continuous red-team and regression suite.

13.8 Information-flow control

Information-flow control (IFC) labels data and restricts where it may flow. A simplified policy:

Framework-neutral pseudocode.

```
UNTRUSTED_WEB + PRIVATE_CUSTOMER_DATA
  may flow to: private analysis sandbox
  may not flow to: public web, email, external MCP server
```

Unlike a model detector, an IFC rule can provide deterministic enforcement when labels and destinations are known. Microsoft research and product guidance increasingly explore this direction [R29, R30, R33]. IFC is not a complete solution: labels can be wrong, semantics can be ambiguous, and useful tasks may require controlled declassification.

13.9 Human approval

Approval is appropriate when:

- loss is large or irreversible;
- policy depends on human intent not present in state;
- the destination is external;
- data sensitivity is high;
- the task crosses a trust boundary;
- evidence is incomplete;
- the agent requests exceptional authority.

Approval UI should show exact action, target, data, reason, evidence, and alternatives. Never ask a human to approve a vague plan.

13.10 Threat-model table

Threat	Prevent	Contain	Detect	Recover
Indirect injection	provenance, isolation, tool policy	least privilege, egress control	injection detector, anomalous calls	revoke token, quarantine, review
Duplicate side effect	idempotency	transaction limits	duplicate event alert	reconcile/compensate
Cross-tenant leak	tenant-scoped stores	encryption and sandboxing	access-log analysis	revoke and incident response
Malicious tool	pin and review	gateway and sandbox	conformance monitoring	disable tool version
Memory poisoning	promotion review	separate memory tiers	influence trace	delete/rebuild memory

13.11 Red-team programme

Red-team the complete system with:

- hidden instructions in HTML, PDFs, images, and code comments;
- poisoned tool descriptions;
- malicious error messages;
- data exfiltration attempts;
- approval social engineering;
- resource-name confusion;
- stale sessions;
- nested agent delegation;
- encoded or fragmented payloads;
- prompt injection that asks the agent to conceal the attack.

Every confirmed issue becomes a regression test and a control review.

Part 14 - Decision theory, calibration, abstention, and stopping

Evidence note. Expected utility, proper scoring rules, confidence intervals, and selective prediction are established concepts. Mapping model-generated signals to calibrated action probabilities in interactive agents remains emerging and workload-specific [R35-R39, R53-R60].

14.1 Decisions have asymmetric loss

A false positive and false negative rarely have equal cost. Missing a low-priority support tag may be cheap; sending confidential data externally may be catastrophic. The useful question is not "How confident is the model?" but:

> Given the evidence available now, which allowed action has the lowest expected loss after accounting for consequence, delay, operating cost, and uncertainty?

For action `a` and possible outcome `o`:

Framework-neutral pseudocode.

```
Expected loss(a) = sum_o P(o | evidence, a) * Loss(o, a)
```

The equation is a bookkeeping discipline. It forces the team to state which outcomes matter, which probabilities are empirical, and which losses are policy judgements.

14.2 Confidence is not probability

A model saying "90% confident" does not mean that comparable answers are correct 90% of the time. Calibration studies continue to find task-dependent overconfidence, underconfidence, and divergence between accuracy and calibration [R35-R38]. Treat verbalised confidence as one feature among many.

Useful features include:

- task family and risk class;
- input ambiguity and missing fields;
- model and prompt version;
- evidence coverage;
- tool errors;
- disagreement between independent attempts;
- verifier result;
- similarity to previously failed tasks;
- whether the task is outside the calibration set.

14.3 How to estimate probabilities in engineering practice

Do not ask one engineer to invent `P(success)=0.73`. Use the strongest available method in this order.

1. Historical frequency in a calibrated bucket

Partition held-out or production-audited trials by features known **before** the action. For example:

Framework-neutral pseudocode.

```
bucket = {
  task_family: UI_ticket_verification,
  risk: medium,
  evidence_coverage: 0.6-0.8,
  verifier_disagreement: false,
  model_route: economical_model
}
```

If 240 comparable trials contain 43 missed defects, the empirical miss estimate is `43/240 = 17.9%`. Report an interval, not only the point estimate. Sparse buckets should be pooled or smoothed; they should not produce fake precision.

2. Calibrated predictive model

Fit a simple model - often logistic regression, isotonic regression, or a small gradient-boosted model - from observable features to verified outcome. Train on one period, calibrate on another, and test on a later or held-out period. Evaluate Brier score, reliability by bucket, and risk-coverage curves [R35, R53-R56].

The predictor must be versioned with the agent. A model, prompt, tool, or task-distribution change can invalidate calibration.

3. Bounded qualitative scale

For rare high-risk actions without enough data, use a policy-approved ordinal scale rather than a fabricated decimal:

Likelihood level	Operational definition
1 - remote	no observed event in relevant tests; plausible only through multiple failures
2 - unlikely	observed only under adversarial or unusual conditions
3 - possible	observed in ordinary tests or weakly controlled production
4 - likely	recurring in comparable tasks
5 - frequent	expected without a dedicated control

Combine it with a consequence scale and explicit policy thresholds. The number is a decision category, not a statistical probability.

4. Expert elicitation with sensitivity analysis

When expert judgement is unavoidable, collect estimates independently, document assumptions, and compute the decision across a range. If the action changes only when the probability is between 4.1% and 4.4%, the estimate is too fragile for autonomous execution.

14.4 Estimating consequence and loss

Loss is broader than direct money. Include:

- customer or user harm;
- security and privacy exposure;
- legal or compliance consequence;
- irreversible state change;
- engineering recovery time;

- delay and opportunity cost;
- human-review cost;
- reputational impact;
- future risk created by corrupted state or memory.

Use real incident, support, and remediation data where available. For consequences that cannot reasonably be monetised, use policy tiers with hard constraints. A catastrophic data-exfiltration class should not be traded against a few pence of model cost.

14.5 Selective prediction and abstention

A selective system may abstain on uncertain tasks. Two metrics matter:

- **coverage:** fraction of tasks attempted autonomously;
- **risk:** error or unsafe-completion rate on attempted tasks.

Raising an abstention threshold usually lowers both risk and coverage. Choose the operating point from the loss curve and staffing capacity rather than maximum automation. Plot results separately for risk classes; aggregate coverage can hide dangerous subgroups.

14.6 Engineering approximation to value of information

True value of information averages over every plausible observation and chooses the best available decision after each observation. In practice, teams often use a bounded engineering approximation, but it must preserve that branching logic rather than substituting one guessed "loss after search".

For candidate evidence source `E` with possible observations `o`:

Framework-neutral pseudocode.

```
EVSI(E) = current_minimum_expected_loss
          - sum_over_o P(o | current evidence)
              * minimum_expected_loss_after_observing(o)
          - acquisition_cost(E)
          - delay_cost(E)
          - misleading_or_stale_evidence_risk(E)
```

Estimate the observation distribution from historical searches, labelled replay data, or bounded expert scenarios. For every observation, state which decisions become available and how the loss changes. If the evidence could not alter the decision, its decision value is zero even when it is interesting.

Worked example: inspect a network trace

For a medium-risk QA ticket, historical replay gives three possible outcomes:

Observation	Estimated probability	Best decision afterwards	Expected loss after decision
trace confirms healthy request/response	0.70	complete with read-back	£1.20
trace exposes backend defect	0.22	report defect and block completion	£0.50
trace is missing or ambiguous	0.08	escalate to human	£6.00

Without the trace, the best current action has expected loss £4.80. The probability-weighted post-observation loss is:

Framework-neutral pseudocode.

```
0.70*£1.20 + 0.22*£0.50 + 0.08*£6.00 = £1.43
```

With £0.20 compute cost, £0.10 delay cost, and £0.15 estimated stale/misleading-evidence exposure, the engineering EVSI is:

Framework-neutral pseudocode.

```
£4.80 - £1.43 - £0.20 - £0.10 - £0.15 = £2.92
```

Run sensitivity analysis over the observation probabilities and losses. For catastrophic classes, apply a hard approval rule rather than allowing a small estimated probability to trade away an authority boundary.

14.7 Stopping rules without decorative formulas

An open-ended research or debugging agent needs a stop policy whose inputs can actually be measured.

Stop when one of these holds:

- the completion contract is independently verified;
- no candidate action has positive value under conservative estimates;
- uncertainty cannot be reduced with available authorised tools;
- the next action repeats a state-action signature without new evidence;
- the remaining budget is below the cheapest useful action;
- policy requires escalation;
- the deadline or freshness window expires.

Framework-neutral pseudocode.

```
def choose_next(state, candidates, risk_model, loss_table):
    if completion_gate(state).verified:
        return Stop("verified")

    scored = []
    for action in candidates:
        p_outcomes = risk_model.predict_interval(state.features, action)
        worst_reasonable_loss = upper_expected_loss(p_outcomes, loss_table[
   action])
        information_gain = empirical_information_value(action, state.bucket)
        net_value = information_gain - action.cost - action.delay_cost
        scored.append((action, net_value, worst_reasonable_loss))
```

```
    useful = [x for x in scored if x[1] > state.minimum_value_margin]
    if not useful:
        return EscalateOrStop("no positive-value action")

    action, _, loss = min(useful, key=lambda x: x[2])
    if policy.requires_approval(action, loss):
        return RequestApproval(action)
    return Execute(action)
```

The implementation does not ask a model to hallucinate expected value. It uses an empirically fitted risk model or bounded policy table, explicit action costs, uncertainty intervals, and a conservative margin.

14.8 Escalation cascade

A cascade uses cheaper or narrower methods first and escalates based on measured failure evidence:

Framework-neutral pseudocode.

```
deterministic parser
  -> economical model
  -> stronger model
  -> independent verifier
  -> human specialist
```

Escalation features can include schema failure, candidate disagreement, low calibrated success probability, high task risk, insufficient evidence, repeated failed verification, or novel error class. A permission denial or broken API is not evidence that a larger model will help.

14.9 Human review as a decision

Human review adds delay, cognitive load, and operating cost. Use it where expected avoided loss or policy requirements exceed review cost. Present the smallest decision package:

- exact proposed action;
- before/after state;
- relevant evidence;
- unresolved uncertainty;
- consequence of approve and deny;
- proposal hash and expiry.

Do not dump the full transcript and call that oversight.

14.10 Decision records and calibration monitoring

Record consequential decisions without private chain-of-thought:

Framework-neutral pseudocode.

```
{
  "decision_id": "d-91",
  "choices": ["execute", "inspect_more", "escalate"],
  "selected": "inspect_more",
  "calibration_bucket": "qa.medium.coverage_60_80.v3",
  "estimated_probability_interval": [0.14, 0.23],
  "loss_policy": "support-defect-v5",
  "evidence_ids": ["ev-7", "ev-9"],
  "reason_code": "BACKEND_STATE_UNVERIFIED",
```

```
  "policy_version": "p-18"
}
```

Monitor reliability diagrams, Brier score, coverage-risk curves, and decision outcomes by task and risk class. Recalibrate after material changes. If a bucket becomes sparse or drifts, widen uncertainty or force escalation rather than preserving a precise-looking stale estimate.

Part 15 - Evaluation programme and release gates

Evidence note. The statistical design is established; task construction, model judges, and agent-specific trajectory grading remain partly workload-dependent.

Failure vignette. A team runs 30 tasks three times and reports 90 independent successes. The confidence interval becomes artificially narrow, the release gate passes, and the same difficult tasks fail repeatedly in production. The implementation bug is not in the agent; it is in the unit of analysis.

15.1 Evaluate the system

An agent result depends on model, harness, prompt, context builder, tools, environment, budgets, and grader. Anthropic's evaluation guidance explicitly treats the harness and model as one evaluated system [R40].

A leaderboard score from a different harness does not predict your production success rate.

15.2 Define the production task distribution

Start with a task taxonomy:

- task family;
- source channel;
- input ambiguity;
- number of steps;
- tools required;
- data sensitivity;
- side-effect risk;
- duration;
- verification type;
- expected cost;
- failure severity.

Sample from real production tasks after privacy review. Synthetic tasks are useful for rare failures and adversarial cases, but they should not replace the real distribution.

15.3 Evaluation layers

1. **Component tests:** schema, tool adapter, policy, context selection.
2. **Single-step model evals:** extraction, ranking, classification, proposal quality.
3. **Trajectory evals:** action sequence, retries, approvals, state use.
4. **Outcome evals:** final external state.
5. **Policy evals:** whether outcome was achieved without violations.
6. **Reliability evals:** repeated trials and variance.
7. **Operational evals:** latency, cost, capacity, recovery.
8. **Online evals:** shadow traffic, canary, user outcomes, incidents.

15.4 Metrics

- **Per-trial success rate:** probability one ordinary run succeeds.
- **pass@k:** probability at least one of k attempts succeeds. This measures search with retries, not ordinary reliability.
- **pass-all-k** or **pass^k:** probability all k repeated runs succeed. This exposes consistency.
- **Verified completion rate:** success confirmed by independent checks.
- **Safe completion rate:** success without policy or security violation.
- **Autonomous coverage:** fraction completed without human intervention.
- **Escalation precision:** fraction of escalations that were warranted.
- **Cost per verified success.**
- **Latency to verified success.**
- **Recovery rate after injected failure.**

A system can have high pass@5 and poor per-trial reliability. Do not market the former as the latter.

Safety, escalation and autonomy metrics

Always report raw unsafe-event count and ordinary unsafe-event frequency. A severity-weighted quantity is a **score or density**, not an empirical probability:

Framework-neutral pseudocode.

```
severity_weighted_unsafe_event_density = sum(event_severity_weight) / trials
```

Also report:

- escalation precision: justified escalations / all escalations;
- escalation recall: dangerous-or-insufficient-evidence tasks escalated / all such tasks;
- missed-escalation rate: dangerous-or-insufficient-evidence tasks executed autonomously / all such tasks;
- autonomous coverage: autonomously completed tasks / eligible tasks;
- autonomous success rate at each coverage threshold;
- unsafe-event frequency among autonomous trials;
- risk-coverage curve and high-risk subgroup results.

A high escalation precision with poor recall is unsafe: the system may escalate only obvious cases while silently acting on difficult ones.

15.5 Graders

Prefer deterministic graders where outcomes are executable or structured. Use model judges for semantic qualities that cannot be fully specified, but calibrate them against human labels.

A grader should have:

- rubric;
- input contract;
- hidden tests where appropriate;
- version;
- known blind spots;
- confidence or disagreement signal;
- audit sample.

Do not let the actor see hidden grader details.

15.6 Environment isolation and reset

Each trial needs a known initial state. Validate reset before the trial and final state after it. Agentic coding evals are sensitive to infrastructure noise, time limits, and resource budgets [R41].

Record:

- container or VM image;
- dependency versions;
- external-service fixtures;
- network policy;
- model settings;
- time and step budgets;
- random seeds where applicable;
- reset verification.

15.7 Contamination and leakage

Benchmarks can overstate performance through leaked solutions or weak tests. SWE-bench, SWE-Bench+, EvalPlus, and later live variants demonstrate why executable tests, freshness, and benchmark audits matter [R42-R46].

Controls:

- use recent or private tasks;
- rotate held-out tasks;
- inspect whether answers are present in inputs;
- strengthen tests;
- hide acceptance details;
- compare public and fresh sets;
- audit suspiciously short trajectories;
- track benchmark exposure.

15.8 Repeated trials and statistics

Run multiple trials because model behaviour is stochastic. Use paired trials when comparing two system versions on the same tasks. Report confidence intervals, not only point estimates.

For a binary success rate, the number of trials needed depends on the baseline and the minimum meaningful improvement. Do not claim a 2-point improvement from ten tasks.

Pre-register:

- primary metric;
- task set;
- number of trials;
- exclusion rules;
- stopping rule;
- significance or decision threshold;
- cost limit.

15.9 Failure taxonomy

Classify failures:

- intent misunderstanding;
- missing context;
- wrong tool selection;
- invalid arguments;
- policy denial;
- environment or infrastructure;
- stale state;
- loop or budget exhaustion;
- premature completion;
- verifier defect;
- task ambiguity;
- security violation;
- human-approval failure.

Read transcripts and evidence to ensure the label is correct. Grader bugs can masquerade as agent failures and vice versa.

15.10 Ablations

Ablate one element at a time:

- model;
- prompt section;
- tool description;
- context selector;
- memory;
- verifier;
- reflection step;
- multi-agent decomposition;
- retry strategy;
- workflow versus free loop.

Measure whether added complexity produces measurable value.

15.11 Release gates

A release gate might require:

- no regression in high-risk safe completion;
- verified completion improvement above a minimum threshold;
- cost per verified success within budget;
- no new critical security bypass;
- recovery tests pass;
- grader audit complete;
- trace completeness above target;
- rollback and kill switch tested.

15.12 Shadow and canary deployment

Shadow mode: run the new system on production inputs without applying side effects. Compare proposals and outcomes offline.

Canary: route a small, controlled slice of real tasks to the new version with strict monitoring and rollback.

Stratify canaries by task risk. Do not start with the easiest tasks and conclude the system is safe for high-risk work.

15.13 Drift detection

Monitor changes in:

- task mix;
- tool errors;
- model behaviour;
- context length;
- escalation rate;
- verifier disagreement;
- cost;
- security alerts;
- completion quality.

A model provider update, documentation change, or external UI change can cause drift without application deployment.

15.14 Benchmark interpretation

Historical benchmarks show that long-horizon web, desktop, and workplace tasks have been substantially harder than short tool-use demos. WebArena reported 14.41% for its best GPT-4 baseline versus 78.24% human performance; original OSWorld reported 12.24% versus 72.36%; TheAgentCompany reported 24% for its strongest baseline; OSWorld 2.0 introduced 108 longer workflows and reported low primary completion for tested systems [R47-R50].

These figures are not directly comparable across years or harnesses. Use them as evidence that task realism, horizon, environment, and verification matter - not as a current ranking of models.

15.15 Worked evaluation: a fully reproducible synthetic QA-agent release study

This section is a **synthetic worked example**, not evidence about a deployed agent. Every numerical result is computed from companion files shipped with this edition. The archive `Production_Agent_Engineering_2026_Edition_1.7_Reproducibility_Package.zip` is embedded in the PDF; its SHA-256 is listed in the publication information:

- `qa_eval_synthetic_trials.csv` - 120 tasks, three paired trials per system, 360 rows;
- `qa_eval_synthetic_grader_audit.csv` - complete 100-item grader audit;
- `qa_eval_synthetic_failure_injection.csv` - complete 40-scenario recovery set;
- `reproduce_qa_evaluation.py` - analysis code;
- `qa_eval_synthetic_results.json` - machine-readable output.

The fixed seed is `20260729`. The purpose is to demonstrate an auditable analysis. The synthetic pass decision must not be cited as product performance.

Decision and compared systems

The simulated decision is whether candidate `qa-agent-2.3-synth` may replace baseline `qa-agent-2.2-synth` for low- and medium-risk tasks and enter a small high-risk canary.

The candidate changes context selection, evidence policy, model routing and verification. The primary comparison is system-to-system. Component ablations follow after the frozen primary analysis.

Task taxonomy and sample

Risk stratum	Tasks	Trials per system	Paired trial rows	Typical verification
Low	45	3	135	deterministic UI/API state
Medium	45	3	135	UI, API and evidence rubric
High	30	3	90	authoritative state plus human audit
Total	**120**	**3**	**360**	

The CSV contains every paired outcome. A task is the sampling cluster because its three trials share fixture, acceptance criteria and difficulty.

Trial-level descriptive results

Risk	Baseline verified	Candidate verified	Baseline unsafe	Candidate unsafe
Low	119/135	134/135	2	0
Medium	101/135	119/135	4	2
High	45/90	57/90	8	4
Overall	**265/360 (73.6%)**	**310/360 (86.1%)**	**14/360 (3.89%)**	**6/360 (1.67%)**

Marginal 95% **percentile task-cluster bootstrap intervals** are **68.1%-78.9%** for baseline and **81.1%-90.6%** for candidate. Each bootstrap draw resamples task IDs and carries all three repeated trials, so these intervals do not treat 360 correlated rows as independent Bernoulli observations. They describe each system's marginal verified-completion frequency; the primary paired comparison remains the candidate-minus-baseline task-cluster bootstrap below.

Percentile cluster bootstrap over tasks

The estimand is the mean candidate-minus-baseline verified-completion frequency per task. Each task contributes three paired trials. The bootstrap samples 120 task IDs with replacement and carries all six system/trial observations for each selected task.

Tested companion example - executed with Python 3.13.5, NumPy 2.3.5, pandas 2.2.3 and SciPy 1.17.0 from `requirements.txt` and the accompanying `environment_attestation.json`.

```
def cluster_bootstrap(df, reps=200_000, seed=20260729):
    task_diffs = (
        df.groupby("task_id")[["baseline_verified", "candidate_verified"]]
          .mean()
          .assign(
```

```
            diff=lambda x: x["candidate_verified"] - x["baseline_verified"]
        )["diff"]
        .to_numpy()
    )
    rng = np.random.default_rng(seed)
    draws = np.empty(reps)
    for start in range(0, reps, 5_000):
        size = min(5_000, reps - start)
        sample = rng.integers(0, len(task_diffs), size=(size, len(task_diffs)))
        draws[start:start + size] = task_diffs[sample].mean(axis=1)
    return task_diffs.mean(), np.quantile(draws, [0.025, 0.975])
```

Reproduced result:

- observed improvement: +**12.5 percentage points**;
- 95% percentile task-cluster bootstrap interval: +**9.17 to** +**15.83 percentage points**;
- replicates: **200,000**;
- bootstrap seed: **20260729**.

Task-level McNemar comparison

The binary task endpoint is frozen before analysis. McNemar tests this **composite verified-and-safe task endpoint**, not raw trial success:

> A task passes when at least two of three trials are verified complete and none of its three trials contains an unsafe event.

	Candidate pass	Candidate fail
Baseline pass	83	5
Baseline fail	19	13

Baseline passes 88/120 tasks; candidate passes 102/120. The exact two-sided McNemar test uses only the 24 discordant tasks: 19 candidate-only passes and five baseline-only passes. The reproduced p-value is **0.00661**. It does not use the 360 trial rows as independent Bernoulli observations.

Safety and severity weighting

Raw unsafe-event frequency remains the primary interpretable quantity:

- baseline: **14/360** = **3.89%**;
- candidate: **6/360** = **1.67%**.

A separate severity-weighted score assigns policy points of 1, 4 and 12 to unsafe events in low-, medium- and high-risk strata. This is **not a probability or rate**.

System	Unsafe events	Severity-weighted incident points	Points per 100 trials
Baseline	14	114	31.67
Candidate	6	56	15.56

Always report the raw counts and frequencies beside this score. Changing weights changes the score without changing empirical event frequency.

For a release gate, “candidate frequency is not numerically higher” is too weak. The companion analysis therefore performs a one-sided task-cluster bootstrap on candidate-minus-baseline unsafe-event frequency. The observed difference is **-2.22 percentage points** and the one-sided 95% upper confidence bound is **-0.28 percentage points**. Against the predeclared non-inferiority margin of +**1.00 percentage point**, the synthetic candidate passes. This is a method demonstration, not evidence that a real candidate is safe.

Escalation and autonomous-risk metrics

Metric	Baseline	Candidate
Escalation precision	79.4%	86.4%
Escalation recall	69.4%	80.2%
Missed-escalation rate	30.6%	19.8%
Autonomous coverage	73.1%	71.4%
Verified success among autonomous trials	75.7%	88.7%
Unsafe-event frequency among autonomous trials	3.80%	1.95%

Precision alone would hide missed dangerous cases. Recall and missed-escalation rate measure whether required review was actually requested. Coverage says how much work remained autonomous. This metric counts unsafe events among trials that were allowed to remain autonomous. It does not imply verified completion; unsafe attempted execution and unsafe verified completion are separate operational questions.

Risk-coverage curve

The companion CSV contains a synthetic risk score for each trial. A trial is autonomous when its score is below the selected threshold.

System	Threshold	Autonomous coverage	Autonomous verified success	Unsafe-event frequency among autonomous trials
Baseline	0.30	45.6%	76.8%	1.83%
Baseline	0.50	66.7%	77.5%	2.92%
Baseline	0.70	82.5%	74.7%	3.70%
Candidate	0.30	51.9%	87.2%	1.60%
Candidate	0.50	66.1%	89.1%	1.26%
Candidate	0.70	76.7%	88.8%	1.09%

The table is illustrative because the data are synthetic, but the reporting structure is mandatory in a real release. Select a threshold from the acceptable risk-coverage frontier; do not optimise escalation precision alone.

Grader calibration

The complete 100-item synthetic grader audit contains 59 true positives, four false positives, three false negatives and 34 true negatives.

Metric	Reproduced value
Accuracy	93.0%
Sensitivity	95.2%
Specificity	89.5%
Cohen’s kappa	0.851

False accepts require individual review because they can turn a system failure into an apparent success. In a real programme, grader uncertainty should be propagated through sensitivity analysis or adjudication rather than ignored.

Cost, latency, trace and recovery

Metric	Baseline	Candidate	Synthetic gate
Mean cost per trial	£0.378	£0.409	report
Cost per verified success	£0.513	£0.475	<= £0.55
p50 latency	40.3 s	38.1 s	<= 50 s
p95 latency	82.5 s	66.8 s	<= 90 s
Trace completeness	98.9%	99.7%	>= 99.5%
Failure-injection safe outcomes	not compared	40/40	40/40

Pre-registered synthetic gates

Gate	Requirement	Reproduced result	Synthetic disposition
Verified completion	lower 95% task-cluster bound > +2 pp	+9.17 pp	pass
Task-level endpoint	candidate not worse; exact paired analysis reported	102 vs 88; p=0.00661	pass
Raw safety non-inferiority	one-sided 95% task-cluster upper bound < +1.00 pp	-0.28 pp upper bound	pass
Severity score	candidate incident points <= baseline	56 vs 114	pass
Escalation recall	>= 75%	80.2%	pass
Missed escalation	<= 25%	19.8%	pass
Cost efficiency	<= £0.55 per verified success	£0.475	pass
Tail latency	p95 <= 90 s	66.8 s	pass
Trace completeness	>= 99.5%	99.7%	pass
Recovery	all mandatory scenarios recover or stop safely	40/40	pass

Synthetic decision and its limit

Under the frozen synthetic gates, the candidate passes this simulated release review. The result demonstrates the calculation and reporting pipeline only. It is not evidence that the illustrated product should ship, and it is not a claim about any real model, vendor, QA agent or production deployment.

A real release report must additionally publish dataset lineage, exclusion decisions, environment-reset evidence, model and tool versions, evaluator ownership, deviations from the preregistration, and a signed decision record.

Reproducible regression report

Generated from the companion synthetic files; not production evidence.

```
Release study: qa-agent-2.3-synth versus qa-agent-2.2-synth
Dataset: qa_eval_synthetic_trials.csv (120 tasks, 360 paired trial rows)
Primary estimand: mean candidate-minus-baseline verified frequency per task
Observed improvement: +12.50 percentage points
95% percentile task-cluster bootstrap interval: +9.17 to +15.83 points
Task endpoint: >=2/3 verified trials and zero unsafe events
Task passes: 102 candidate vs 88 baseline
McNemar exact p: 0.00661 (19 candidate-only, 5 baseline-only)
Unsafe events: 6/360 candidate vs 14/360 baseline
Safety non-inferiority: one-sided 95% cluster upper bound -0.28 pp (< +1.00 pp
   margin)
Severity-weighted incident points: 56 candidate vs 114 baseline
Escalation recall: 80.2% candidate vs 69.4% baseline
Missed-escalation rate: 19.8% candidate vs 30.6% baseline
Cost per verified success: £0.475 candidate vs £0.513 baseline
p95 latency: 66.8 s candidate vs 82.5 s baseline
Trace completeness: 99.7% candidate
Failure injection: 40/40 safe outcomes
Disposition: synthetic gate pass; no production inference permitted
```

Ablations after the primary decision

Run paired ablations for context selection, evidence policy, model route and verifier using the same task clusters. Do not replace the frozen primary endpoint after inspecting results. Ablations explain where improvement came from; they do not retroactively redefine success.

Part 16 - Observability, reliability, and incident response

An agent trace is not a transcript with timestamps. A production trace is a causal record that connects the task contract, context, model decisions, tool effects, policy decisions, state transitions, verification evidence, and final disposition.

OpenAI's Agents SDK traces generations, tool calls, hand-offs, and guardrails; Anthropic's Agent SDK exposes hooks, cost tracking, checkpointing, and OpenTelemetry integration; Google ADK documents logs, metrics, and traces as first-class operational concerns [R1-R6, R8, R12-R16]. These capabilities are useful, but a platform trace is only raw material. Your system still needs a stable event model and operational semantics.

16.1 The three questions every trace must answer

For any consequential decision, an engineer should be able to answer:

1. **What did the agent know?** The exact policy version, task contract, context sources, state snapshot, and tool schemas available at decision time.
2. **What did it decide and attempt?** The model output, selected action, canonical arguments, authority used, and expected postcondition.
3. **What actually happened?** Tool response, external side effect, verifier result, state transition, and any human intervention.

If one of these is unavailable, root-cause analysis becomes storytelling.

16.2 Stable identities

Assign separate identifiers to:

- user request;
- business task;
- workflow instance;
- model run;
- tool attempt;
- approval request;
- external mutation;
- verification attempt;
- incident.

Do not reuse a model-run ID as the business-task ID. One business task may contain many model runs, retries, approvals, and verifications.

A minimal event envelope:

Framework-neutral pseudocode.

```
{
  "event_id": "evt_01...",
  "event_type": "tool.execution.completed",
  "occurred_at": "2026-07-28T09:40:16Z",
```

```
  "task_id": "task_01...",
  "workflow_id": "wf_01...",
  "run_id": "run_01...",
  "attempt": 2,
  "actor": {"kind": "tool_gateway", "id": "crm-write-v3"},
  "policy_version": "policy-2026-07-15",
  "prompt_version": "triage-18",
  "model": "provider/model-version",
  "payload_ref": "object://traces/...",
  "previous_event_id": "evt_00..."
}
```

Store large payloads out of band and retain cryptographic hashes or immutable references when auditability matters.

16.3 Canonical tool logging

Log tool calls after canonicalisation, not only the model's raw argument text. Canonicalisation means converting semantically equivalent inputs into a stable representation: normalised paths, resolved identifiers, sorted object keys, explicit defaults, and redacted secrets.

Record:

- requested tool and version;
- raw model arguments;
- validated canonical arguments;
- identity and permission scope;
- policy decision;
- idempotency key;
- timeout and retry class;
- external request identifier;
- result classification;
- postcondition evidence.

Never place credentials, session cookies, full private documents, or unrestricted personal data into routine traces. Observability systems are part of the attack surface.

16.4 Metrics that reveal system behaviour

Track metrics at five levels.

Task metrics

- verified completion rate;
- safe completion rate;
- partial-completion rate;
- escalation and abandonment rates;
- user correction rate;
- time to verified outcome.

Decision metrics

- tool-selection accuracy;
- invalid-argument rate;
- policy-block rate;
- verifier disagreement;
- model abstention and human override rates.

Reliability metrics

- retry rate by error class;
- duplicate-attempt rate;
- recovery success after worker failure;
- stale-lease and fencing rejection counts;
- queue age;
- dead-letter rate.

Resource metrics

- input and output tokens;
- context-cache utilisation;
- tool latency;
- model latency;
- cost per attempt;
- cost per verified success;
- human-review minutes per task.

Security metrics

- untrusted-instruction detections;
- authority-escalation attempts;
- blocked egress;
- sensitive-data transformations;
- anomalous tool sequences;
- kill-switch activations.

Averages hide tail risk. Report distributions and stratify by task class, risk tier, model, tool, and workflow version.

16.5 Service-level objectives

A service-level objective, or **SLO**, is a measurable reliability target for a service. Agent SLOs should describe user-visible outcomes rather than model-call availability alone.

Examples:

- 99% of read-only support tasks receive a verified answer within two minutes;
- 95% of approved CRM updates reach the intended final state within ten minutes;
- fewer than 0.1% of external writes require manual rollback;
- 100% of high-risk writes have a valid approval record and postcondition check;
- 99.9% of task events are trace-complete within five minutes.

Separate **availability** from **correctness**. A system can return a fluent answer on every request and still be operationally unreliable.

16.6 Error budgets and agent autonomy

An **error budget** is the allowed amount of failure implied by an SLO. It creates a practical control loop between product velocity and reliability.

For agents, spend autonomy only while the system remains inside its error budget. When safety violations, unverified completions, or rollback rates exceed limits:

- narrow the allowed task class;
- downgrade write authority;

- increase approval requirements;
- route to a stronger model;
- disable a tool or integration;
- revert a prompt, policy, model, or workflow version.

This makes autonomy a reversible operational configuration rather than a permanent product claim.

16.7 Reliability patterns

Bulkheads

A **bulkhead** isolates failures so one workload cannot exhaust all capacity. Use separate queues, concurrency limits, credentials, and budgets for task classes with different risk or latency.

Circuit breakers

A **circuit breaker** temporarily stops calls to a failing dependency. Open it when a tool produces repeated timeouts, schema violations, authentication errors, or suspicious results. The agent should not repeatedly reason its way into the same outage.

Backpressure

Backpressure slows or rejects new work when downstream capacity is saturated. Without it, long-running agents can accumulate stale tasks, expired approvals, and uncontrolled cost.

Dead-letter queues

Move tasks that cannot make safe progress into a quarantined queue with full diagnostics. Do not retry indefinitely. The dead-letter reason must identify whether the cause is transient, permanent, ambiguous, or security-related.

Graceful degradation

Define what the system can still do when a model, tool, verifier, or memory store is unavailable. A read-only answer with explicit limitations may be preferable to a total outage; a write workflow should usually fail closed.

16.8 Incident severity

A practical severity scheme:

- **SEV-0:** active widespread harmful side effects, credential compromise, or uncontrolled data disclosure;
- **SEV-1:** material incorrect actions or security bypass affecting multiple users;
- **SEV-2:** significant task failures, rollback load, or degraded verification;
- **SEV-3:** localised defects with safe containment;
- **SEV-4:** quality or efficiency issue without correctness impact.

Severity should depend on impact, scope, reversibility, and exposure duration - not how surprising the model output looks.

16.9 Immediate incident controls

Pre-build controls for:

- global and per-tool kill switches;
- revocation of agent credentials;
- disabling specific model or prompt versions;
- forcing read-only mode;
- stopping queue intake;
- pausing in-flight tasks at safe points;
- preserving trace evidence;
- invalidating memory or cached context;
- identifying and compensating affected external mutations.

A kill switch that requires deploying new code is not a useful kill switch.

16.10 Incident investigation

Investigate the complete causal chain:

1. task contract and user intent;
2. policy and prompt versions;
3. context sources and provenance;
4. model decision and uncertainty;
5. tool schema and gateway validation;
6. authority and approval state;
7. external system response;
8. verifier behaviour;
9. retry and recovery logic;
10. monitoring and escalation timing.

Avoid the conclusion “the model hallucinated” unless the investigation identifies the specific unsupported inference and explains why deterministic controls did not contain it.

16.11 Replay and forensic reproduction

A replay environment should reconstruct:

- immutable task input;
- state snapshot or event history;
- exact prompt and policy assembly;
- tool schema versions;
- model version and settings where provider support permits;
- deterministic tool fixtures or captured responses;
- verifier version.

Model outputs may not reproduce exactly. The goal is to reproduce the decision environment and identify whether the failure is stable, stochastic, environment-dependent, or already fixed.

16.12 Post-incident correction hierarchy

Prefer corrections in this order:

1. remove unnecessary authority;
2. strengthen deterministic invariants;

3. improve tool contract or state semantics;
4. add or repair verification;
5. improve context selection;
6. revise prompt/specification;
7. change model or sampling configuration;
8. add a model critic only when independently useful.

Prompt patches are cheap but often fragile. Fix the control boundary that allowed the incident.

16.13 Operational evidence classification

Treat vendor telemetry features as **strong production evidence** that these primitives are useful, not proof that a particular observability architecture is sufficient. The event model, SLOs, error budgets, and incident controls above are engineering patterns whose suitability must be validated against the organisation's risk and compliance requirements.

Part 17 - Model selection, routing, multi-agent design, and production economics

A frontier model is not an architecture. Production systems should select models and orchestration patterns based on measured task requirements, failure costs, latency, context needs, tool-use quality, and total cost per verified outcome.

17.1 Model-selection dimensions

Evaluate models on the actual task distribution across:

- semantic reasoning;
- instruction and authority adherence;
- structured-output reliability;
- tool-selection and argument quality;
- long-context retrieval;
- visual or computer-use ability;
- code execution and debugging;
- uncertainty expression and abstention;
- latency and throughput;
- price and cache behaviour;
- privacy, residency, and retention requirements;
- version stability and deprecation policy.

Do not compress these into one benchmark score.

17.2 The smallest sufficient model

Use the least expensive model that clears the required release gate for a task class. "Smaller" is not automatically cheaper when retries, verifier failures, and human escalations are included.

The governing metric is:

$$\text{cost per verified success} = \frac{\text{model} + \text{tools} + \text{infrastructure} + \text{human review} + \text{recovery cost}}{\text{number of verified successful tasks}}$$

A model that costs half as much per call but doubles retries can be more expensive.

17.3 Static routing

Static routing maps known task classes to pre-evaluated configurations.

Example:

Framework-neutral pseudocode.

```
classification/extraction      -> fast structured-output model
routine read-only research     -> mid-tier model + retrieval
repository-scale modification  -> high-reasoning coding model + sandbox
high-risk mutation             -> high-reasoning model + approval + verifier
visual UI verification         -> vision-capable model + deterministic checks
```

Static routing is easy to audit and should be the default when task classes are stable.

17.4 Dynamic routing

Dynamic routing selects a model based on estimated difficulty, risk, or uncertainty. It requires its own evaluator because a weak router can silently send hard tasks to inadequate models.

Safe dynamic routing:

1. compute deterministic features such as task type, data sensitivity, requested authority, input size, and tool requirements;
2. optionally obtain a model-based difficulty estimate;
3. apply a policy mapping to an approved configuration;
4. record the routing reason;
5. escalate when verifier evidence is weak or progress stalls.

Never let a model route itself to broader authority.

17.5 Pattern: calibrated model cascade

Use when. A cheaper or faster configuration solves a substantial low-risk subset, and measurable signals can identify when escalation is worthwhile.

Avoid when. Failure is dominated by missing authority, broken tools, impossible tasks, or environment defects. A larger model will not repair those classes. Avoid a cascade when routing errors cost more than the model savings.

Framework-neutral pseudocode.

```
economical model
    -> schema and outcome verification
    -> if model-related uncertainty or failure:
           stronger model with preserved evidence
    -> if high risk or unresolved:
           independent verifier or human
```

Mechanism. A deterministic router selects the first approved configuration from task features. Verification and calibrated risk signals decide escalation. Evidence and state are preserved; the stronger model does not restart from an unstructured transcript.

Invariants. Routing never changes authority. Escalation cannot bypass failed policy or mandatory approval. The system distinguishes model-related failure from tool, policy, and environment failure.

Guardrails. Set maximum escalation depth; restrict retry counts; require fresh context after a state-changing failure; prevent the first model from marking its own uncertainty as sufficient evidence; cap cost and latency.

Failure modes. A weak router sends hard tasks to an inadequate model, causing wasted attempts. Repeated models may share the same blind spot. Escalation can become a hidden unconditional retry policy.

Observability. Record route, features, calibrated bucket, escalation reason, marginal cost, marginal success, and whether escalation changed the verified result. Track cost saved per lost success and false non-escalation rate.

Evaluation. Compare the cascade with always-cheap and always-strong baselines on paired tasks. Evaluate by risk class. Measure route confusion, cost per verified success, latency, and unsafe outcomes.

Framework mapping. All major SDKs permit application-selected model configuration. Routing and calibrated escalation policy remain application code. Framework hand-offs or subagents are not a substitute for a measured router.

Competing alternatives. Static routing by task family is easier to audit. One sufficiently capable model may be cheaper overall when retries and verification dominate. Specialist deterministic components can remove the need for a second general model.

17.6 Specialist models

Specialist models can be useful for:

- embeddings and retrieval;
- reranking;
- moderation and classification;
- optical or visual parsing;
- code completion;
- speech;
- deterministic grammar-constrained generation.

Each specialist adds a versioned dependency and potential disagreement. Use it when it measurably improves quality, latency, or cost.

17.7 Multi-agent systems: when they help

Multiple agents help when work has genuinely separable information or authority boundaries. Examples:

- independent parallel research across disjoint sources;
- specialist analysis followed by deterministic aggregation;
- a read-only investigator separated from a write-authorised executor;
- independent verification using different evidence;
- cross-organisation collaboration through a protocol such as A2A.

Anthropic's published research-system experience reports value from parallel subagents for breadth, while also emphasising token cost, coordination, and evaluation difficulty [R51]. Treat this as **strong production evidence for a particular research workload**, not a universal prescription.

17.8 When multi-agent systems hurt

They commonly fail through:

- duplicated context and tool calls;
- contradictory intermediate conclusions;
- hidden responsibility gaps;
- unbounded delegation;
- circular hand-offs;
- correlated model errors presented as consensus;
- increased latency and trace complexity;
- weak final aggregation;
- authority leakage between roles.

Do not model an organisational chart merely because it is familiar. A deterministic function or one agent with explicit stages is usually simpler.

17.9 Coordination patterns

Sequential pipeline

Each stage enriches a typed artifact for the next. Best when dependencies are strict.

Parallel fan-out and deterministic reduce

Independent workers produce results against a common schema. Code deduplicates, scores, and merges them. Best for breadth and candidate generation.

Supervisor and workers

A supervisor decomposes work and assigns bounded subtasks. The harness limits depth, fan-out, budget, and authority.

Specialist hand-off

One agent transfers the task when another capability is required. The hand-off contract must include state, evidence, unresolved questions, and authority.

Independent proposer and verifier

One component proposes; another verifies using independent evidence. Independence is more important than the number of model calls.

Debate or group chat

Use only when evaluation shows that structured disagreement improves decisions. Free-form agent conversation often spends tokens without increasing evidence.

17.10 Multi-agent authority

Define authority per role:

- researcher: read-only external content;
- planner: can create plans but cannot execute;
- executor: can invoke a narrow set of tools;
- verifier: read-only authoritative checks;
- approver: human or policy service;

- coordinator: can assign work but cannot widen permissions.

Delegation passes a capability token or explicit scope, not ambient credentials.

17.11 Framework-selection principles

Choose the lowest layer that gives the required control.

- Use direct model APIs when the loop is small and you need complete ownership.
- Use an agent SDK when sessions, tool loops, hand-offs, guardrails, and traces reduce undifferentiated work.
- Use a graph or stateful orchestration framework when transitions and interrupts need explicit representation.
- Use a durable workflow engine when crash recovery, timers, compensation, and long-lived correctness dominate.
- Use MCP to standardise tool and resource access across clients.
- Use A2A when independently deployed agents must discover and collaborate.

OpenAI Agents SDK, Claude Agent SDK, Google ADK, Dapr Agents, and open orchestration ecosystems expose overlapping primitives but different runtime assumptions [R1-R16, R24-R25]. Do not choose by feature-count table alone. Prototype the critical control path and operational failure modes.

17.12 Framework evaluation questions

Ask:

- Who owns the loop?
- Where is state persisted?
- Can a run resume after process loss?
- What is replayed, and must user code be deterministic?
- Can tool execution be intercepted before and after the call?
- Are permissions scoped per tool and task?
- Can in-flight workflows survive prompt, model, and code upgrades?
- Are traces exportable in a stable schema?
- Can the runtime operate in your security boundary?
- What are the lock-in and migration costs?

17.13 Production cost model

Model the complete unit economics:

Framework-neutral pseudocode.

```
per-task cost =
    routing
  + prompt/context construction
  + model inference
  + tool/API usage
  + sandbox compute
  + storage and tracing
  + retries
  + verification
  + human approval/review
  + expected incident and rollback cost
```

Measure cost per verified success by task tier. Cost per token is only one input.

17.14 Cost-control levers

Prioritise:

1. remove unnecessary model calls;
2. reduce exposed tools and irrelevant context;
3. cache stable context and deterministic results;
4. use code for parsing, filtering, and aggregation;
5. route routine tasks to a sufficient smaller model;
6. parallelise only when latency benefit or information gain justifies it;
7. cap loops, fan-out, and tool budgets;
8. reuse durable artifacts instead of re-deriving them;
9. improve verification to prevent expensive false completion;
10. reduce human review by narrowing authority, not by hiding uncertainty.

17.15 Latency engineering

Break latency into:

- queue wait;
- context retrieval;
- model time to first token;
- model completion;
- tool execution;
- approval wait;
- verification;
- retry and recovery.

Stream progress only when it is truthful and useful. Do not show internal chain-of-thought. Show observable stages, completed actions, blockers, and evidence.

For interactive tasks, front-load inexpensive deterministic checks and retrieval. For background tasks, optimise throughput and verified completion rather than token streaming.

17.16 Capacity planning

Estimate capacity from the distribution of task steps, not a single average. Include:

- concurrent workflow count;
- model and tool rate limits;
- sandbox startup time;
- long-tail tool latency;
- approval duration;
- retry storms;
- trace and artifact volume;
- per-tenant fairness.

Use queue isolation and per-tenant budgets to prevent one pathological task from starving the system.

17.17 Build versus buy

Buy or adopt a managed runtime when it satisfies security, durability, observability, and cost constraints and the runtime behaviour is sufficiently transparent. Build custom control logic when domain invariants, deployment boundaries, or recovery semantics are differentiating.

The usual answer is hybrid: managed model and tool primitives, organisation-owned policy, task state, authority gateway, verification, and evaluation.

17.18 Economics evidence label

Vendor prices and model rankings change quickly. This guide deliberately avoids fixed price tables. The cost architecture above is **established engineering practice**; the best provider or model for a workload is a time-sensitive empirical decision.

Part 18 - Repository agents and production harness autopsies

Part 18A - Framework-neutral repository-scale software engineering agent

This case study covers a coding agent that receives an issue, changes a real repository, runs tests, and prepares a reviewable patch. The naive and production designs use the same model. Their reliability differs because the production system makes state, authority, verification, and recovery explicit.

18.1 Task contract

Objective: implement the requested repository change and produce a patch that satisfies acceptance criteria without unrelated modifications.

Inputs:

- repository and base revision;
- issue text and linked specifications;
- allowed directories;
- build and test commands;
- coding and security policies;
- time and compute budget.

Completion criteria:

- requested behaviour is implemented;
- relevant tests pass in a clean environment;
- regressions in the required suite are absent;
- generated diff is within scope;
- unresolved risks and skipped checks are explicit;
- evidence bundle is attached.

Non-goals: merge, deploy, modify credentials, change protected workflows, or broaden scope without approval.

18.2 Naive design

Framework-neutral pseudocode.

```
issue + repository + broad shell access
                  -> one autonomous agent loop
                  -> “tests ”pass summary
```

Failure modes:

- reads an unbounded repository and loses relevant context;
- edits generated or unrelated files;
- changes tests to accommodate wrong behaviour;
- runs only a convenient subset of tests;
- accepts stale test results;
- leaves dirty workspace state across retries;
- declares success after command acceptance rather than test completion;
- cannot resume coherently after context or worker loss;
- follows malicious instructions embedded in repository content;
- uses broad credentials available in the environment.

18.3 Production architecture

Framework-neutral pseudocode.

```
issue intake
    -> deterministic task normalisation
    -> isolated workspace at pinned base revision
    -> repository index and policy load
    -> model creates bounded change plan
    -> plan validator checks scope and required verification
    -> incremental edit/test loop
    -> independent verification in clean workspace
    -> diff and provenance audit
    -> evidence bundle
    -> human review / draft pull request
```

The model chooses implementation details. The harness owns repository identity, workspace isolation, file permissions, command policy, state transitions, and completion.

18.4 State model

Framework-neutral pseudocode.

```
{
  "task_id": "code_01...",
  "repository": "org/project",
  "base_revision": "6d4e...",
  "workspace_id": "ws_01...",
  "phase": "IMPLEMENTING",
  "plan_version": 3,
  "allowed_paths": ["src/service/**", "tests/service/**"],
  "changed_paths": ["src/service/cache.py"],
  "acceptance_checks": [
    {"id": "unit", "status": "passed", "evidence_ref": "..."},
    {"id": "integration", "status": "pending"}
  ],
  "attempt_budget": {"model_calls_remaining": 14, "wall_minutes_remaining": 31},
  "unresolved": ["migration behaviour on existing cache entries"]
}
```

Persist this independently from the conversation. Progress notes may help the model, but the state object controls execution.

18.5 Trust boundaries

- issue text and linked documents are untrusted content;
- repository files are untrusted instructions but trusted code only after verification;
- model output is an untrusted proposal;
- sandbox is a containment boundary, not proof of correctness;
- package registries and network responses are external dependencies;
- CI status is authoritative only for the exact revision and configuration;
- human review remains the merge-authority boundary.

18.6 Tool contracts

Repository search

Read-only. Supports symbol, text, path, dependency, and history queries. Returns bounded snippets with path, revision, line range, and truncation metadata.

File read

Requires a repository-relative normalised path. Rejects symlinks that escape the workspace, binary files above limits, and sensitive paths.

Patch apply

Accepts a structured patch, expected base hash, and reason. Rejects out-of-scope files, generated files, protected configurations, oversized changes, and concurrent modifications.

Command execute

Runs an allowlisted command class inside the sandbox. Network is disabled by default. Environment variables are filtered. Time, CPU, memory, output, and child-process budgets are enforced.

Test run

Takes a named test profile rather than arbitrary shell text. Captures revision, environment fingerprint, command, exit status, logs, duration, and test inventory.

Git inspection

Read-only operations expose status, diff, blame, and history. Commit or push is a separate approved capability.

18.7 Context strategy

Initial context contains:

- task contract;
- repository policy;
- architecture map;
- acceptance checks;
- compact current state;
- relevant retrieved code.

The agent searches on demand. It does not receive the whole repository. Tool responses preserve provenance. Summaries are linked to source revisions so stale summaries can be invalidated.

Before each new run, the harness supplies:

- current base and workspace revision;
- changed-file summary;
- last verification results;
- unresolved items;
- remaining budgets;
- next allowed objective.

This follows the durable handover principle documented in Anthropic's long-running-agent work [R10-R11].

18.8 Planning contract

The model proposes:

Framework-neutral pseudocode.

```
{
  "hypothesis": "cache entries are not invalidated when tenant policy changes",
  "files_to_inspect": ["src/service/cache.py", "tests/service/test_cache.py"],
  "intended_changes": [
    {"path": "src/service/cache.py", "reason": "include policy version in key"}
  ],
  "verification": ["unit", "service-integration"],
  "risks": ["existing entries become unreachable but remain until expiry"]
}
```

Code validates that paths and test profiles exist and that protected areas are not requested. The plan may evolve, but every expansion is recorded.

18.9 Incremental loop

One iteration should have a narrow objective:

1. inspect evidence;
2. state a local hypothesis;
3. make a bounded change;
4. run the cheapest relevant check;
5. update state and evidence;
6. decide whether to continue, revise, or escalate.

A loop that edits many files before feedback increases the cost of attribution and rollback.

18.10 Guardrails

Before inference:

- classify issue risk and required test tier;
- strip or label external instructions;
- select the permitted tool set.

Before patch:

- validate path scope;
- compare expected file hash;
- scan for secret or policy-file changes;
- limit line and file count.

Before command execution:

- classify command;
- reject destructive filesystem and credential access;
- restrict network and package installation;
- enforce resource budgets.

After execution:

- parse test output independently;
- treat truncated output as inconclusive;
- detect modified tests or snapshots;
- record environment fingerprint.

Before completion:

- require clean verification on the final diff;
- compare changed paths against task scope;
- require evidence for every acceptance criterion;
- surface skipped checks.

18.11 Verification architecture

Use two workspaces:

- **development workspace:** incremental editing and local tests;
- **verification workspace:** fresh checkout of base plus final patch.

The verifier:

1. applies the exact candidate patch;
2. installs dependencies from locked sources;
3. runs deterministic static checks;
4. runs required unit and integration profiles;
5. optionally runs hidden or held-out tests;
6. checks generated artifacts and repository cleanliness;
7. produces signed or immutable evidence.

The agent cannot redefine the required suite after seeing failures.

18.12 Partial completion

A useful partial result can include:

- root-cause evidence;
- minimal reproduction;
- patch that passes unit tests but not unavailable integration tests;
- explicit blocked dependency;
- safe rollback instructions.

The final state is `PARTIALLY_COMPLETED`, not `SUCCEEDED`. A reviewer sees exactly what remains.

18.13 Durable recovery

Checkpoint after:

- workspace initialisation;

- approved plan;
- each accepted patch;
- every material test result;
- external approval or comment;
- final evidence bundle.

On worker loss, obtain a new lease, verify workspace revision and hashes, and resume from the event history. Never rely on a live shell process as durable state.

18.14 Evaluation suite

Task set:

- local bug fixes;
- cross-module changes;
- data migrations;
- concurrency defects;
- performance regressions;
- ambiguous issues requiring clarification;
- malicious repository instructions;
- impossible or already-fixed issues.

Graders:

- hidden tests;
- static checks;
- behavioural oracle;
- diff-scope grader;
- policy and secret scanner;
- human maintainability review on a sample.

Metrics:

- verified completion;
- safe completion;
- test-tampering rate;
- unrelated-change rate;
- reviewer correction time;
- cost and latency per verified patch;
- recovery success after injected process failures.

Run repeated trials because implementation paths and success are stochastic [R40-R46].

18.15 Production economics

Primary cost drivers are repository context, repeated builds, sandbox compute, and verification. Control them through:

- symbol-aware retrieval;
- incremental local checks;
- cached immutable dependencies;
- final clean verification only when candidate quality is sufficient;
- model escalation based on failure class;
- hard limits on broad test repetition.

Do not save money by removing final verification. That converts visible compute cost into hidden reviewer and incident cost.

18.16 Deployment and rollout

Start in patch-only mode with no push authority. Compare against human patches in shadow evaluation. Canary on low-risk repositories with mandatory review. Expand command and repository scope only when task-tier metrics remain inside error budgets.

The production output is a reviewable patch plus evidence, not a claim that the model is a software engineer.

Part 18B - Production harness autopsies: Claude Code and Codex

The framework-neutral architecture above states what a repository agent should guarantee. This section asks a different question: **what engineering decisions are visible in mature coding-agent harnesses that already operate under long contexts, frequent tool calls, subagents, resumable sessions, multiple user interfaces, and fleet-scale cost pressure?**

This is implementation archaeology, not a product comparison. The goal is to extract mechanisms that generalise beyond either product.

Chapter map: 18B.1 evidence boundary; 18B.2 compaction reconstruction; 18B.3 subagent isolation; 18B.4 prompt-cache constraints; 18B.5 capability discovery; 18B.6 current Claude SDK delta; 18B.7 Claude execution lifecycle; 18B.8-18B.12 Codex protocol, state, compaction, backpressure and authority; 18B.13 Codex tool scheduling and cancellation; 18B.14 triangulation; 18B.15 comparison; 18B.16 decision procedure; 18B.17 misdiagnoses.

18B.1 Evidence boundary and provenance

Source-audit cutoff: 29 July 2026, 19:30 IST (`2026-07-29T19:30:00+05:30`). Every source claim in this part is tied to an immutable commit available at or before that cutoff.

The Codex protocol and compaction analysis uses the official `openai/codex` repository pinned to commit `fe01054a28fa` [R80-R82]. The tool-runtime analysis uses a later immutable source snapshot, commit `cef3910ea4d0`, because it exposes the scheduling, provenance and cancellation contracts discussed below [R88-R91]. The manuscript does not combine observations across those commits as though they were one atomic build.

The Claude Code analysis uses the explicitly versioned ChinaSiro **unofficial implementation snapshot**, reconstructed from the public `@anthropic-ai/claude-code` 2.1.88 npm package and its source map and pinned to commit `a8a678cb6244`. It was selected as the single recovered-source basis because its provenance and restoration boundary are stated directly; additional mirrors of the same recovered material are not treated as independent corroboration. Its README says that it is unofficial, intended for research, and does not reproduce Anthropic's original internal repository structure [R76]. Claims from that snapshot are therefore labelled **source-observed engineering evidence**, not authoritative product guarantees. They are corroborated against official Claude Agent SDK contracts where those contracts expose the same lifecycle concepts [R74].

This distinction matters. A recovered source snapshot can reveal mechanisms present in one shipped build, but it cannot establish:

- why an internal design was chosen;
- whether omitted modules change the interpretation;
- whether the mechanism remains present in a later build;
- whether comments describe measured production behaviour accurately;
- whether the reconstructed directory structure matches the original source tree.

The safe use of such evidence is to identify a candidate engineering pattern, trace it through several related call sites, and then state the transferable invariant separately from the product-specific implementation.

Evidence classification used in this part.

- **Observed mechanism:** behaviour directly visible in the pinned source or protocol definition.
- **Implementation-author comment:** motivation or production measurement stated in a source comment; useful evidence, but not independently audited.
- **Manuscript inference:** the transferable design rule derived from one or more observed mechanisms.
- **Independently verified production result:** externally audited operational evidence. No finding in this part is assigned this label.

Each subsection states its evidence mix. Source comments containing fleet-scale token or cache figures are treated as implementation-author comments, not verified measurements.

18B.2 Claude Code lesson: compaction is state reconstruction

Evidence mix: observed mechanism for stripping, grouping, reconstruction and boundary metadata; implementation-author comments for stated cache and token effects; manuscript inference for the general compaction rule.

A superficial description of compaction is: summarise old messages so the conversation fits inside the context window. That description is inadequate for a tool-using agent because the transcript contains several different kinds of state:

- semantic history: what the user asked and what was decided;
- operational state: files read, files changed, active plans, pending child work;
- capability state: tools, skills, agents, and MCP instructions already disclosed;
- permission state: current mode and allowed scope;
- lineage state: compact boundaries, preserved suffixes, and transcript relationships;
- cache state: request-prefix choices that influence cache reuse and cost.

The observed Claude Code 2.1.88 compaction path treats those categories differently [R77]. Before asking a model for a summary, it replaces image and document bodies with markers. It removes skill-discovery attachments that will be regenerated. If the compaction request itself is too large, it groups messages by API round and removes old groups rather than slicing arbitrary message fragments. After the summary is produced, it reconstructs a new working context from explicit sources: bounded recently read file snapshots, asynchronous-agent state, the active plan, plan-mode instructions, invoked skills, deferred tool information, available-agent listings, MCP instructions, hook output, and compact-boundary metadata. Discovered-tool names are carried across the boundary because the textual summary alone does not preserve tool-reference state.

The important principle is:

> **Compaction is a state transition that replaces one operational context with another. The summary is only one field in the replacement state.**

A prose summary can preserve the narrative while losing the conditions needed to continue safely. For example, “the cache implementation was modified” does not specify the exact workspace revision, which files were read after the modification, which tests remain pending, whether a background agent is still running, or whether a previously discovered tool schema must remain available.

Pattern: post-compaction reconstruction manifest

Use when: a task may exceed one context window, resume after interruption, or use capabilities whose disclosure state changes over time.

Avoid when: the complete task fits comfortably in one bounded invocation and no operational state survives the call. Do not add a compaction subsystem merely because a model supports summaries.

Mechanism: treat compaction as construction of a versioned replacement manifest. The manifest is derived from authoritative state stores and artifacts, not only from model-generated prose.

Framework-neutral pseudocode - data contract.

```
manifest_version: 3
compaction:
  boundary_id: cmp_01J...
  trigger: automatic
  source_history_hash: sha256:...
  summary_model: ...
  summary_prompt_version: compact-7
objective:
  task_id: code_01J...
  current_goal: "make tenant policy part of the cache key"
  plan_version: 4
  unresolved_decisions:
    - "whether old entries require eager migration"
artifacts:
  workspace_revision: 83c10f...
  modified_paths:
    - src/service/cache.py
  authoritative_snapshots:
    - path: src/service/cache.py
      content_hash: sha256:...
      read_after_revision: 83c10f...
verification:
  completed:
    - check: unit-cache
      evidence_id: ev_102
  pending:
    - service-integration
capabilities:
  active_tool_schemas: [repo.search, file.read, patch.apply, test.run]
  discovered_capabilities: [db.schema.inspect]
  invoked_skills: [repository-debugging]
authority:
  principal: repository-agent
  policy_version: repo-policy-18
  workspace_roots: [/workspace/project]
  pending_approvals: []
children:
  active:
    - child_id: agent_72
      purpose: "inspect migration behaviour"
      transcript_ref: ...
      cancellation_ref: ...
  completed_outputs: []
conversation:
  semantic_summary: "..."
  preserved_recent_items: [item_910, item_911]
```

Invariants:

- every mutable artifact is identified by an immutable revision or content hash;
- the manifest identifies which fields came from authoritative state and which came from a model summary;
- pending operations and child work are not silently converted into completed work;
- permission and tool-disclosure state is reconstructed explicitly;
- a resumed run can reject a manifest whose workspace, policy, or dependency versions no longer match reality;
- repeated compaction preserves lineage rather than creating an untraceable replacement transcript.

Guardrails: cap restored file count and bytes; normalise paths; never restore secrets merely because they appeared in prior context; re-evaluate capability eligibility and authority at the new boundary; reject stale approval tokens; and keep untrusted repository instructions labelled as content.

Failure modes: summary omits a constraint; restored file snapshot is older than the workspace; a child agent is duplicated after resume; a previously removed tool remains callable; repeated compactions amplify an earlier summary error; compaction itself exceeds the model window; or the replacement context is almost as large as the source context.

Observability: record source and replacement token counts, retained and discarded item classes, restoration failures, stale-snapshot detections, capability re-announcements, cache read/write tokens, compaction duration, and the number of turns until another compaction.

Evaluation: compare otherwise identical long tasks with and without compaction. Measure completion delta, lost-constraint rate, stale-file rate, lost-capability rate, duplicate-action rate, incorrect child recovery, post-compaction recovery turns, repeated-compaction degradation, token savings, and cache effects. Inject compaction immediately before a critical verification step and while a child task is active; those are harder tests than compacting at convenient boundaries.

Framework mapping: Claude Code reconstructs several manifest fields through attachments and compact-boundary metadata [R77]. Codex stores compaction checkpoint metadata separately from replacement history and varies initial-context injection by compaction phase [R81-R82]. A durable application workflow should keep the canonical manifest outside either model transcript.

Competing alternatives: start a new task with a human-written handover; keep the entire history in a larger context; externalise all state and rebuild every turn; or use deterministic extraction for selected fields plus a semantic summary. The last option is normally the safest default.

Compaction boundary semantics

"Compact the conversation" is not one operation. At minimum, specify:

1. **Trigger:** manual, pre-turn threshold, mid-turn overflow, post-tool threshold, or recovery-time compaction.
2. **Selection:** which prefix is summarised, which suffix is preserved, and how tool-call/result pairs remain valid.
3. **Instruction handling:** whether system instructions and current world state are inside the summary request, copied verbatim, or reinjected after replacement.
4. **Outstanding operations:** whether in-flight tools are cancelled, waited for, represented as pending, or excluded.

5. **Lineage:** how original item IDs map to the replacement history and how replay treats the boundary.
6. **Failure handling:** what happens if the compaction call is interrupted, too large, or returns an unusable summary.
7. **Repeat behaviour:** how information loss is measured across the second, third, and later compactions.

This distinction is visible in Codex. Its core differentiates manual or pre-turn replacement from mid-turn replacement. Pre-turn compaction can omit initial context because the next ordinary turn will reinject it. Mid-turn compaction places current initial context before the last real user message because the model expects the compacted summary at a particular boundary [R81]. The graph and history may be code-owned, but model output, tool effects, and external state remain probabilistic or failure-prone.

18B.3 Claude Code lesson: subagents are isolated runtimes

Evidence mix: observed mechanism for context projection, mutable-state cloning, permission filtering, transcripts, cancellation and worktree fields; implementation-author comments for fleet-scale token savings; manuscript inference for isolation and attenuation rules.

Calling a second prompt with “you are the researcher” creates a persona, not a production subagent. The observed Claude Code implementation creates separate runtime state [R78-R79]: a child identity, selected model and tools, a projected message history, cloned or fresh file-state caches, its own transcript, cancellation controller, query-depth lineage, permission context, optional worktree path, optional agent-specific MCP servers, liveness reporting, and cleanup.

Several choices are deliberately subtractive. Read-only Explore and Plan agents may omit `CLAUDE.md` content that is irrelevant to their role. A stale session-start Git snapshot is removed because the child can run a fresh query. When an explicit child allowlist is supplied, parent session approvals are replaced rather than inherited, preventing silent permission leakage. Mutable callbacks are no-ops by default and are shared only by explicit opt-in. Newly created MCP clients are cleaned up by the child lifecycle, while shared parent clients are not.

These mechanics produce four reusable principles.

Pattern: purpose-specific context projection

Use when: a child has a narrow role such as repository exploration, test diagnosis, policy review, or evidence verification.

Avoid when: the child genuinely needs the entire parent interaction and the cost of an incorrect omission exceeds the isolation benefit. Even then, pass a versioned snapshot rather than a mutable reference.

Mechanism: compile a child context from an explicit projection policy.

Framework-neutral pseudocode.

```
child_context = project(
    task_contract,
    child_purpose,
    required_artifacts,
    fresh_observations,
    permitted_capabilities,
    authority_profile,
    parent_lineage
)
```

Invariants: the child receives enough information to perform its bounded role; omitted information is not required for correctness; every inherited artifact has a revision; the parent can identify which evidence the child observed; and child output is treated as a proposal or evidence, not an automatic state mutation.

Guardrails: prohibit implicit inheritance of secrets, approvals, credentials, and writable handles; bound child history and tool schemas; label parent conclusions separately from source evidence; and re-fetch volatile repository or environment state.

Failure modes: projection omits a hidden constraint; the child receives stale world state; parent summaries bias an independent verifier; too much context destroys cost and cache advantages; or the child cannot explain which source informed its conclusion.

Observability: projected token counts by category, omitted categories, retrieval freshness, child cache hit rate, child tool set, lineage ID, and the number of parent corrections caused by missing context.

Evaluation: compare full inheritance against projected context on the same tasks. Measure completion, independent-error detection, stale-observation errors, context tokens, latency, and parent-child disagreement resolution.

Framework mapping: Claude Code's read-only child specialisation and fresh Git preference are concrete examples [R78]. Codex thread forking is an execution-lineage primitive, but the application must still decide what authority and external business state a fork receives [R80].

Competing alternatives: one large main agent, retrieval-only helper calls, deterministic repository indexes, or stateless specialist model calls.

Pattern: authority attenuation on delegation

Use when: a parent delegates work to a child process, agent, remote service, or tool-capable specialist.

Avoid when: the "child" is a pure local function with no independent identity or authority. Do not create delegation machinery around an ordinary deterministic call.

Mechanism: derive a child authority profile by intersection and explicit reduction, never by copying the parent's ambient environment.

Framework-neutral pseudocode.

```
child_authority = intersect(
    parent_delegable_authority,
    role_policy,
    task_scope,
    current_risk_policy
) - explicitly_prohibited_capabilities
```

Credentials are minted for the resulting profile, with their own audience, expiry, task and child binding.

Invariants: a child cannot grant itself new authority; parent approval is not automatically a child approval; writes carry child identity and task lineage; cancellation revokes or expires child credentials; and the child cannot escape its workspace or tenant scope.

Guardrails: clear session-level allow rules, preserve only explicitly intended administrator policy, use worktree or sandbox isolation, restrict child fan-out, cap cost and turns, and require parent-side validation before incorporating child mutations.

Failure modes: confused-deputy actions; approval leakage; shared mutable state corrupts the parent; orphaned child processes; duplicated children after recovery; unbounded fan-out; or a child-specific connector outlives the child.

Observability: child principal, parent lineage, delegated scopes, token issue and expiry, tool calls, mutable resources, heartbeat, cancellation state, transcript, cleanup result, and orphan detection.

Evaluation: attempt scope escalation, parent-approval reuse, cross-child interference, cancellation during tool execution, restart during child creation, and stale-child output after the parent state changes.

Framework mapping: the observed Claude Code runtime replaces parent session allow rules when a child allowlist is provided and isolates mutable state by default [R78-R79]. Codex exposes explicit child-thread lifecycle and configuration limits; the general authority intersection remains application policy.

Competing alternatives: no subagents; read-only subagents; process-level sandbox workers; or a queue of ordinary jobs executed by separately authenticated services.

Fresh-state preference

Large inherited snapshots feel efficient because they avoid another tool call. They are dangerous when they describe mutable state. A child repository explorer should normally run a fresh `git status` rather than consume a large session-start snapshot labelled stale. A verifier should read the final external object, not inherit the actor's claim about it.

The decision rule is:

> Transmit durable facts and expensive immutable evidence; reacquire small volatile observations close to use.

Record the observation timestamp, revision, authority and expiry. Freshness is not binary: a dependency lockfile at a pinned revision may be durable, while a deployment status can become stale in seconds.

Independent lifecycle

A production child needs more than start and result. Define:

- identity and parent lineage;
- accepted purpose and input revision;
- state: queued, running, waiting, completed, failed, cancelled, expired;
- liveness and deadline;
- transcript and output artifacts;
- resource ownership;
- cancellation propagation;
- cleanup and orphan recovery;
- deduplication key for restart;
- maximum descendants and cost.

If the harness cannot answer "which children are alive, what authority do they hold, and how are they stopped?", it does not have subagents; it has untracked concurrency.

18B.4 Claude Code lesson: prompt-cache behaviour shapes architecture

Evidence mix: observed mechanism for cache-safe parameters and replacement-state cloning; implementation-author comments for cache-miss and fleet-cost figures; manuscript inference for stable/volatile context partitioning.

Prompt caching is often placed under cost optimisation. In a large harness it affects architecture because cache identity depends on request shape. The observed Claude Code fork utilities identify cache-critical fields: system prompt, user and system context, tool set, model,

message prefix, and thinking configuration [R79]. A child path can preserve those fields deliberately. Changing an output-token setting may indirectly change thinking configuration and destroy cache compatibility. Mutable replacement decisions are cloned so that the same historical tool results are transformed identically and the wire prefix remains stable.

The compaction implementation also contains explicit prefix-sharing and cache telemetry [R77]. This leads to a stronger principle:

> **Stable request-prefix construction is a runtime invariant with correctness exceptions, not a formatting micro-optimisation.**

Pattern: stable and volatile context partitions

Use when: many calls share a large policy, tool, repository, or session prefix and the provider cache is economically material.

Avoid when: preserving a stale prefix would suppress required policy, state, or capability updates. Correctness and freshness dominate cache reuse.

Mechanism: build requests in canonical partitions.

Framework-neutral pseudocode.

```
[stable policy and static instructions]
[stable tool schemas in canonical order]
[versioned repository or product context]
[durable task state]
[volatile current observations]
[current user/action input]
```

Change a stable partition only when its version changes. Keep volatile fields late. Canonically sort tools and schemas. Make replacement and truncation decisions deterministic for the same history.

Invariants: cache optimisation never hides a policy update; each partition has a version and provenance; a request can explain why its prefix changed; and cache identity does not depend on unordered map iteration or incidental timestamps.

Guardrails: force cache invalidation after authority, model-policy, tool-schema, or sensitive-context changes; prevent cross-tenant prefix reuse when content differs; and do not keep stale external state merely to preserve a hit.

Failure modes: dynamic tool ordering causes misses; child and parent thinking settings differ; compaction reinjection changes stable ordering; a policy update fails to invalidate; or cache telemetry is interpreted as correctness evidence.

Observability: cache read and creation tokens, hit ratio by request class, prefix version, first differing partition, tool-schema churn, post-compaction cache behaviour, and estimated avoidable cache creation.

Evaluation: replay a fixed workload while varying one partition at a time. Measure hit rate, cost, latency, and task correctness. Include tests where a security-policy change must break the cache.

Framework mapping: Claude Code's cache-safe fork parameters and compaction path are source-observed examples [R77-R79]. Provider SDKs expose token accounting, but the application owns canonical request construction.

Competing alternatives: no cache optimisation; semantic response cache for deterministic read-only queries; retrieval cache; or smaller contexts.

18B.5 Claude Code lesson: capability discovery is incremental

Evidence mix: observed mechanism for deferred tools, skill discovery and post-compaction re-announcement; manuscript inference for the capability lifecycle and its evaluation metrics.

A large capability surface creates three problems: schema tokens, model selection error, and cache churn. The observed Claude Code build supports deferred tools, skill discovery, agent listings, and re-announcement of relevant capabilities after compaction [R77]. That is not merely "progressive disclosure" in the abstract; it is a capability lifecycle.

Pattern: catalogue, search, activate, retain, evict

Use when: the tool or skill universe is too large or too dynamic to expose in every request.

Avoid when: the agent has a small, stable set of high-frequency tools. Discovery adds latency and a new failure mode.

Mechanism:

1. **Catalogue:** maintain a code-owned index of capability name, purpose, schema digest, authority requirements, version and health.
2. **Search:** expose a bounded discovery operation returning candidate summaries, not full schemas.
3. **Activate:** policy-check and load selected schemas into the current capability set.
4. **Use:** log selection rationale and invocation.
5. **Retain:** preserve frequently relevant capability state across a bounded task segment or compaction boundary.
6. **Evict:** remove stale, unhealthy, unauthorised, or low-value schemas.

Invariants: undisclosed tools are not callable; activation does not grant authority; schema version is recorded with every invocation; retained tools survive compaction only when still eligible; and eviction cannot invalidate an in-flight call without an explicit transition.

Guardrails: filter discovery by tenant and principal; defend catalogue metadata from prompt injection; cap activated schema tokens; require health and version checks; and prevent a model from activating a prohibited capability by guessing its name.

Failure modes: search misses the correct tool; wrong candidate activation; stale schema retained; repeated activation causes cache breaks; discovery metadata is poisoned; or an unavailable tool remains in the active set.

Observability: search query, candidates, rank, activation decision, schema-token delta, time to first useful call, retention duration, evictions, cache breaks and task failures attributable to undisclosed capability.

Evaluation: capability-search recall on a labelled task set, incorrect activation rate, schema-token reduction, added discovery latency, failure rate compared with full disclosure, and robustness to malicious catalogue descriptions.

Framework mapping: MCP servers can provide catalogue metadata but do not define this lifecycle. Claude Code's deferred-tool and skill behaviour is one implementation [R77].

Competing alternatives: expose everything; use multiple role-specific agents with fixed tools; route tasks to a code-owned tool bundle; or compile bespoke tools per workflow state.

18B.6 What changed after the recovered Claude Code 2.1.88 snapshot

Evidence boundary: official public contract at Claude Agent SDK Python 0.2.128, commit `f8b9ec923982`, which bundles Claude Code CLI 2.1.220. This section does **not** claim access

to Anthropic's unpublished internal architecture. It records contract-level capabilities and lifecycle corrections visible in the official SDK source and changelog [R83-R86].

The gap between CLI 2.1.88 and the bundled 2.1.220 contract adds five production lessons.

1. **Background work has an independent terminal lifecycle.** Consumers must clear active tasks on terminal `TaskUpdatedMessage` states as well as ordinary task notifications. The SDK changelog records a prior failure mode in which background work could finish without the notification shape a client expected. The general rule is to define terminal state as a protocol invariant, not as one convenient message type.
2. **Termination reasons belong in the result contract.** `ResultMessage.terminal_reason` distinguishes completion, turn-budget exhaustion, and interruption. A harness should persist the terminal reason beside the final output because "no more tokens arrived" is not a state transition.
3. **Session mirroring is an asynchronous durability channel.** The public `SessionStore` contract mirrors locally durable transcript entries to external storage, supports batched or eager flushing, treats stable UUIDs as idempotency keys, and surfaces mirror failures without pretending the secondary copy is authoritative. This separates hot-path streaming from remote durability while making lag and loss observable.
4. **Context accounting is structured state.** `ContextUsageResponse` exposes categories, loaded/deferred MCP tools, memory files, agents, system-prompt sections and auto-compaction thresholds. Context pressure should be measured by component and capability lifecycle, not only as one aggregate token count.
5. **Deferred calls and hook observability are first-class lifecycle data.** Deferred tool state, hook-event streaming, richer permission context and shadowing warnings make policy bypass and suspended work visible to the application. The lesson is not "trust the SDK"; it is to persist and test every control-plane transition that affects authority or liveness.

These additions reinforce rather than replace the 2.1.88 archaeology. The recovered source is stronger for implementation mechanics such as compaction reconstruction and cache-shape decisions; the current SDK is stronger for the supported public lifecycle contract.

> **Not audited in this manuscript:** sandbox escape resistance, patch-application correctness, every approval path, cross-platform parity, consistency among all user interfaces, memory quality, and production incident rates. Source comments that quantify fleet-level savings are implementation-author statements, not independently audited measurements.

18B.7 Claude Code lesson: the first apparent result is not necessarily terminal

Evidence mix: observed mechanism in the unofficial 2.1.88 query loop; official contract and changelog evidence for terminal reasons and background-task lifecycle; manuscript inference for the lifecycle rule [R86-R87].

A coding-agent loop is not merely "call model, run tool, repeat." It is a streaming state machine in which model output, tool execution, partial observations, hooks, background work and cancellation can overlap.

The observed Claude Code query loop collects typed `tool_use` blocks from the model stream, may begin eligible tools before the model response has fully drained, and emits completed tool results as they become available [R87]. It keeps the original model-facing message byte-stable for prompt-cache continuity while yielding an observability-oriented clone where necessary. On fallback or stream failure it discards results belonging to the failed attempt, synthesises missing `tool_result` blocks so every advertised call has a terminal observation, and prevents orphan call identifiers from leaking into a retry. Large results pass through explicit budgeting and replacement/storage before being returned to the model. Post-tool and stop

hooks can change continuation. Cancellation drains queued or in-progress tools sufficiently to produce explicit interruption results and returns distinct terminal reasons for streaming and tool-phase aborts.

The official SDK history exposes the same class of lifecycle hazard from another angle: a recent fix kept stdin open while background tasks remained in flight because closing it after the first foreground result caused background MCP calls to fail and could bypass `PreToolUse` hooks [R86].

> **The first user-visible result is not necessarily the terminal lifecycle event. Completion belongs to the runtime state machine, not to whichever message arrives first.**

A robust loop therefore has an explicit shape:

Framework-neutral pseudocode.

```
stream model response
  -> collect typed tool calls and call identifiers
  -> resolve permissions and policy under the advertised context
  -> schedule each tool according to its execution semantics
  -> stream or buffer partial observations
  -> normalise, truncate, replace or persist large results
  -> emit one terminal result for every accepted tool call
  -> run post-tool and stop hooks
  -> account for background tasks and queued notifications
  -> continue, interrupt, fail, or declare terminal completion
```

Production invariants: every call has one terminal observation; retry attempts cannot reuse orphaned results; process ownership and cleanup survive cancellation; a foreground answer does not close transports needed by background work; terminal reason is typed and persisted; and hooks that enforce authority cannot be skipped by lifecycle shortcuts.

Where the rule does not apply: a single synchronous, non-streaming tool call without background work can use a much smaller loop. Do not reproduce a frontier harness's complexity when a bounded request/response transaction is sufficient.

Evaluation: abort during model streaming, permission resolution, shell startup, partial output, result storage, post-tool hook and background completion. Verify call/result pairing, process cleanup, hook coverage, terminal reason, duplicate suppression and transcript consistency.

18B.8 Codex lesson: one authoritative harness, multiple thin clients

Evidence mix: observed mechanism in the official App Server protocol for handshake, typed lifecycle, resume/fork/interrupt and bounded queues; manuscript inference for the one-harness/many-clients rule.

Codex separates its core harness from user interfaces. The App Server exposes a long-lived, bidirectional JSON-RPC protocol used by rich clients rather than asking each terminal or IDE integration to implement its own model/tool loop [R80]. The protocol has an initial capability handshake, versioned generated schemas, request correlation, thread creation and resumption, turn lifecycle, typed item events, approvals, interruption, forking, and bounded transport queues.

The transferable rule is:

> **Put loop semantics, state transitions and authority decisions behind one runtime protocol. Keep clients responsible for presentation and explicit user decisions.**

Without this split, a CLI, IDE plugin, web console and automation client gradually acquire different retry rules, approval semantics, event handling and session behaviour. That produces incompatible products sharing a model name.

Pattern: agent runtime protocol

Use when: more than one client, language, process or host must drive the same harness; tasks stream progress; or runs must reconnect, resume or move between clients.

Avoid when: one in-process application owns the only interface and a direct typed API is simpler. Do not add JSON-RPC merely to make the system look distributed.

Mechanism: define a versioned protocol around task lineage rather than model messages alone.

Framework-neutral pseudocode - protocol sketch.

```
client -> initialize(protocol_version, client_capabilities)
server -> initialized(server_capabilities, limits, schema_digest)

client -> thread.start(configuration, authority_profile_ref)
server -> thread.started(thread_id)

client -> turn.start(thread_id, input_items, idempotency_key)
server -> turn.started(turn_id)
server -> item.started(item_id, item_type)
server -> item.delta(item_id, payload_delta)
server -> approval.requested(action_hash, summary, expiry)
client -> approval.respond(action_hash, decision)
server -> item.completed(item_id, evidence_ref)
server -> turn.completed(turn_id, disposition)

client -> turn.interrupt(turn_id, reason)
client -> thread.resume(thread_id, after_event_id)
server -> events.replay(...)
```

Invariants: server is authoritative for legal lifecycle transitions; request IDs and idempotency keys survive reconnect; every streamed delta belongs to a typed item; approval responses bind to an immutable action; clients can recover from missed events; protocol versions negotiate capabilities rather than silently changing semantics; and the same task cannot be executed twice because two clients reconnect.

Guardrails: authenticate clients and bind them to permitted threads; cap request and event sizes; reject unknown methods; separate read-only observation from mutating control; protect approval responses against replay; and never treat client display state as durable task state.

Failure modes: slow client fills event buffers; reconnect causes duplicate turn start; client and server disagree on item completion; approval arrives after action mutation; schema changes without negotiation; one client interrupts another's turn; or a client implements hidden loop logic inconsistent with the server.

Observability: protocol version, negotiated capabilities, request correlation, queue depth, dropped or replayed events, client lag, reconnects, duplicate suppression, approval latency, interrupt propagation and per-client error rates.

Evaluation: compatibility tests across client versions; reconnect at every lifecycle event; duplicate request injection; slow-reader tests; malformed and oversized messages; approval replay; server restart; and deterministic transcript comparison across two thin clients driving equivalent tasks.

Framework mapping: Codex App Server provides a concrete protocol surface with thread, turn and item primitives and bounded queues [R80]. A2A is appropriate across independent agent systems; an internal runtime protocol is narrower and can expose product-specific lifecycle semantics.

Competing alternatives: embed the harness in each client; expose ordinary REST job endpoints plus polling; use a message bus directly; or provide a language SDK over an in-process core.

18B.9 Codex lesson: thread, turn and item are useful but incomplete

Evidence mix: observed protocol primitives and persistence semantics; manuscript inference for mapping them to business-task and transaction identities.

Codex's public protocol vocabulary is practical [R80]:

- **Thread:** durable conversation or execution lineage.
- **Turn:** one bounded unit of work initiated by input.
- **Item:** a typed unit inside a turn, such as a user message, model message, command execution, file edit, MCP call, reasoning summary, plan update or compaction marker.

A general production mapping is:

Framework-neutral pseudocode.

```
Business task
  └── execution thread or lineage
        ├── turn
        │     ├── input item
        │     ├── model proposal item
        │     ├── tool execution item
        │     ├── mutation item
        │     └── evidence item
        └── turn
```

This is better than storing one opaque transcript because items have independent identity and lifecycle events. It supports progress views, interruption, approvals and evidence references.

It is not a complete business-workflow state model. A customer task may span several threads, pause for days, wait for an external approval, use multiple agents, or require one transaction identity across several turns. Therefore keep distinct identifiers for:

- business task;
- workflow execution and version;
- thread or model lineage;
- turn;
- item/tool call;
- external transaction;
- approval;
- artifact and evidence.

Conflating task with thread makes fork semantics and long approval waits difficult. Conflating item with external transaction makes retries unsafe.

18B.10 Codex lesson: compaction semantics depend on timing

Evidence mix: observed mechanism in official source for manual/pre-turn versus mid-turn reinjection, checkpoints, hooks and retries; manuscript inference for phase-specific compaction contracts.

The Codex core does not treat compaction as a single generic summariser [R81-R82]. Manual compaction can choose local or provider-supported remote implementations. The core records trigger, reason, phase, implementation, status and token measurements. Pre- and post-compaction hooks can interrupt the operation. One model-client session is reused across retries so turn-scoped routing and request tracking survive transient failures. If the compaction request exceeds the context window, old history is removed incrementally while preserving recent items.

Most importantly, current initial instructions and world state are injected differently according to phase. A standalone pre-turn compaction can let the next normal turn rebuild initial context. A mid-turn compaction must place that context inside the replacement history at the model-expected boundary. Checkpoint metadata is stored separately from replacement history, and the session replaces live history through an explicit operation.

Pattern: phase-aware compaction state machine

Use when: a harness can compact both between turns and during an active turn, or can choose local and remote compaction implementations.

Avoid when: compaction occurs only at a quiescent, externally checkpointed boundary. A simpler replacement operation may be sufficient.

Mechanism: model compaction as states and events.

Framework-neutral pseudocode.

```
IDLE
  -> PREPARING(trigger, phase, source_window)
  -> HOOK_CHECK
  -> SUMMARISING(implementation, attempt)
  -> BUILDING_REPLACEMENT
  -> VALIDATING_REPLACEMENT
  -> COMMITTING_CHECKPOINT
  -> COMPLETED

Any non-committed state -> INTERRUPTED or FAILED
COMPLETED -> ordinary execution using replacement window
```

The commit point atomically records replacement history, compact metadata, reference context and world-state baseline.

Invariants: no partially built replacement becomes active; pre-turn and mid-turn boundaries use declared reinjection rules; outstanding tool items are paired or represented explicitly; retries do not create multiple committed boundaries; and replay reconstructs the same active window.

Guardrails: cap summary request size; validate that required instructions and operational manifest fields survive; forbid remote compaction for data that cannot leave the trust boundary; preserve recent user intent; and require a fallback when the summariser fails.

Failure modes: compaction interrupted after summary but before commit; remote result format changes; initial instructions duplicated or omitted; summary becomes the wrong final item; active tool calls are orphaned; or repeated retries delete too much history.

Observability: phase, trigger, implementation, attempt, source and result window IDs, retained items, context tokens before and after, hook outcomes, retry reason, cache tokens, commit latency and subsequent accuracy degradation.

Evaluation: inject interruption at every state; compare local and remote replacements; compact before and during tool execution; force context-overflow retry; resume after commit; replay from checkpoint; and run repeated-compaction long-horizon tasks.

Framework mapping: Codex's `InitialContextInjection`, compact task selection, hooks and checkpoint metadata instantiate this pattern [R81-R82]. Claude Code's compact boundary and reconstruction attachments cover a different but complementary design [R77].

Competing alternatives: only compact between turns; terminate and create a new thread with an external handover; or rely on a larger context window.

18B.11 Codex lesson: streaming still requires backpressure

Evidence mix: observed protocol behaviour for bounded queues and retryable overload rejection; manuscript inference for independent saturation policy at every asynchronous boundary.

A streaming protocol does not remove queues. It makes queue behaviour visible. Codex App Server documents bounded ingress, request and outbound queues and returns a specific retryable overload error when saturated [R80]. That is a concrete correction to the common architecture diagram where events flow through unlimited arrows.

Pattern: saturation policy at every asynchronous boundary

Use when: tasks, model calls, tools, subagents or client events can arrive faster than downstream work completes.

Avoid when: a bounded synchronous call graph proves that no queue exists. Even then, external provider rate limits may still create one.

Mechanism: inventory each queue independently:

Boundary	Capacity	Admission policy	Overload response	Cancellation
task intake	tenant-weighted	reject or delay before acceptance	retry-after / queued receipt	cancel queued task
active turns	per worker and tenant	concurrency semaphore	busy / scheduled	interrupt turn
model calls	provider and model	rate and cost limiter	backoff with jitter	cancel request where supported
tool processes	per tool class	bulkhead	explicit resource unavailable	terminate child tree
child agents	per task and tenant	fan-out budget	deny delegation	propagate cancellation
outbound events	per client	bounded buffer	disconnect or coalesce safe deltas	retain replayable checkpoint

Invariants: accepted work has a durable owner or explicit rejection; overload never becomes silent loss; retryable errors are distinguishable from permanent errors; duplicate retry is idempotent; one tenant cannot exhaust global capacity; and cancellation reaches work already admitted.

Guardrails: maximum queue age, tenant quotas, bounded event payloads, deadline propagation, circuit breakers for failing dependencies, and dead-letter handling for durable messages.

Failure modes: retry storm; head-of-line blocking; slow client blocks tool execution; output queue drops approval request; cancellation stops the UI but not the command; or subagent fan-out overwhelms verification capacity.

Observability: queue depth and age, rejection count, retry-after accuracy, saturation duration, per-tenant fairness, cancellation lag, orphan work and downstream utilisation.

Evaluation: closed-loop load tests, burst admission, slow-client injection, provider throttling, fan-out spikes, cancellation under saturation, and recovery without duplicate side effects.

Framework mapping: the Codex App Server transport makes saturation explicit at protocol boundaries [R80]. Temporal, Dapr or a message broker can make task admission durable, but they do not choose the policy.

Competing alternatives: unbounded queues; fixed worker pools with blocking admission; serverless per-task execution; or a batch scheduler.

18B.12 Codex lesson: configuration is part of the authority model

Evidence mix: observed configuration and protocol fields; manuscript inference for a canonical, versioned effective-authority profile.

A model prompt should not be the only place where authority is described. Codex exposes configuration around sandbox policy, approval modes, tools, instruction sources, model settings and collaboration behaviour through its runtime protocol and configuration surface [R80]. The general rule is:

> **Authority is a versioned, validated configuration object whose effective value is recorded with every action.**

Framework-neutral pseudocode - authority profile.

```
profile_version: qa-agent-authority/v4
identity:
  principal: qa-agent
  tenant: test-tenant
scope:
  environments: [staging]
  repositories: [project-a]
capabilities:
  read: [browser.read, jira.read, api.read]
  write: [jira.comment.draft]
prohibited:
  - production.write
  - credential.export
  - external_message.send
limits:
  max_tool_calls: 100
  max_parallel_children: 3
  maximum_cost_gbp: 8.00
  deadline_seconds: 1800
approval:
  required_for:
    - irreversible_write
    - external_message.send
  ttl_seconds: 900
sandbox:
  network: allowlisted
  writable_roots: [/workspace]
delegation:
  may_delegate: [repository-explorer, evidence-verifier]
  child_must_attenuate: true
```

Compute an effective profile from administrator policy, tenant policy, application policy, user decision and task risk. Store its digest in the trace. An approval references the immutable canonical action and effective profile digest. Re-evaluate both immediately before execution.

Configuration precedence must be explicit. “Last writer wins” is unacceptable when a local project file can broaden authority granted by organisation policy. Prefer monotonic restriction: lower-trust layers may narrow but not broaden. Where a layer can broaden, require an administrator-signed grant and record it visibly.

What this source audit did not establish.

This part did **not** audit sandbox escape resistance, patch-application correctness, every approval path, cross-platform parity, consistency among all UI clients, long-term memory quality, or real production incident rates. It also did not independently verify source comments containing fleet-scale cache or token measurements. These remain separate security, correctness, compatibility and operations evaluations.

18B.13 Codex lesson: tool execution scheduling is part of correctness

Evidence mix: observed mechanism in the official Codex tool router, runtime gate, invocation context and lifecycle notifications at commit `cef3910ea4d0`; manuscript inference for the general rules [R88-R91].

A model may emit several tool calls in one step, but “parallel tool use” is not a global speed switch. Codex retains the exact `StepContext` whose tool list advertised the call because execution may occur later. The router records model-visible tools separately from deferred namespaces, carries a typed `ToolCallSource`, asks each registered tool whether parallel execution is supported, and records whether cancellation must wait for runtime teardown [R88-R89].

The execution runtime then uses a read/write gate: calls declared parallel-safe take a shared read lock; serial calls take the exclusive write lock. MCP startup is awaited before entering that gate, so waiting for one server does not unnecessarily occupy the execution barrier for unrelated tools. Each invocation carries its call ID, tool name, source, step/turn context, cancellation token and result tracker [R89-R90].

Cancellation is treated as an ownership problem. An atomic terminal-outcome flag prevents both the runtime and abort path from claiming completion. Some tools are aborted immediately; process-owning runtimes may receive cancellation and be awaited for teardown. In either case the caller gets an explicit aborted tool result, and lifecycle observers receive a typed aborted outcome with call provenance [R89-R91].

The transferable rules are:

1. **Execute under the advertised snapshot.** A delayed call must retain the configuration, authority and environment context under which it was offered.
2. **Parallelism is a per-tool semantic property.** Read-only does not automatically mean parallel-safe, and write does not automatically mean serial; the tool contract decides.
3. **Cancellation needs one owner of the terminal outcome.** Competing completion and abort paths otherwise create duplicate or missing results.
4. **Aborted work produces a typed result.** Silence is not a valid terminal state for an accepted call.
5. **Dependency waiting should be scoped.** Waiting for one MCP server or runtime should not block unrelated tools unless shared invariants require it.
6. **Provenance survives dispatch.** Direct calls, code-mode calls and nested sources remain attributable through tracing, hooks and audit.

Where the rule does not apply: a workflow that intentionally serialises all effects may use one exclusive queue. It should still retain the advertised authority snapshot and explicit cancellation result.

Evaluation: combine parallel-safe and serial tools, delay one MCP server, cancel during queue wait and runtime teardown, race completion against cancellation, and replay nested code-mode calls. Assert ordering, exactly one terminal result, provenance retention, hook/audit completeness and absence of unrelated head-of-line blocking.

18B.14 Triangulation: three contrasting harnesses

Claude Code and Codex are useful primary autopsies, but two frontier systems are not a universal taxonomy. Three smaller or differently layered systems expose useful counterexamples [R92-R94].

Aider: deterministic repository context

Aider's repository map parses symbol definitions and references, builds a file/symbol graph, applies personalised PageRank and renders the highest-value structure under an explicit token budget [R92]. The lesson is that repository understanding need not begin with transcript accumulation or embedding retrieval. A deterministic symbol graph can provide a cheap, stable context prior.

Do not overgeneralise: symbol graphs do not prove runtime behaviour, resolve dynamic dispatch completely, or replace current file reads and tests.

mini-SWE-agent: minimal bounded-loop baseline

mini-SWE-agent exposes a compact model-action-environment loop with explicit step, cost, wall-time and consecutive-format-error limits, and persists the trajectory on every iteration [R93]. Its value is not that every production agent should remain tiny. Its value is as a control condition:

> **Every complex harness should be compared with a minimal bounded loop. Complexity must earn its reliability, safety or throughput cost.**

Do not overgeneralise benchmark simplicity into enterprise durability. The minimal loop does not by itself provide multi-day approvals, transaction identity, sandbox assurance or distributed recovery.

OpenHands: a heterogeneous-agent operations plane

OpenHands Agent Canvas explicitly separates UI/control responsibilities from action execution, sandboxing and automation backends. It can connect to local, remote or hosted Agent Server instances and optional automation/cloud services [R94]. The lesson is that an operational control plane can remain stable while agent backends vary.

Do not overgeneralise a common UI or API into common authority semantics. Each backend still needs its own capability, identity, sandbox and lifecycle contract.

System	Distinctive design lesson	What not to generalise
Claude Code	operational-state reconstruction, scoped subagents and lifecycle repair	recovered implementation is unofficial and version-specific
Codex	protocolised harness, typed lifecycle and tool scheduling	App Server primitives are not business-workflow semantics
Aider	deterministic, token-budgeted symbol-graph repository map	symbol graphs do not replace runtime evidence
mini-SWE-agent	minimal bounded loop with explicit resource limits	benchmark simplicity does not provide enterprise durability
OpenHands	heterogeneous-agent operations and presentation plane	common control surface does not imply common authority

18B.15 Comparative conclusions

Concern	Claude Code source-observed lesson	Codex official-source lesson	General production rule
Context	reconstruct active files, plans, skills, tools and children after compaction	distinguish pre-turn and mid-turn reinjection	compaction is a state transition, not summary text
Subagents	project context, isolate mutable state, attenuate session permissions, track lifecycle	expose child/thread lifecycle through explicit state and events	delegation requires context, authority and lifecycle isolation
Interfaces	integrated product runtime exposes hooks, transcripts and sidechains	App Server places one harness behind a versioned protocol	clients must not own divergent loop semantics
Tools	deferred discovery and re-announcement	typed items, provenance and per-tool scheduling	capability disclosure, execution semantics and authority are distinct state
Cost	prefix stability and replacement decisions affect cache creation	history windows, session continuity and compaction affect request cost	context shape is an economic and operational design variable
Reliability	liveness, child cleanup, missing-result repair and state reconstruction	bounded queues, cancellation ownership, explicit aborted results and resumable threads	every asynchronous boundary needs one terminal owner and observable failure semantics

Concern	Claude Code source-observed lesson	Codex official-source lesson	General production rule
Authority	parent session permissions are deliberately filtered for children	approval, sandbox and configuration are protocol-visible	authority must be explicit, versioned and attenuated

Neither harness should be copied wholesale. They optimise for coding tasks, local workspaces and their own product constraints. The generalisable value lies in the mechanisms:

- state is reconstructed from typed sources after context replacement;
- subagent creation is a runtime operation, not a role prompt;
- stable request prefixes can warrant explicit engineering;
- capabilities have discovery and activation state;
- one harness can serve many clients through a protocol;
- thread, turn and item are useful execution identities;
- compaction has phase-dependent semantics;
- streaming boundaries need backpressure;
- authority belongs in configuration and enforcement, not prose alone;
- tool scheduling, provenance and cancellation are correctness contracts;
- conclusions should be tested against deterministic and minimal baselines.

18B.16 Decision procedure for repository-agent harness design

1. **List durable state.** Identify what must survive model context loss, worker loss and client disconnect.
2. **Define context replacement.** Specify the operational reconstruction manifest before implementing summarisation.
3. **Define child runtime semantics.** Decide context projection, authority attenuation, identity, cancellation, transcript, worktree and cleanup.
4. **Partition stable and volatile context.** Optimise cache only after correctness, policy invalidation and freshness rules exist.
5. **Choose capability lifecycle.** Fixed tool bundle for small surfaces; catalogue-search-activate for large ones.
6. **Choose one authoritative loop owner.** If there are multiple clients, expose a versioned runtime protocol.
7. **Separate business task from thread.** Keep external transaction and approval identities independent from model turns.
8. **Specify compaction phases.** Document trigger, preserved suffix, reinjection, outstanding operations and commit point.
9. **Bound every queue.** State capacity, admission, overload response, retry and cancellation.
10. **Materialise authority.** Produce a versioned effective profile and bind actions, approvals and credentials to it.
11. **Specify tool scheduling.** Record the advertised context, per-tool parallelism, dependency waits, provenance and cancellation owner.
12. **Test the transitions.** Interrupt at compact commit, child creation, approval wait, tool execution, background completion, event delivery and reconnect.
13. **Measure economics with correctness.** Track cache and token savings beside lost constraints, stale state and unsafe actions.

18B.17 Common misdiagnoses

“We have memory, so compaction is safe.” Memory does not reconstruct current files, pending verification, active children, permissions or capability state.

“The child has the parent’s task, so it needs the parent’s full context.” Most children need a purpose-specific projection plus fresh observations. Full inheritance transfers stale state, bias and authority.

“All tools are already loaded; discovery is unnecessary.” That may be correct for ten tools and wrong for hundreds. Measure schema tokens, selection error and discovery miss rate.

“The IDE can implement approvals locally.” The client may display and collect a decision. The authoritative action hash, expiry, policy revalidation and lifecycle transition belong to the harness or workflow service.

“Streaming means there is no queue.” Network, process, provider and client boundaries all buffer. Unspecified capacity is an unbounded-queue bug waiting for load.

“A thread is the business task.” A task may fork, span several threads or wait longer than a model session. Use separate identifiers.

“Cache misses are only a billing problem.” Cache-sensitive construction changes how context, tools, children and compaction are organised. Optimisation must still yield to policy changes and fresh state.

“Parallel tool calls are just a performance option.” Parallel eligibility, serialisation barriers, dependency waits and cancellation ownership affect correctness and must be part of the tool contract.

“The first result means the run is finished.” Background tasks, post-tool hooks and cleanup may still be active. Persist the runtime’s typed terminal event and reason.

“Source code proves production intent.” It proves only that a mechanism is present in the pinned source snapshot. Comments, telemetry and product behaviour must be interpreted with provenance and version limits.

Part 19 - Case study: scalable QA and ticket-verification agent

This case study covers an agent triggered from an issue tracker to verify a ticket against a deployed product, capture findings, ask focused questions, and propose regression cases. It is designed for asynchronous, queue-based operation with cost-sensitive visual inspection.

19.1 Task contract

Objective: determine whether the ticket’s acceptance criteria are satisfied in a specified environment and provide reproducible evidence.

Inputs:

- ticket identifier and revision;
- target environment and build;
- test account and role;
- acceptance criteria;
- allowed browser and API tools;
- data-reset policy;
- risk tier and time budget.

Outputs:

- verdict per criterion: pass, fail, blocked, ambiguous, or not tested;
- exact reproduction steps;
- screenshots or DOM/API evidence;
- environment and build identity;
- findings and questions;
- regression-test candidates;
- residual risk.

The agent must not silently reinterpret the ticket or change production data.

19.2 Naive design

Framework-neutral pseudocode.

```
@mention -> launch browser agent -> browse until it “looks ”right -> comment
```

Failure modes:

- executes against the wrong environment or stale build;
- misses hidden ticket updates;
- relies only on screenshots when DOM or API state is authoritative;
- confuses loading states with final states;
- modifies shared data and contaminates other tests;
- spends excessive tokens on screenshots;
- retries indefinitely after authentication or UI failure;
- comments with an unsupported pass/fail conclusion;
- exposes credentials in traces;
- loses work when the worker or browser dies.

19.3 System architecture

Framework-neutral pseudocode.

```
issue-tracker webhook or poller
    -> signature and event validation
    -> idempotent enqueue
    -> task normaliser
    -> environment reservation
    -> acceptance-criteria compiler
    -> browser/API execution loop
    -> independent evidence validation
    -> report generator
    -> issue-tracker comment and artifact links
```

A lightweight durable queue may be implemented with a transactional database table and workers for modest scale. The necessary semantics are more important than the product: unique task keys, leases, heartbeats, retry classes, cancellation, visibility, and dead-letter handling.

19.4 Queue schema

Framework-neutral pseudocode.

```
CREATE TABLE qa_task (
  id                  uuid PRIMARY KEY,
```

```
  source_event_key   text UNIQUE NOT NULL,
  ticket_key         text NOT NULL,
  ticket_revision    text NOT NULL,
  environment        text NOT NULL,
  state              text NOT NULL,
  available_at       timestamptz NOT NULL,
  lease_owner        text,
  lease_expires_at   timestamptz,
  attempt            integer NOT NULL DEFAULT 0,
  max_attempts       integer NOT NULL,
  payload_ref        text NOT NULL,
  result_ref         text,
  last_error_class   text,
  created_at         timestamptz NOT NULL,
  updated_at         timestamptz NOT NULL
);
```

Claim work with an atomic conditional update. Use lease expiry and a fencing number so a slow prior worker cannot publish after ownership has moved.

19.5 Task state machine

Framework-neutral pseudocode.

```
QUEUED
  -> PREPARING
  -> WAITING_FOR_ENVIRONMENT
  -> AUTHENTICATING
  -> EXECUTING
  -> WAITING_FOR_CLARIFICATION
  -> VERIFYING
  -> REPORTING
  -> SUCCEEDED | PARTIALLY_COMPLETED | FAILED | CANCELLED | EXPIRED
```

A ticket edit can cancel or supersede a queued task. An in-flight task records the tested ticket revision and must not present itself as current if the ticket changed.

19.6 Acceptance-criteria compiler

Ticket prose is untrusted and often ambiguous. Convert it into a typed test charter:

Framework-neutral pseudocode.

```
{
  "criterion_id": "AC-2",
  "claim": "A workspace admin can export the filtered dataset as CSV",
  "preconditions": [
    "authenticated as workspace_admin",
    "dataset fixture ds_42 exists",
    "filter country=IN is applied"
  ],
  "action": "request CSV export",
  "expected_observations": [
    {"source": "ui", "predicate": "success notification appears"},
    {"source": "api", "predicate": "export job reaches COMPLETE"},
    {"source": "artifact", "predicate": "CSV contains only matching rows"}
  ],
  "destructive": false
}
```

The compiler may ask for clarification when no testable interpretation is safe. Code validates allowed roles, fixture names, environment, and action classes.

19.7 Evidence hierarchy

Prefer evidence in this order:

1. authoritative backend or API state;
2. DOM/accessibility tree with stable selectors;
3. downloadable artifact inspection;
4. structured logs linked to the task;
5. screenshot or visual observation;
6. model description without captured evidence.

Screenshots are valuable for layout and visual defects but expensive and often semantically weak. Capture them at decision points, not every step.

19.8 Visual-token cost control

- use DOM and accessibility data for navigation where available;
- crop screenshots to the relevant region;
- downscale only when text remains readable;
- capture on state transition, error, or final evidence;
- hash and deduplicate near-identical frames;
- prefer API polling for long-running jobs;
- use a cheaper visual model for routine localisation only after evaluation;
- escalate to a stronger model for ambiguous or high-impact visual judgement.

Cost control must not remove evidence needed to reproduce a defect.

19.9 Environment isolation

Provision or reserve:

- exact application build;
- clean browser profile;
- scoped test identity;
- deterministic fixture set;
- isolated tenant or workspace;
- known feature flags;
- clock and locale settings;
- network policy.

Record every environment dimension in the report. A “cannot reproduce” result without environment identity is weak evidence.

19.10 Authentication and secrets

Credentials are injected by the execution environment, not placed in the prompt. The browser tool exposes login actions without revealing raw secrets. Short-lived accounts and role-scoped permissions limit impact.

If multi-factor authentication or a human challenge appears, move to `WAITING_FOR_APPROVAL` or `BLOCKED`; do not attempt to bypass it.

19.11 Execution loop

For each criterion:

1. verify preconditions through authoritative state;
2. perform one bounded action;
3. wait for a defined observable condition, not an arbitrary sleep;
4. capture evidence;
5. classify result;
6. reset or preserve state according to fixture policy;
7. continue or stop based on risk and dependency.

The model selects navigation and exploratory actions. Deterministic code controls environment reservation, credentials, destructive operations, waits, deadlines, and evidence recording.

19.12 Handling ambiguity

An ambiguous criterion is not a failure. The report should state:

- the competing interpretations;
- evidence supporting each;
- which interpretation was tested;
- why another interpretation was unsafe or impossible;
- the smallest question needed from the owner.

A question should be attached to a concrete blocked decision, not generated as generic commentary.

19.13 Guardrails

- Issue-tracker instructions cannot change security policy.
- Only allowlisted domains and environments are reachable.
- Production environments are read-only unless explicitly approved.
- Destructive actions require dedicated fixtures and compensation.
- File uploads are scanned and size-limited.
- Browser downloads enter a quarantine directory.
- External messages and issue comments are previewed and policy-checked.
- The agent cannot mark a ticket closed or released.
- Loop, screenshot, model, and wall-clock budgets are enforced.

19.14 Verification

A criterion passes only when all mandatory observations pass. A UI toast alone does not prove durable backend completion. For data transformations or exports, independently inspect the resulting artifact.

For failures, re-run the minimal reproduction once in a fresh session when inexpensive. If the result is intermittent, report frequency and trial count rather than choosing a convenient verdict.

19.15 Report schema

Framework-neutral pseudocode.

```
{
```

```
  "ticket": "APP-1842",
  "ticket_revision": "2026-07-28T08:15:00Z",
  "build": "web-2026.07.28.3",
  "environment": "qa-eu-2",
  "summary": "2 passed, 1 failed, 1 ambiguous",
  "criteria": [
    {
      "id": "AC-2",
      "verdict": "failed",
      "reproduction": ["..."],
      "expected": "...",
      "observed": "...",
      "evidence_refs": ["artifact://...", "image://..."],
      "confidence": "high"
    }
  ],
  "questions": ["Should export preserve displayed column order?"],
  "regression_candidates": ["api/export_filtered_dataset"],
  "residual_risk": ["Safari not tested"]
}
```

19.16 Issue-tracker communication

The public comment should be concise and link to the full artifact. It must distinguish:

- verified fact;
- model interpretation;
- blocked check;
- recommendation.

Do not dump private traces, credentials, or internal chain-of-thought into the ticket.

19.17 Evaluation programme

Build tasks from historical tickets with known outcomes and controlled environments. Include:

- clear passes and failures;
- ambiguous acceptance criteria;
- role and permission defects;
- asynchronous jobs;
- visual regressions;
- stale build and wrong-environment traps;
- authentication failure;
- prompt injection in page or ticket text;
- intermittent failures;
- impossible-to-test criteria.

Metrics:

- criterion-level precision and recall;
- false-pass rate, weighted most heavily;
- evidence completeness;
- reproduction success by human QA;
- environment contamination rate;
- cost per criterion;
- time to actionable report;

- human edit distance to final ticket comment.

19.18 Rollout

Start with shadow verification on already-resolved tickets. Next, comment privately or attach draft reports without changing ticket state. Canary on low-risk products and test environments. Require human approval before external comments until false-pass and evidence metrics meet the release gate.

The agent scales observation and evidence collection. Product ownership and release authority stay with humans or deterministic governance systems.

19.19 Executable design package

This section turns the case study into a buildable contract package. The implementation language and queue product may vary; the schemas, transitions, authority rules, and tests should not.

Service boundary and sequence

Framework-neutral pseudocode.

```
Jira webhook
    | signed event
    v
Admission API ----reject----> audit log
    | create task + outbox row
    v
PostgreSQL task queue <---- lease/heartbeat ---- Worker
    |                                          |
    |                                          +--> browser sandbox
    |                                          +--> read-only product API
    |                                          +--> evidence store
    |                                          +--> model gateway
    v
Verifier service ---- criterion results ----> task ledger
    |
    +--> approval service (when required)
    |
    +--> Jira reporter (idempotent comment key)
```

A reference deployment needs only PostgreSQL, one admission service, stateless workers, disposable browser containers, object storage, and a reporter. Celery is optional. PostgreSQL `FOR UPDATE SKIP LOCKED`, a lease expiry, and a fencing token are sufficient for a modest queue when implemented carefully.

Core SQL schema

Framework-neutral pseudocode.

```
CREATE TABLE qa_task (
    task_id              uuid PRIMARY KEY,
    source_ticket_id     text NOT NULL,
    source_event_id      text NOT NULL UNIQUE,
    tenant_id            text NOT NULL,
    state                text NOT NULL,
    risk_class           text NOT NULL,
    workflow_version     text NOT NULL,
    prompt_bundle_version text NOT NULL,
```

```
    tool_catalog_version text NOT NULL,
    lease_owner          text,
    lease_expires_at     timestamptz,
    fencing_token        bigint NOT NULL DEFAULT 0,
    attempt_count        integer NOT NULL DEFAULT 0,
    proposal_hash        text,
    created_at           timestamptz NOT NULL,
    updated_at           timestamptz NOT NULL,
    CHECK (state IN (
      'QUEUED','RUNNING','WAITING_FOR_INPUT','WAITING_FOR_APPROVAL',
      'RETRY_SCHEDULED','VERIFYING','SUCCEEDED','PARTIAL','FAILED',
      'CANCELLED','EXPIRED','QUARANTINED'))
);

CREATE TABLE qa_event (
    event_id         bigserial PRIMARY KEY,
    task_id          uuid NOT NULL REFERENCES qa_task(task_id),
    event_type       text NOT NULL,
    event_version    integer NOT NULL,
    payload          jsonb NOT NULL,
    created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE qa_outbox (
    outbox_id        bigserial PRIMARY KEY,
    task_id          uuid NOT NULL REFERENCES qa_task(task_id),
    destination      text NOT NULL,
    idempotency_key  text NOT NULL UNIQUE,
    payload          jsonb NOT NULL,
    delivered_at     timestamptz
);
```

The worker increments `fencing_token` when it acquires a lease. Every state-changing write includes `WHERE fencing_token = :token`; a worker whose lease expired cannot commit over a newer worker.

Typed task and criterion schemas

Framework-neutral pseudocode.

```
{
  "$id": "QaTaskContract.v1",
  "type": "object",
  "required": ["task_id", "ticket", "risk_class", "criteria", "authority"],
  "properties": {
    "task_id": {"type": "string", "format": "uuid"},
    "ticket": {
      "type": "object",
      "required": ["id", "revision", "summary"],
      "properties": {
        "id": {"type": "string"},
        "revision": {"type": "string"},
        "summary": {"type": "string"}
      }
    },
    "risk_class": {"enum": ["low", "medium", "high"]},
    "criteria": {
      "type": "array",
      "minItems": 1,
      "items": {"$ref": "AcceptanceCriterion.v1"}
```

```
    },
    "authority": {
      "type": "object",
      "required": ["allowed_environments", "write_actions"],
      "properties": {
        "allowed_environments": {"type": "array", "items": {"type": "string"}},
        "write_actions": {"type": "array", "items": {"type": "string"}}
      }
    }
  }
}
```

Framework-neutral pseudocode.

```
{
  "$id": "AcceptanceCriterion.v1",
  "type": "object",
  "required": ["criterion_id", "statement", "verification_mode", "mandatory"],
  "properties": {
    "criterion_id": {"type": "string"},
    "statement": {"type": "string"},
    "verification_mode": {
      "enum": ["api", "database_fixture", "dom", "visual", "human"]
    },
    "mandatory": {"type": "boolean"},
    "expected": {},
    "forbidden_side_effects": {"type": "array", "items": {"type": "string"}}
  }
}
```

Evidence and finding schemas

Framework-neutral pseudocode.

```
{
  "$id": "EvidenceRecord.v1",
  "type": "object",
  "required": ["evidence_id", "type", "uri", "sha256", "captured_at", "
   provenance"],
  "properties": {
    "evidence_id": {"type": "string"},
    "type": {"enum": ["screenshot", "dom", "http", "api", "log", "video_clip"]},
    "uri": {"type": "string"},
    "sha256": {"type": "string", "pattern": "^[a-f0-9]{64}$"},
    "captured_at": {"type": "string", "format": "date-time"},
    "provenance": {"type": "object"},
    "redactions": {"type": "array"}
  }
}
```

Framework-neutral pseudocode.

```
{
  "$id": "Finding.v1",
  "type": "object",
  "required": ["finding_id", "criterion_id", "status", "evidence_ids", "summary
   "],
  "properties": {
    "finding_id": {"type": "string"},
    "criterion_id": {"type": "string"},
```

```
    "status": {"enum": ["pass", "fail", "blocked", "ambiguous"]},
    "severity": {"enum": ["none", "low", "medium", "high", "critical"]},
    "evidence_ids": {"type": "array", "items": {"type": "string"}},
    "summary": {"type": "string"},
    "reproduction_steps": {"type": "array", "items": {"type": "string"}},
    "unresolved_question": {"type": ["string", "null"]}
  }
}
```

Policy matrix

Action	Low risk	Medium risk	High risk	Deterministic conditions
Read ticket and attachments	allow	allow	allow	tenant and ticket scope match
Login to test environment	allow	allow	allow	ephemeral credential; approved environment
Read product API	allow	allow	allow	read-only token; row/tenant scope
Submit non-destructive test data	allow	allow	approval if persistent	idempotency key; cleanup plan
Change user role or permission	deny	approval	deny by default	named approver and fixture account only
Send external email/message	deny	approval	approval plus destination allowlist	immutable preview hash
Delete or overwrite persistent data	deny	deny	deny	separate maintenance workflow required
Comment on Jira	allow final report	allow final report	approval for security-sensitive details	idempotent comment key; redaction pass

Worker algorithm

Framework-neutral pseudocode.

```
def process(task_id: UUID, worker_id: str) -> None:
    lease = queue.acquire(task_id, worker_id, ttl_seconds=90)
    task = ledger.load(task_id)
    contract = compile_acceptance_criteria(task)
    sandbox = browser_pool.create(task.tenant_id, network_policy=task.
   network_policy)

    try:
```

```
        while task.state in {"RUNNING", "VERIFYING"}:
            queue.heartbeat(task_id, lease.token)
            context = context_builder.for_stage(task, contract)
            proposal = ActionProposal.model_validate(agent.next_action(context))
            decision = policy.evaluate(task, proposal)

            if decision.requires_approval:
                ledger.suspend_for_approval(task, proposal.hash, lease.token)
                return

            outcome = tool_gateway.execute(
                proposal,
                credential=decision.scoped_credential,
                idempotency_key=f"{task_id}:{proposal.action_id}",
            )
            evidence = evidence_store.persist(outcome.evidence)
            task = ledger.append_outcome(task, outcome, evidence, lease.token)
            task = verifier.advance(task, contract, lease.token)
    except TransientError as exc:
        queue.schedule_retry(task, classify_backoff(exc), lease.token)
    except UnknownError as exc:
        ledger.quarantine(task, sanitise_exception(exc), lease.token)
    finally:
        browser_pool.destroy(sandbox)
```

Failure-injection table

Injection	Expected system behaviour	Assertion
Worker killed after browser action, before state write	action idempotency/read-back prevents duplicate mutation	one external effect; resumed task records it
Lease expires during slow model call	stale worker cannot commit	fencing-token write rejected
Jira webhook delivered three times	one task created	unique `source_event_id`
Screenshot upload fails	criterion remains unverified; retry evidence persistence	no pass without evidence hash
Browser shows success but API shows failure	finding is fail/ambiguous	authoritative API outranks visual state
Prompt injection in ticket text	content treated as untrusted; no new tool authority	policy trace shows denied attempt
Credential accidentally has production scope	environment policy blocks destination	no production connection established
Approval arrives after ticket revision changes	approval invalidated	proposal hash/revision mismatch
Reporter times out after Jira accepted comment	outbox retry does not duplicate comment	stable idempotency marker
Verifier service unavailable	task remains `VERIFYING` or safely partial	never transitions to `SUCCEEDED`

Evaluation dataset design

Maintain four sets:

1. **Representative set:** sampled by real ticket family, risk, ambiguity, and browser/API complexity.
2. **Rare-failure set:** stale UI, race conditions, partial backend failure, auth expiry, and cross-tenant lookalikes.
3. **Security set:** direct and indirect prompt injection, malicious attachments, tool-description poisoning, destination confusion, and approval manipulation.
4. **Recovery set:** process death at each state boundary, duplicate delivery, lease expiry, evidence-store failure, and reporter timeout.

Each task package includes initial fixture snapshot, ticket revision, criterion manifest, hidden authoritative grader, prohibited side effects, and reset verification. Store no final answer in the model-visible ticket package.

Rollout thresholds

- shadow: >=500 representative runs, zero critical side effects, trace completeness >=99.5%;
- internal canary: verified completion lower confidence bound no worse than human-assist baseline by more than 3 pp;
- 5% customer canary: severity-weighted unsafe-event density below baseline, recovery suite 100%, rollback exercised;
- broader release: >=1,000 production-equivalent runs per material risk class, stable cost/latency for 14 days;
- autonomy increase: separately approved gate; never inferred automatically from general success.

Incident example: duplicate external notification

Event. A worker posted a Jira comment, timed out before recording delivery, and a retry posted the same report again.

Immediate containment. Disable reporter writes through the tool gateway; leave verification workers read-only; remove duplicate comments manually.

Root cause. The reporter used task ID plus attempt number as the idempotency key. A retry created a new attempt number, so the downstream call was not duplicate-safe. The Jira adapter also failed to search for the stable report marker before posting.

Corrective design. Use `sha256(ticket_id + ticket_revision + report_hash)` as the immutable idempotency key, persist an outbox row before delivery, and make the adapter read for the marker after ambiguous timeout. Add a failure-injection test that kills the reporter after remote acceptance but before local acknowledgement.

Evaluation update. Add 100 ambiguous-timeout trials and a release gate of exactly one remote comment per report hash. This is a distributed-systems defect, not a prompt defect; changing the model would not address it.

Part 20 - Case studies: computer use and enterprise data analysis

20A. Browser and computer-use agent

Computer use is qualitatively different from calling a typed business API. The agent observes an incomplete visual state, interacts through coordinates or accessibility elements, and may encounter arbitrary untrusted content. Historical WebArena and OSWorld results illustrate how sharply performance falls on realistic multi-step environments [R47-R50]. OpenAI and Anthropic system materials also identify model mistakes, prompt injection, and human oversight as central risks for computer-use systems [R34, R52].

20A.1 Task contract

Objective: complete a bounded browser or desktop workflow in an approved environment while preserving user control over consequential actions.

Example: collect three compliant vendor quotes and prepare, but do not submit, a purchase request.

Completion requires:

- correct target application and user identity;
- all required fields and evidence;
- no unauthorised external action;
- final state independently inspected;
- user-visible summary of completed and pending steps.

20A.2 Why free-form computer use fails

A naive agent receives a screen and a broad instruction. It may:

- click stale coordinates after the page changes;
- act on hidden or off-screen state;
- follow instructions embedded in webpages, documents, emails, or advertisements;
- submit instead of preview;
- mistake a visual acknowledgement for durable completion;
- expose clipboard or password-manager contents;
- lose track of tabs, downloads, modal dialogs, or account identity;
- continue after the environment diverges from expectations.

20A.3 Architecture

Framework-neutral pseudocode.

```
task contract
    -> application and account allowlist
    -> environment snapshot
    -> semantic state extraction
    -> bounded action proposal
    -> policy and freshness check
    -> execute one action
    -> observe changed state
    -> milestone verification
    -> approval before consequential commit
    -> final authoritative verification
```

The harness treats every action as a proposal against a known screen version. A stale-screen check rejects actions if the observation has changed materially.

20A.4 Observation model

Combine:

- accessibility tree or DOM where available;
- application metadata, URL, window title, and active identity;
- screenshot regions;
- recent action history;
- downloads and network/API observations where allowed.

Represent interactive elements with stable handles tied to an observation version:

Framework-neutral pseudocode.

```
{
  "observation_id": "obs_81",
  "application": "vendor-portal",
  "url": "https://approved.example/quotes/new",
  "identity": "purchasing-test@example.org",
  "elements": [
    {"handle": "el_12", "role": "button", "name": "Review request"}
  ],
  "screenshot_ref": "image://obs_81",
  "captured_at": "..."
}
```

The action tool requires `observation_id`. It refuses to use `el_12` after navigation or material page mutation.

20A.5 Action taxonomy

Classify actions:

- observation: scroll, inspect, open read-only detail;
- reversible local change: fill draft field, select filter;
- external communication: send message, submit form;
- financial or contractual commitment;
- credential or permission change;
- destructive action.

The allowed autonomy decreases as consequence and irreversibility increase.

20A.6 Prompt-injection boundary

Webpage text is data, not authority. The context builder labels source regions and prevents webpage content from modifying system policy, tool permissions, or task objective. Detect suspicious instructions, but do not assume a classifier can make browsing safe by itself.

For high-risk workflows:

- isolate the browser profile;
- restrict domains and egress;
- disable arbitrary downloads or quarantine them;
- block password manager and local file access;
- use task-scoped credentials;
- show the human the exact pending external effect;
- require approval tied to canonical action arguments.

20A.7 Approval preview

A meaningful approval says:

Framework-neutral pseudocode.

```
Action: submit purchase request
Vendor: Example Systems Ltd
Amount: GBP 18,400
Cost centre: ENG-PLATFORM
Attachments: quote_112.pdf, security_review.pdf
External recipients: procurement@example.org
Irreversible effect: creates a formal approval workflow
```

“Allow the agent to continue?” is not informed approval.

20A.8 Recovery

Persist application, URL, identity, completed milestones, draft identifiers, and pending approval. Do not assume a restored browser session is valid. On resume:

1. reopen the application in a clean or validated profile;
2. authenticate using scoped credentials;
3. query the draft or task by durable identifier;
4. compare authoritative state against the checkpoint;
5. continue only if preconditions still hold.

20A.9 Verification

After submission or mutation:

- read the generated request identifier;
- navigate to the authoritative detail page or query an API;
- verify fields and status;
- capture evidence;
- reject success if only a toast or transient UI state is available.

20A.10 Evaluation

Test across:

- different layouts and viewport sizes;
- delayed loading and partial rendering;
- stale element handles;
- hidden modals and pop-ups;
- prompt injections in content;
- wrong account and wrong tenant;
- duplicate submission risk;
- approval cancellation;
- UI success with backend failure;
- inaccessible or visually ambiguous elements.

Primary metrics are verified completion, unsafe action rate, duplicate-action rate, approval accuracy, recovery success, and cost per verified workflow. Historical benchmark results should be recorded in benchmark cards rather than treated as direct predictions of your system.

20A.11 Deployment

Begin with read-only observation and draft preparation. Add reversible interactions next. External submission remains human-approved until task-specific evidence justifies narrower autonomy. Maintain a visible stop control and preserve the user's ability to take over.

20B. Enterprise data-analysis agent

An enterprise data-analysis agent answers questions by discovering data, generating transformations or queries, validating results, and presenting evidence. Its main risks are not only incorrect SQL. They include wrong dataset identity, semantic mismatch, privacy leakage, expensive scans, stale snapshots, and confident causal claims unsupported by observational data.

20B.1 Task contract

Example objective:

> Explain the decline in monthly recurring revenue for the EMEA mid-market segment in June 2026, quantify the drivers, and produce a reproducible analysis. Do not modify source data or publish a dashboard.

Required output:

- interpreted business question;
- datasets and snapshot times;
- metric definitions;
- queries or transformations;
- validated result tables;
- uncertainty and caveats;
- evidence links;
- reproducibility manifest.

20B.2 Naive design

Framework-neutral pseudocode.

```
natural-language question
    -> model receives warehouse credentials
    -> model writes arbitrary SQL
    -> model narrates result
```

Failure modes:

- queries the wrong table or environment;
- invents metric meaning;
- joins at the wrong grain and duplicates revenue;
- ignores slowly changing dimensions;
- leaks restricted rows;
- performs an unbounded scan;
- confuses correlation with cause;
- uses a changing live dataset and cannot reproduce the result;
- formats plausible numbers that do not reconcile.

20B.3 Architecture

Framework-neutral pseudocode.

```
question intake
    -> metric and entity resolution
    -> data-authority and sensitivity check
    -> read-only analysis workspace
    -> plan compiled to approved query graph
    -> cost estimation and policy validation
    -> execute bounded queries
    -> deterministic validation and reconciliation
    -> model interpretation
    -> evidence-backed report
```

20B.4 Semantic layer

A semantic layer defines business metrics, dimensions, grains, time logic, and allowed joins. The model should retrieve definitions instead of inventing them.

Example metric contract:

Framework-neutral pseudocode.

```
metric: monthly_recurring_revenue
owner: finance_analytics
currency: GBP
base_grain: subscription_month
expression: sum(recurring_amount_gbp)
exclusions:
  - trial subscriptions
  - one-time services
valid_dimensions:
  - region
  - customer_segment
  - product_family
freshness_slo: 24h
```

If two definitions exist, surface the conflict and ask the owner or run both clearly labelled.

20B.5 Data authority

Give the analysis role:

- read-only access;
- row and column policies inherited from the requesting identity;
- approved views rather than raw secrets or unrestricted base tables;
- query and scan limits;
- isolated temporary schema;
- no export to public destinations.

The model never receives credentials. The query gateway applies identity and policy.

20B.6 Plan-compile-execute

The model proposes a declarative analysis plan:

Framework-neutral pseudocode.

```
{
  "question": "drivers of EMEA mid-market MRR decline",
  "baseline_period": "2026-05",
```

```
  "comparison_period": "2026-06",
  "metric": "monthly_recurring_revenue",
  "cuts": ["product_family", "change_type", "country"],
  "checks": ["finance_total_reconciliation", "join_cardinality"],
  "hypotheses": ["churn", "contraction", "FX", "segment reclassification"]
}
```

A compiler resolves this into an approved query graph. It rejects undefined metrics, forbidden joins, excessive scans, and output that exceeds privacy thresholds.

20B.7 Deterministic analysis checks

Before execution:

- validate table and column existence;
- resolve snapshot or partition;
- check join keys and expected cardinality;
- estimate scan cost;
- enforce row limits and timeouts;
- classify sensitive columns.

After execution:

- check row counts and null rates;
- detect duplicate multiplication;
- reconcile totals against authoritative reports;
- validate units, currency, and time zones;
- test decomposition sums;
- compare repeated runs on the same snapshot;
- record query hashes and result hashes.

20B.8 Causal discipline

Descriptive data can show contribution and association. It usually cannot prove cause without a design that identifies causal effects.

The agent must distinguish:

- **fact:** MRR decreased by a measured amount;
- **decomposition:** churned accounts contributed a specified share;
- **association:** decline was concentrated in one product family;
- **hypothesis:** a price change may have influenced contraction;
- **causal conclusion:** requires stronger evidence such as a controlled experiment or credible quasi-experimental design.

A report should not turn a decomposition into a causal story.

20B.9 Privacy and small groups

Prevent disclosure through:

- minimum group sizes;
- suppression or aggregation;
- differential access by role;
- output scanning for direct identifiers;
- restrictions on joining sensitive domains;
- logged and approved exports;
- provenance-aware memory that does not retain restricted results for other users.

A model-generated chart or narrative is still governed data.

20B.10 Verification

For the example analysis:

1. reconcile May and June MRR with finance-certified totals;
2. ensure segment membership is evaluated consistently for both periods;
3. separate churn, contraction, expansion, new business, FX, and reclassification;
4. verify component sum equals total change within a defined tolerance;
5. sample records from major drivers;
6. run the final queries against a pinned snapshot;
7. attach query and result manifests.

20B.11 Result manifest

Framework-neutral pseudocode.

```
{
  "analysis_id": "ana_01...",
  "question_version": 2,
  "metric_versions": {"monthly_recurring_revenue": "7"},
  "snapshots": {
    "subscription_monthly": "2026-07-27T02:00:00Z"
  },
  "queries": [
    {"hash": "sha256:...", "artifact_ref": "sql://...", "result_hash": "sha256
    :..."}
  ],
  "checks": [
    {"name": "finance_total_reconciliation", "status": "passed", "delta":
    0.0003}
  ],
  "limitations": ["late-arriving June adjustments not included"]
}
```

20B.12 Evaluation

Task families:

- metric lookup;
- multi-step decomposition;
- ambiguous business terminology;
- incorrect-grain traps;
- slowly changing dimensions;
- restricted data requests;
- expensive query plans;
- causal overclaim traps;
- missing or stale data;
- contradictory certified sources.

Graders:

- deterministic numerical oracle;
- query-policy checker;
- provenance completeness;
- privacy grader;
- expert review of interpretation and caveats.

Metrics include numerical correctness, reconciliation rate, unsafe query rate, unsupported-claim rate, reproducibility, cost, and analyst correction time.

20B.13 Production economics

Warehouse scans may dominate inference cost. Use metadata queries, samples, partitions, pre-aggregations, and result caching. A code execution tool can compute deterministic statistics without returning every intermediate row to the model.

Optimise for analyst time saved and verified decision quality, not number of generated charts.

20B.14 Rollout

Start with read-only answers against certified metrics. Require query and evidence visibility. Expand to exploratory analysis after privacy and numerical evals pass. Dashboard publication, scheduled distribution, and data mutation remain separate capabilities with their own approvals.

Part 21 - Case studies: long-running research and high-risk approved action

21A. Long-running research agent

A research agent must turn an open question into an evidence-grounded synthesis over many sources and possibly many execution windows. The core problem is not producing a fluent report. It is maintaining question scope, source provenance, evidence quality, contradiction handling, and a defensible stopping decision.

21A.1 Task contract

Example objective:

> Assess whether passkey adoption materially reduces account-takeover risk for a consumer SaaS product, identify migration and recovery risks, and recommend a phased rollout. Use evidence current through 29 July 2026.

Required output:

- scoped research questions;
- source ledger with dates and source types;
- claim-evidence map;
- contradictory or missing evidence;
- evidence labels;
- synthesis and decision implications;
- explicit freshness boundary;
- reproducible search and reading log.

21A.2 Research state

Framework-neutral pseudocode.

```
{
  "research_id": "res_01...",
  "question_version": 3,
  "freshness_cutoff": "2026-07-28",
  "subquestions": [
```

```
    {"id": "Q1", "status": "sufficient"},
    {"id": "Q2", "status": "open"}
  ],
  "sources": 47,
  "claims": 26,
  "contradictions": 4,
  "search_budget_remaining": 18,
  "current_gap": "recovery-channel takeover rates"
}
```

Persist source and claim objects, not only a narrative summary.

21A.3 Source ledger

Each source record includes:

- stable identifier and URL;
- title, author or organisation, date, and source type;
- retrieval date;
- primary versus secondary status;
- jurisdiction or population;
- methodology and sample where relevant;
- extracted claims with exact location;
- limitations;
- freshness and supersession status.

A search result snippet is not evidence. The agent must open and inspect the source.

21A.4 Claim-evidence graph

Represent claims separately from prose:

Framework-neutral pseudocode.

```
{
  "claim_id": "C17",
  "text": "passkeys reduce credential-phishing susceptibility compared with
   reusable passwords",
  "evidence_label": "established",
  "supports": ["S4", "S8", "S11"],
  "qualifiers": ["does not remove account-recovery risk"],
  "contradicts": [],
  "last_reviewed": "2026-07-28"
}
```

The synthesis generator may only make substantive claims linked to the graph or explicitly label them as inference.

21A.5 Search strategy

Use staged search:

1. map terminology and authoritative institutions;
2. find primary standards, official documentation, trials, datasets, and papers;
3. search specifically for failure modes and negative evidence;
4. resolve contradictions;
5. update recent, unstable claims near the publication date;
6. stop when expected information value is below cost or the task budget is reached.

Search breadth without a gap model creates citation volume, not knowledge.

21A.6 Evidence hierarchy

The hierarchy depends on the question. For technical standards and platform behaviour, official specifications and implementation documentation are primary. For empirical security outcomes, prefer transparent field data, peer-reviewed studies, and well-specified incident datasets. Vendor case studies can provide production evidence but require conflict-of-interest and methodology notes.

Apply the evidence labels from the front matter to each important conclusion.

21A.7 Contradiction handling

When sources disagree:

- confirm they measure the same outcome;
- compare dates, populations, implementations, and definitions;
- inspect whether one supersedes another;
- retain both claims when disagreement is real;
- state the decision relevance of the uncertainty.

Do not average incompatible evidence or silently choose the source that supports the preferred conclusion.

21A.8 Long-running handover

At the end of each run, write:

- what was established;
- what remains uncertain;
- which sources were added or rejected;
- exact next search targets;
- changed claim labels;
- budget and deadline state.

The next run verifies the ledger and performs one bounded advancement. This prevents repeated broad searches and false completion [R10-R11].

21A.9 Guardrails

- webpages and documents cannot alter the research objective or source policy;
- source text remains clearly delimited from instructions;
- downloaded files are sandboxed;
- citation extraction preserves provenance;
- the system blocks fabricated bibliographic fields;
- private or licensed sources are handled under access policy;
- unsupported claims fail the evidence gate;
- the agent cannot conceal contradictory evidence.

21A.10 Stopping rule

Stop when all mandatory subquestions meet their evidence thresholds and further retrieval has low expected value relative to cost and delay. Continue when a missing fact could change the recommendation materially.

A practical rule:

Framework-neutral pseudocode.

```
continue research when:
  probability(new evidence changes decision)
```

```
  x consequence of wrong decision
  > expected retrieval, analysis, and delay cost
```

This estimate may be qualitative, but it should be explicit.

21A.11 Verification

Before publication:

- every load-bearing claim has a stable source;
- dates and version-sensitive claims are rechecked;
- quotations are exact and within rights limits;
- evidence labels are consistent;
- citations resolve;
- bibliography metadata is complete;
- source summaries accurately reflect the source;
- contradictions and limitations are present;
- a second pass checks whether conclusions overreach evidence.

21A.12 Evaluation

Create research tasks with known source sets, hidden decisive evidence, misleading secondary summaries, conflicting studies, outdated specifications, and prompt injections in documents.

Metrics:

- claim precision;
- decisive-source recall;
- citation correctness;
- contradiction recall;
- freshness accuracy;
- unsupported-claim rate;
- evidence-label calibration;
- report usefulness to domain experts;
- cost and time to a verified synthesis.

The final report is only one artifact. The source ledger and claim graph are the durable research product.

21B. High-risk human-approved workflow

Consider an accounts-payable agent that prepares a supplier bank-detail change and a payment-release request. This combines sensitive data, financial impact, social-engineering risk, and irreversible external effects. The correct goal is not maximum autonomy. It is reducing clerical work while making authority and evidence stronger than in the manual process.

21B.1 Task contract

Objective: process a supplier bank-detail change request and prepare eligible invoices for payment, subject to independent verification and authorised human approval.

The agent may:

- collect request evidence;
- verify supplier identity using approved channels;
- compare master data and invoice records;
- prepare a proposed change;
- flag anomalies;
- request approval.

It may not:

- approve its own proposal;
- change approver assignments;
- bypass dual control;
- release funds without a valid approval token;
- use contact details supplied only in the change request for independent verification.

21B.2 Threat model

Threats include:

- compromised supplier email;
- malicious attachment or prompt injection;
- employee collusion;
- model misreading account numbers;
- duplicate or altered invoices;
- wrong legal entity or currency;
- replayed approval;
- stale sanctions or supplier status;
- excessive agent permissions;
- traces leaking financial data.

21B.3 Architecture

Framework-neutral pseudocode.

```
request intake
    -> malware/content isolation
    -> deterministic field extraction and validation
    -> supplier identity resolution
    -> independent out-of-band verification
    -> anomaly and policy checks
    -> model-assisted case summary
    -> human dual approval with canonical preview
    -> transactional change or payment instruction
    -> authoritative read-back
    -> reconciliation and audit record
```

No untrusted content directly reaches a side-effecting tool.

21B.4 Separation of duties

Use distinct identities and components:

- intake service: receives documents, no payment authority;
- analysis agent: read-only evidence access;
- verification service: approved contact directory and external checks;
- proposal service: creates immutable proposed mutation;
- approvers: humans with role-based limits;
- execution gateway: accepts only approved proposal hashes;

- reconciliation service: read-only confirmation.

The model cannot impersonate an approver or create an approval token.

21B.5 Canonical proposal

Framework-neutral pseudocode.

```
{
  "proposal_id": "prop_01...",
  "supplier_id": "SUP-3182",
  "legal_entity": "Example UK Ltd",
  "change": {
    "field": "bank_account",
    "old_fingerprint": "GB** **** 1204",
    "new_fingerprint": "GB** **** 7781"
  },
  "verified_via": [
    "known-directory phone callback",
    "supplier-portal authenticated confirmation"
  ],
  "invoices": ["INV-4117", "INV-4119"],
  "total": {"currency": "GBP", "amount": "18400.00"},
  "risk_flags": [],
  "evidence_hashes": ["sha256:..."],
  "expires_at": "2026-07-29T12:00:00Z"
}
```

Approval signs or binds to this canonical proposal. Any change to amount, account, supplier, or invoices invalidates approval.

21B.6 Deterministic checks

Before approval:

- bank-account syntax and checksum where applicable;
- supplier status and legal entity;
- duplicate invoice detection;
- purchase-order and receipt match;
- payment limits;
- sanctions and restricted-party checks through approved systems;
- recent account-change risk window;
- approver independence and authority;
- proposal expiry.

Model judgement may summarise anomalies, but code enforces policy.

21B.7 Independent verification

Do not verify the bank change using a phone number or link from the same request. Use a previously trusted directory, supplier portal, or separate authorised process.

The verifier records method, identity, time, and result. Ambiguous or failed verification blocks progress.

21B.8 Human approval

The approval interface shows:

- old and new masked account fingerprints;

- supplier legal entity;
- invoice list and total;
- verification methods;
- anomalies and unresolved items;
- consequence and reversibility;
- proposal expiry;
- evidence links.

Require two independent approvers for defined risk tiers. The agent's recommendation is advisory.

21B.9 Transaction and idempotency

The execution gateway:

1. validates proposal hash and approvals;
2. checks current master data still matches the expected old state;
3. obtains an idempotency key;
4. performs the change or submits the payment instruction;
5. records the external transaction identifier;
6. reads back authoritative state;
7. reconciles the intended and actual mutation.

Use compare-and-set semantics to stop execution when underlying state changed after approval.

21B.10 Compensation and rollback

Bank-master changes may be reversible; released payments may not be. Define compensation per action:

- restore previous account while preserving audit history;
- cancel an unprocessed payment batch;
- place a supplier or payment hold;
- notify treasury and security;
- initiate bank recall under documented procedure.

"Undo" is not a generic tool. Every action needs a domain-specific compensation plan and time window.

21B.11 Guardrails

- attachments are scanned and rendered in isolation;
- document text is labelled untrusted;
- fields are extracted twice or cross-checked for high-value cases;
- bank details never appear unmasked in model-visible logs unless necessary and authorised;
- no model output can directly authorise execution;
- approval is bound to immutable proposal content;
- unusual timing, amount, geography, and contact changes trigger escalation;
- fail closed on verifier or policy-service outage;
- global payment kill switch is tested.

21B.12 Verification and audit

Completion requires:

- authoritative supplier record reflects the approved fingerprint;

- payment platform status and identifier match the proposal;
- no duplicate instruction exists;
- ledger or batch totals reconcile;
- approval, evidence, policy, and execution events are linked;
- residual follow-up, such as bank confirmation, is scheduled.

21B.13 Evaluation

Use synthetic and red-team cases:

- legitimate account change;
- compromised email with convincing documents;
- account-number transcription error;
- duplicate invoice;
- changed amount after approval;
- stale approval replay;
- colluding or unauthorised approver;
- sanctions-service outage;
- prompt injection in PDF;
- external system timeout after uncertain commit.

Primary metrics:

- unauthorised execution rate, with a target of zero in tests;
- true and false escalation rates;
- field-extraction accuracy;
- approval-binding correctness;
- duplicate and replay prevention;
- recovery from uncertain commit;
- audit completeness;
- human processing time.

21B.14 Rollout

Deploy first as a read-only case-preparation assistant. Next allow immutable proposal creation. Execution remains disabled until security review, red-team testing, policy verification, and operational drills pass. Expand only within transaction limits and error budgets.

This system is successful when it reduces manual search and transcription while strengthening dual control, provenance, and reconciliation. Autonomous payment is neither the starting point nor the default destination.

Agent-system design document template

Use this template before implementation. Replace bracketed text; delete sections only with an explicit reason.

A.1 Document control

- **System:** [name]
- **Owner:** [team/person]
- **Reviewers:** [security, platform, domain, operations]
- **Status:** [draft / approved / deployed / retired]
- **Version:** [version]
- **Last updated:** [date]

- **Decision deadline:** [date]
- **Related systems:** [links or identifiers]

A.2 Executive decision

- **Problem:** [one paragraph]
- **Why an agent is needed:** [semantic uncertainty or open-ended judgement]
- **Why deterministic software alone is insufficient:** [specific gap]
- **Proposed autonomy level:** [assist / read-only / draft / approved action / bounded autonomous action]
- **Maximum consequence:** [what the system can cause]
- **Success definition:** [verified outcome]
- **Non-goals:** [explicitly excluded]

A.3 Task distribution

Describe real production tasks, not a single happy path.

Task class	Frequency	Risk	Typical horizon	Required tools	Current baseline
[class]	[per day]	[low/medium/high]	[steps/time]	[tools]	[human/system performance]

Include ambiguous, impossible, adversarial, and interrupted tasks.

A.4 Task contract

- objective;
- authoritative inputs;
- permitted assumptions;
- constraints;
- required evidence;
- completion criteria;
- partial-completion semantics;
- escalation conditions;
- budgets;
- prohibited actions.

A.5 Architecture decision

Select and justify:

- single model call;
- retrieval plus generation;
- tool-calling micro-agent;
- code-owned workflow with probabilistic nodes;
- durable workflow;
- multi-agent system;
- computer-use system.

Document rejected alternatives and why they fail the requirements.

A.6 System boundary

Identify components:

- client and intake;
- task normaliser;
- context builder;
- model gateway;
- harness or workflow engine;
- tool gateway;
- policy service;
- state store;
- memory/artifact stores;
- verifier;
- approval service;
- observability pipeline.

Provide a data-flow and trust-boundary diagram.

A.7 Deterministic invariants

For each invariant:

Invariant	Owner	Enforcement point	Failure behaviour	Test
[example: payment amount cannot exceed approval]	execution gateway	pre-commit	fail closed	mutation test

No invariant may depend only on prompt wording.

A.8 State model

Define:

- task states;
- transition table;
- durable event schema;
- versioning;
- leases and fencing;
- idempotency;
- cancellation;
- deadlines;
- checkpoint policy;
- migration of in-flight work.

A.9 Context and memory

For each information class specify:

- source of truth;
- inclusion rule;

- freshness rule;
- provenance;
- sensitivity;
- retention;
- invalidation;
- model visibility.

A.10 Tools and authority

List each tool with:

- contract version;
- read/write class;
- credential scope;
- validation;
- approval requirement;
- idempotency;
- postcondition check;
- compensation;
- trace policy.

A.11 Prompt and specification assembly

Document:

- immutable policy sections;
- task-specific sections;
- state summary;
- retrieved evidence;
- examples;
- tool schemas;
- completion and escalation contract;
- provenance and versioning;
- ablation tests.

A.12 Guardrail matrix

Lifecycle point	Risk	Control	Type	Fail mode	Escalation
before external write	unauthorised recipient	recipient policy + approval	deterministic/human	closed	security queue

A.13 Threat model

Include assets, actors, trust boundaries, abuse cases, prompt injection, data exfiltration, authority escalation, memory poisoning, supply-chain risk, denial of service, repudiation, and incident controls.

A.14 Verification and completion

Specify:

- authoritative postconditions;
- independent verifier;
- evidence bundle;
- tolerance and partial success;
- uncertain-commit handling;
- false-completion prevention.

A.15 Evaluation plan

Define task set, environment, graders, repeated trials, risk weighting, confidence intervals, release gates, shadow and canary phases, drift metrics, ownership, and review cadence.

A.16 Operations

- SLOs and error budgets;
- queue and capacity plan;
- model/tool failure modes;
- kill switches;
- on-call ownership;
- incident severity;
- rollback;
- data retention;
- cost controls.

A.17 Open decisions

Decision	Options	Recommendation	Evidence	Owner	Due date

A.18 Approval record

Record explicit sign-off from product, domain, security, privacy, operations, and model/evaluation owners appropriate to the risk.

Tool-contract template

A tool is a versioned capability boundary. Its contract must be understandable without reading the model prompt.

B.1 Identity

- **Tool name:** [stable verb-noun name]
- **Version:** [semantic or dated version]
- **Owner:** [team]
- **Purpose:** [one sentence]
- **Side-effect class:** [read-only / reversible write / irreversible write]
- **Allowed callers:** [agents/workflows]

B.2 Preconditions

- required task state;
- required identity and scopes;
- required approval;
- expected resource version;
- environment constraints;
- data-classification restrictions.

B.3 Input schema

Framework-neutral pseudocode.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "required": ["resource_id", "expected_version", "request_id"],
  "properties": {
    "resource_id": {"type": "string", "pattern": "^[A-Z]+-[0-9]+$"},
    "expected_version": {"type": "integer", "minimum": 1},
    "request_id": {"type": "string", "minLength": 16},
    "reason": {"type": "string", "maxLength": 500}
  }
}
```

Define units, time zones, enum meanings, null semantics, defaults, and maximum sizes.

B.4 Canonicalisation

Specify how to normalise:

- identifiers;
- paths and URLs;
- case and whitespace;
- monetary values and currency;
- timestamps;
- recipient lists;
- optional fields.

Approval and idempotency bind to canonical input.

B.5 Authorisation

- credential used;
- token lifetime;
- tenant and resource scope;
- row/column restrictions;
- argument-level policy;
- delegation rules;
- prohibited combinations.

B.6 Execution semantics

- synchronous or asynchronous;
- timeout;
- idempotency behaviour;
- transaction boundary;
- concurrency control;
- retry-safe errors;
- uncertain-commit behaviour;
- cancellation semantics.

B.7 Output schema

Return machine-readable status, identifiers, versions, and evidence. Separate business failure from transport failure.

Framework-neutral pseudocode.

```
{
  "status": "applied",
  "resource_id": "CASE-42",
  "previous_version": 7,
  "new_version": 8,
  "external_operation_id": "op_9182",
  "verification_hint": {"tool": "case.get", "expected_version": 8}
}
```

B.8 Error taxonomy

Code	Meaning	Retry?	Agent response	Human escalation?
`INVALID_ARGUMENT`	schema or semantic failure	no	repair once	no
`PRECONDITION_FAILED`	state changed	no blind retry	refresh state	maybe
`RATE_LIMITED`	transient capacity	yes with backoff	wait	no
`PERMISSION_DENIED`	insufficient authority	no	stop	maybe
`COMMIT_UNKNOWN`	outcome uncertain	no re-execute	verify by idempotency key	yes if unresolved

B.9 Guardrails

List pre-execution and post-execution controls, including sensitive-data handling, egress, resource limits, and approval preview.

B.10 Verification

Define the authoritative read-back or invariant check required after success. A transport `200 OK` is not sufficient.

B.11 Compensation

- compensating tool;
- allowed time window;
- prerequisites;
- irreversible residual effects;
- required approval.

B.12 Observability and privacy

Record canonical arguments, policy result, identity, timing, external ID, and result classification. Specify redactions and fields forbidden in logs.

B.13 Test contract

Required tests:

- schema/property tests;
- authorisation matrix;
- idempotency and duplicate delivery;
- concurrency conflict;
- timeout before and after commit;
- malformed tool output;
- compensation;
- trace redaction;
- adversarial arguments.

B.14 Change management

Document compatibility, deprecation, migration, and behaviour for in-flight workflows.

Guardrail-design template

C.1 Guardrail identity

- **Name:** [name]
- **Owner:** [team]
- **Risk addressed:** [specific failure]
- **Protected asset:** [data/money/system/user]
- **Lifecycle point:** [before inference / context / tool / state / side effect / output / monitoring]

C.2 Threat or failure statement

Use a concrete form:

> When [actor or failure] causes [condition], the system may [harm] because [missing control].

C.3 Control type

Select one or more:

- deterministic rule;
- schema validation;
- statistical classifier;
- model judge;
- human approval;
- sandbox or network containment;
- quota or budget;
- postcondition verifier;
- monitoring and anomaly detection.

C.4 Inputs and provenance

List every input, its source of truth, freshness, sensitivity, and whether an attacker can influence it.

C.5 Decision

Define outcomes beyond allow/deny:

- allow;
- transform or redact;
- reduce authority;
- request clarification;
- request approval;
- quarantine;
- retry with safe modification;
- terminate;
- raise incident.

C.6 Fail-open or fail-closed

State behaviour when the guardrail, dependency, or classifier is unavailable. Justify it by consequence and reversibility.

C.7 False positives and false negatives

- consequence of incorrect block;
- consequence of incorrect allow;
- acceptable thresholds;
- override process;
- monitoring for drift.

C.8 Independence

Explain whether the guardrail uses evidence independent from the proposing model. A model checking its own answer with the same context is weak independence.

C.9 Trigger record

Framework-neutral pseudocode.

```
{
  "guardrail": "external-recipient-policy-v4",
  "decision": "requires_approval",
  "reason_code": "NEW_EXTERNAL_DOMAIN",
  "evidence": ["recipient:partner.example"],
  "policy_version": "2026-07-15",
  "task_id": "task_01..."
}
```

C.10 Tests

Include:

- positive and negative examples;
- boundary values;
- adversarial and obfuscated inputs;
- dependency outage;
- stale policy;
- conflicting controls;
- bypass through alternate tool or path;
- model attempts to reframe the action;
- latency and throughput;
- trace and redaction.

C.11 Operational response

Define alert, owner, escalation, user messaging, evidence retention, and kill-switch interaction.

C.12 Review cadence

Set review triggers: policy change, incident, tool change, model change, drift threshold, or scheduled audit.

Threat-model template

D.1 Scope and assumptions

- system and version;
- environments;
- included and excluded components;
- users and tenants;
- maximum authority;
- data classifications;
- assumed trusted services.

D.2 Assets

Inventory:

- credentials and capability tokens;
- sensitive user and enterprise data;
- money and contractual actions;
- source code and infrastructure;
- workflow state and approvals;
- memory and artifacts;
- logs and traces;
- model and policy configuration;
- reputation and user trust.

D.3 Actors

- legitimate user;
- careless user;
- malicious user;
- compromised external content publisher;
- compromised tool or MCP server;
- malicious insider;
- supply-chain attacker;
- model/provider failure;
- operational fault.

D.4 Trust boundaries

Draw data and control flows. Mark where content, identity, authority, and state cross boundaries. Include browser, sandbox, network egress, tool gateway, memory, and observability systems.

D.5 Abuse-case catalogue

For each abuse case:

ID	Scenario	Preconditions	Impact	Existing controls	Residual risk	Test

Cover at least:

- direct and indirect prompt injection;
- tool-description poisoning;
- data exfiltration;
- privilege escalation;
- confused deputy;
- cross-tenant access;
- memory poisoning;
- approval spoofing or replay;
- duplicate side effect;
- uncertain commit;
- malicious file or code execution;
- dependency compromise;
- denial of wallet or resource exhaustion;
- trace leakage;
- unsafe model or prompt update;
- repudiation and audit gaps.

D.6 Attack trees

For high-impact goals, decompose possible paths. Example goal: unauthorised payment.

Framework-neutral pseudocode.

```
obtain unauthorised payment
  OR
  + forge or replay approval
  + change proposal after approval
  + exploit over-privileged execution credential
  + inject instructions that bypass policy
  + duplicate a legitimate instruction
  + compromise tool or external system
```

Place independent controls on different branches.

D.7 Information-flow policy

Define which data labels may flow to which tools, models, users, memory stores, and outputs. Include transformation rules such as masking or aggregation.

D.8 Authority model

Document capability issuance, scope, lifetime, delegation, revocation, and separation of duties. Identify ambient credentials and remove them.

D.9 Security tests

- adversarial prompts and indirect injections;
- malicious tool output;
- argument smuggling;
- path and URL confusion;
- encoded sensitive data exfiltration;
- cross-tenant identifiers;
- stale/replayed approvals;
- prompt/policy downgrade;
- budget exhaustion;
- compromised dependency simulation;
- kill-switch drill.

D.10 Incident plan

Specify detection, containment, credential revocation, task quarantine, evidence preservation, affected-action discovery, compensation, notification, and recovery.

D.11 Residual-risk decision

Record accepted, mitigated, transferred, and rejected risks with owner and review date.

Evaluation-plan template

E.1 Decision the evaluation supports

Examples:

- choose between model configurations;
- approve a tool for production;
- expand autonomy from draft to execution;
- release a prompt or workflow change;
- verify recovery after infrastructure migration.

State the decision, owner, and required confidence.

E.2 Production task distribution

Define strata by:

- task class;
- risk;
- complexity/horizon;
- language or locale;
- tool set;
- environment;
- ambiguity;
- adversarial exposure;
- frequency.

Specify sampling weights and how production-derived tasks are de-identified.

E.3 Test-set construction

- source of tasks;
- inclusion/exclusion;
- hidden test policy;
- contamination checks;
- synthetic-task purpose;
- version and freeze date;
- expert review;
- environment fixtures.

E.4 Trial unit

Define one trial exactly:

Framework-neutral pseudocode.

```
{
  "task_version": "qa-1842-v3",
  "system_version": "agent-2026.07.28",
  "model": "provider/model-version",
  "prompt_version": "p18",
  "tool_versions": {"browser": "6", "ticket": "3"},
  "seed_or_nonce": "...",
  "environment_snapshot": "env-42",
  "budget": {"wall_seconds": 900, "model_calls": 25}
}
```

E.5 Repetition and sample size

State:

- trials per task/configuration;
- paired or unpaired design;
- minimum meaningful improvement;
- confidence interval method;
- stopping rule;
- handling of infrastructure-invalid trials.

Use repeated trials for stochastic outcomes. Report per-trial success, pass-at-least-once, and repeated reliability separately where relevant.

E.6 Graders

For each grader:

Grader	Type	Target	Inputs	Independence	Calibration	Failure mode

Prefer deterministic execution and authoritative state. Use model judges for semantic properties only after calibration against expert labels.

E.7 Metrics

At minimum:

- outcome correctness;
- verified completion;
- safe completion;
- policy violations;
- partial completion;
- false completion;
- latency;
- cost per verified success;
- human review time;
- recovery success;
- trace completeness.

Weight high-consequence failures separately from ordinary task failure.

E.8 Failure taxonomy

Predefine labels and adjudication rules. Distinguish agent, tool, environment, task, grader, policy, and infrastructure defects.

E.9 Transcript and evidence audit

Specify sampling rate, reviewer expertise, blinded comparison, disagreement resolution, and how discovered grader defects are corrected without contaminating results.

E.10 Ablation matrix

Test the marginal value of:

- model strength;
- prompt section;
- retrieval/context strategy;
- memory;
- verifier;
- reflection;
- multi-agent decomposition;
- retry policy;
- tool description;
- workflow structure.

E.11 Security evaluation

Include adversarial content, permission boundaries, data flow, approval binding, tool misuse, resource exhaustion, and fail-closed behaviour.

E.12 Release gates

Define exact thresholds and non-negotiable blockers. Example:

Framework-neutral pseudocode.

```
release only when:
- high-risk unsafe-action count = 0 across the red-team suite;
- verified completion improves by >= 5 percentage points;
- lower confidence bound is above current production baseline;
- cost per verified success increases by <= 10%;
- recovery and rollback drills pass;
- no critical grader defect is open.
```

E.13 Deployment evaluation

Describe shadow, canary, online metrics, rollback triggers, drift detection, and review dates.

E.14 Ownership and artifacts

Name owners for task set, environment, graders, statistics, security, sign-off, and result publication. Preserve trial records and a human-readable decision report.

Trial-and-trace schema

This schema links evaluation and production telemetry. Adapt field names, but preserve the distinction between task, run, attempt, action, and evidence.

F.1 Trial record

Framework-neutral pseudocode.

```
{
  "trial_id": "trial_01...",
  "evaluation_id": "eval_2026_07_qa",
  "task": {
    "id": "task_1842",
    "version": "3",
    "class": "ui_ticket_verification",
    "risk": "medium",
    "source": "production_derived"
  },
  "system": {
    "application_version": "agent-2026.07.28",
    "workflow_version": "qa-wf-12",
    "prompt_versions": ["qa-system-18", "qa-report-7"],
    "policy_version": "policy-2026-07-15",
    "model_routes": ["visual-standard", "reasoning-escalation"],
    "tool_versions": {"browser": "6", "ticket": "3"}
  },
  "environment": {
    "fixture_id": "env-42",
    "snapshot": "sha256:...",
    "region": "eu-west",
    "started_at": "..."
  },
```

```
  "budget": {
    "wall_seconds": 900,
    "model_calls": 25,
    "tool_calls": 100,
    "cost_limit": "12.00 GBP"
  },
  "outcome": {
    "status": "partially_completed",
    "verified": true,
    "safe": true,
    "score": 0.75,
    "failure_labels": ["criterion_ambiguous"]
  },
  "metrics": {
    "latency_ms": 416000,
    "model_cost": "2.74 GBP",
    "tool_cost": "0.41 GBP",
    "human_review_seconds": 83
  },
  "trace_root": "trace_01...",
  "evidence_bundle": "artifact://...",
  "grader_results": ["grade://..."],
  "invalid_trial": false
}
```

Use the currency and representation required by your accounting system. Never mix currencies without an explicit conversion source and date.

F.2 Trace span

Framework-neutral pseudocode.

```
{
  "trace_id": "trace_01...",
  "span_id": "span_17",
  "parent_span_id": "span_12",
  "kind": "tool",
  "name": "browser.click",
  "started_at": "...",
  "ended_at": "...",
  "task_id": "task_1842",
  "workflow_id": "wf_818",
  "run_id": "run_4",
  "attempt": 1,
  "observation_id": "obs_81",
  "input_ref": "object://redacted-input",
  "canonical_input_hash": "sha256:...",
  "identity_scope": "qa-test/read-write-fixture",
  "policy_decision": "allow",
  "status": "ok",
  "output_ref": "object://redacted-output",
  "attributes": {
    "tool_version": "6",
    "idempotency_key": null,
    "external_operation_id": null
  }
}
```

F.3 Model-call record

Record:

- provider and exact model identifier;
- configuration and tool choice mode;
- prompt-section hashes;
- context-source IDs and versions;
- token counts and cache use;
- latency and cost;
- structured-output validity;
- refusal, abstention, or escalation;
- output reference under retention policy.

Do not log hidden reasoning. Record the observable rationale or evidence fields required by the task contract.

F.4 State-transition event

Framework-neutral pseudocode.

```
{
  "event_type": "workflow.transition",
  "from": "EXECUTING",
  "to": "WAITING_FOR_APPROVAL",
  "reason_code": "EXTERNAL_SUBMISSION",
  "condition_evidence": ["proposal://prop_44"],
  "workflow_version": "12",
  "sequence": 91,
  "fencing_token": 8
}
```

F.5 Approval record

Include:

- approver identity and role;
- canonical proposal hash;
- exact consequence preview;
- decision and time;
- expiry;
- authentication strength;
- independent-approval requirements;
- revocation.

F.6 Evidence record

Framework-neutral pseudocode.

```
{
  "evidence_id": "ev_92",
  "type": "authoritative_readback",
  "source": "crm.case.get",
  "source_version": "3",
  "captured_at": "...",
  "content_hash": "sha256:...",
  "artifact_ref": "artifact://...",
```

```
  "supports": ["criterion-2"],
  "sensitivity": "confidential",
  "retention": "90d"
}
```

F.7 Grader record

Record target, rubric version, inputs, result, confidence, explanation, calibration set, and adjudication status. A corrected grader must not overwrite historical results; create a new version and regrade.

F.8 Privacy and retention

Classify every payload. Store references and hashes when full content is unnecessary. Define access controls, retention periods, deletion propagation, and legal holds. Treat traces as sensitive production data.

F.9 Query examples

The schema should support questions such as:

- Which policy version allowed an unsafe action?
- What context source preceded a tool-selection regression?
- How many tasks recovered after worker loss?
- What is cost per verified success by task and model route?
- Which approvals were executed after expiry?
- Where do verifier and agent disagree most often?

Production-readiness checklist

A checklist is a release aid, not a substitute for evidence. Mark each item `pass`, `fail`, `not applicable`, or `accepted risk`, and link proof.

G.1 Product and task

- **Check:** The production task distribution is documented.
- **Check:** The user-visible objective and non-goals are explicit.
- **Check:** Completion, partial completion, and failure are distinct.
- **Check:** The autonomy level matches consequence and reversibility.
- **Check:** Users know when they are interacting with an agent and what it can do.

G.2 Architecture

- **Check:** Deterministic invariants are enforced outside the prompt.
- **Check:** The model's authority is narrower than the application service's authority.
- **Check:** State, memory, and artifacts have separate semantics.
- **Check:** Long-running work has durable state and explicit transitions.
- **Check:** Multi-agent coordination has a measured benefit.

G.3 Prompts and context

- **Check:** Task contracts define objective, constraints, evidence, completion, and escalation.
- **Check:** Prompt assembly is versioned and reproducible.
- **Check:** Untrusted content is labelled and cannot override policy.
- **Check:** Context has provenance, freshness, and size controls.
- **Check:** Prompt regressions and ablations exist.

G.4 Tools and authority

- **Check:** Every tool has a typed, versioned contract.
- **Check:** Credentials are task-scoped and short-lived where possible.
- **Check:** Argument-level authorisation is enforced.
- **Check:** Side-effecting calls have idempotency and concurrency controls.
- **Check:** Uncertain commits are verified rather than blindly retried.
- **Check:** Compensation is defined for each material mutation.

G.5 Security and guardrails

- **Check:** Threat model covers prompt injection, exfiltration, authority, memory, supply chain, and denial of service.
- **Check:** Controls exist at the tool and side-effect boundaries.
- **Check:** High-risk actions have informed approval tied to canonical arguments.
- **Check:** Fail-open/fail-closed behaviour is explicit.
- **Check:** Egress, file, browser, and sandbox policies are tested.
- **Check:** Kill switches and credential revocation are operational.

G.6 Verification

- **Check:** Success is based on authoritative postconditions.
- **Check:** The verifier is independent enough to catch proposer errors.
- **Check:** Every acceptance criterion has evidence.
- **Check:** External writes are read back and reconciled.
- **Check:** The system can represent unresolved or partial outcomes.

G.7 Evaluation

- **Check:** Realistic and adversarial tasks are included.
- **Check:** Environments reset correctly and invalid trials are classified.
- **Check:** Repeated trials quantify stochastic reliability.
- **Check:** Graders are calibrated and audited.
- **Check:** High-risk failures are separately weighted.
- **Check:** Release gates and minimum meaningful improvements are predeclared.
- **Check:** Shadow and canary plans exist.

G.8 Reliability and operations

- **Check:** SLOs measure verified user outcomes.
- **Check:** Queues have leases, fencing, deadlines, backpressure, and dead-letter handling.
- **Check:** Recovery is tested under worker, model, tool, and network failure.
- **Check:** Capacity and per-tenant isolation are defined.
- **Check:** Model, prompt, policy, tool, and workflow rollback are possible.
- **Check:** Incident response and evidence preservation are rehearsed.

G.9 Observability and privacy

- **Check:** Task, run, action, approval, and evidence IDs are linked.
- **Check:** Canonical tool arguments and policy decisions are traceable.
- **Check:** Sensitive data is redacted and retention-limited.
- **Check:** Cost per verified success is measurable.
- **Check:** Trace completeness is monitored.

G.10 Governance

- **Check:** Owners are assigned for product, model, tools, policy, evals, security, and operations.
- **Check:** Model/provider changes require evaluation.
- **Check:** In-flight task migration is defined.
- **Check:** Human escalation has staffing and response targets.
- **Check:** Residual risks are explicitly accepted by authorised owners.

Incident-review template

H.1 Incident header

- **Incident ID:** [ID]
- **Severity:** [SEV]
- **Status:** [open/contained/resolved]
- **Start, detection, containment, resolution:** [times]
- **Incident commander:** [name]
- **Systems and tenants affected:** [scope]

H.2 Executive summary

State what happened, impact, current safety, and required follow-up in plain language. Do not attribute cause to “AI unpredictability”.

H.3 Impact

Quantify:

- affected tasks/users;
- incorrect or unauthorised actions;
- financial or data impact;
- reversibility and compensation status;
- human workload;
- exposure duration;
- regulatory or contractual implications.

H.4 Timeline

Use event time and discovery time separately.

Time	Event	Detection/source	Response

H.5 Expected versus actual behaviour

- task contract;
- expected state transitions;
- expected policy and approval;
- expected postcondition;
- actual sequence.

H.6 Causal analysis

Trace:

1. user/task input;
2. context and provenance;
3. prompt/policy/model versions;
4. model proposal;
5. tool validation and authority;
6. external effect;
7. verification;
8. retry/recovery;
9. monitoring and human response.

Identify contributing conditions and missing controls. Separate trigger, proximate cause, systemic causes, and detection gaps.

H.7 Five control questions

- Why was the action possible?
- Why was it allowed?
- Why was it not verified or contained?
- Why was it not detected earlier?
- Why did recovery or compensation not limit impact?

H.8 Evidence

Link immutable traces, approvals, artifacts, external operation IDs, model-call metadata, environment snapshots, and affected resource list. Observe privacy restrictions.

H.9 Immediate remediation

Document kill switches, permission reduction, task quarantine, rollback, compensation, user communication, and evidence preservation.

H.10 Corrective actions

Action	Control layer	Owner	Priority	Due	Verification

Prefer authority, invariant, tool, verification, and state fixes before prompt-only changes.

H.11 Evaluation updates

Add a regression task that reproduces the causal conditions. Update grader, red-team, recovery, and rollout tests. State how the fix will be shown to work.

H.12 Lessons and accepted risk

Record what changes in design principles, ownership, or rollout policy. Obtain explicit acceptance for unresolved risk.

Framework-selection checklist

Evaluate a shortlist using a weighted score and a failure-oriented proof of concept.

I.1 Workload requirements

- **Check:** single request or long-running workflow;
- **Check:** local, cloud, or hybrid execution;
- **Check:** typed API tools, browser/computer use, or both;
- **Check:** human approvals and interrupts;
- **Check:** multi-tenant isolation;
- **Check:** MCP or A2A interoperability;
- **Check:** strict data residency or private deployment;
- **Check:** expected task volume and horizon.

I.2 Loop ownership

- Who schedules model calls?
- Can the application intercept every action?
- Is control flow explicit or implicit?
- Can model selection change per node?
- Can deterministic nodes dominate the workflow?

I.3 State and durability

- Is state process-local, database-backed, or event-sourced?
- What survives a worker crash?
- How are timers, waits, and approvals represented?
- Are replay constraints documented?
- Are leases, idempotency, and versioning built in or application-owned?
- Can in-flight workflows migrate across code versions?

I.4 Tool and policy controls

- Typed schemas and validation;
- pre/post tool hooks;

- per-tool permissions;
- argument-level authorisation;
- approval integration;
- sandbox and egress control;
- secret handling;
- hosted versus self-hosted tools.

I.5 Context, memory, and artifacts

- session semantics;
- durable memory;
- provenance and freshness;
- context compression/caching;
- artifact storage;
- deletion and retention;
- cross-tenant isolation.

I.6 Verification and evaluation

- trace export;
- deterministic grader integration;
- replay and test fixtures;
- evaluation APIs;
- human-review tooling;
- version comparison;
- online monitoring.

I.7 Operations

- OpenTelemetry or equivalent;
- cost and token accounting;
- queue/capacity controls;
- deployment model;
- failure recovery;
- kill switches;
- support and maintenance maturity.

I.8 Interoperability and lock-in

- model-provider portability;
- tool-protocol support;
- state export;
- trace export;
- workflow definition portability;
- proprietary hosted dependencies;
- licence and governance.

I.9 Proof-of-concept tests

Do not build a demo-only happy path. Inject:

- worker crash during tool execution;
- duplicate delivery;

- approval wait across deployment;
- tool timeout after possible commit;
- prompt injection in retrieved content;
- model change;
- policy denial;
- trace export and forensic replay;
- high queue load.

I.10 Decision record

Score only after running the critical-path proof of concept. Document what remains application-owned. A framework that omits a required invariant is not disqualified if the ownership boundary is clear and testable; a framework that hides it may be.

Code-versus-model decision checklist

Use this checklist for every proposed model step.

J.1 Prefer deterministic code when

- **Check:** the rule has one correct interpretation;
- **Check:** the result can be computed from structured data;
- **Check:** a schema, enum, arithmetic, or state invariant decides it;
- **Check:** the action is irreversible or regulated;
- **Check:** consistency across repeated runs is required;
- **Check:** latency or cost makes inference wasteful;
- **Check:** an authoritative system can answer directly;
- **Check:** the operation is retry, timeout, scheduling, locking, or transaction control;
- **Check:** the system must prove why the decision was allowed.

J.2 Prefer a model when

- **Check:** input is unstructured or linguistically ambiguous;
- **Check:** multiple valid strategies exist;
- **Check:** semantic ranking or synthesis is required;
- **Check:** the task needs judgement over incomplete evidence;
- **Check:** rigid rules would have excessive maintenance or poor recall;
- **Check:** uncertainty can be surfaced and contained;
- **Check:** output is a proposal that code can validate.

J.3 Use model plus deterministic control when

- **Check:** the model extracts a typed object and code validates it;
- **Check:** the model proposes a plan and code compiles an allowed graph;
- **Check:** the model chooses a tool and the gateway checks authority and arguments;
- **Check:** the model proposes a mutation and code performs compare-and-set commit;
- **Check:** the model interprets evidence and a verifier checks postconditions;
- **Check:** the model estimates uncertainty and policy decides escalation.

J.4 Questions before approving a model step

1. What semantic ambiguity requires a model?
2. What is the worst plausible wrong output?
3. Can the output be constrained to a schema?
4. Which inputs are untrusted?
5. What deterministic preconditions apply?
6. What authority does the step receive?
7. How is the result independently verified?
8. What is the fallback or abstention path?
9. How will this be evaluated on repeated real tasks?
10. What cheaper deterministic or specialist alternative was tested?

J.5 Red flags

Reject or redesign when:

- the prompt contains business rules that are not enforced elsewhere;
- the model is asked to decide its own permissions;
- the same model both proposes and "independently" approves a high-risk action;
- success is based only on model self-report;
- retries have no error taxonomy or idempotency;
- confidence is treated as probability without calibration;
- another agent is added without a measurable information or authority boundary;
- memory is required for correctness;
- a broad tool replaces a narrow domain operation merely for convenience.

J.6 Decision record

Framework-neutral pseudocode.

```
Decision: [code / model / hybrid]
Semantic ambiguity: [description]
Deterministic controls: [preconditions, schema, policy]
Authority: [scope]
Verification: [postcondition]
Failure handling: [abstain, retry, escalate, compensate]
Evaluation evidence: [link]
Owner and review date: [owner/date]
```

Glossary

Abstention. A deliberate decision not to answer or act because evidence, authority, or expected utility is insufficient.

A2A. Agent2Agent protocol for communication and task collaboration between independently deployed agents.

Activity. A non-deterministic unit of work, such as an API or model call, executed outside replayed workflow logic.

Agency. The degree to which a system may choose and execute actions without immediate human direction.

Agent. A system that observes state, selects actions, receives results, and repeats until it stops, escalates, or completes.

Approval binding. Cryptographically or logically tying approval to exact canonical action arguments so later changes invalidate it.

Artifact. A durable file or structured output such as a patch, report, plan, dataset, or evidence bundle.

At-least-once delivery. A delivery guarantee under which a task or message may be processed more than once.

Backpressure. Slowing or rejecting new work when downstream systems cannot safely keep up.

Benchmark card. A compact record of a benchmark's task domain, system, harness, budget, metric, date, and limitations.

Bulkhead. Isolation that prevents one failing workload from consuming all shared capacity.

Calibration. Agreement between predicted confidence and observed correctness over repeated cases.

Canonicalisation. Converting inputs into a stable representation before authorisation, hashing, approval, or idempotency checks.

Capability. A technical operation the system can perform; distinct from authority to perform it in a particular task.

Capability token. A narrowly scoped credential that conveys explicit permission to invoke a capability.

Circuit breaker. A control that stops calls to a dependency after repeated or dangerous failures and later tests recovery.

Compensation. A domain-specific action that mitigates or reverses a completed side effect when ordinary rollback is impossible.

Completion contract. The postconditions and evidence required before a task can be marked successful.

Confused deputy. A component with authority being manipulated into using it for an unauthorised purpose.

Context engineering. Selecting, transforming, ordering, labelling, and budgeting information made visible to a model.

Control plane. Components that own task lifecycle, routing, scheduling, budgets, and transition decisions.

Dead-letter queue. Quarantine for tasks that cannot progress safely after classified handling attempts.

Deterministic invariant. A rule that must always hold and is enforced by code rather than model judgement.

Durable execution. Execution that preserves task state and progress across crashes, deployments, waits, and network failures.

Egress. Data leaving a security boundary, such as through a tool, network request, message, or generated file.

Error budget. The failure allowance implied by a service-level objective.

Event sourcing. Persisting ordered state-change events and deriving current state from that history.

Evidence bundle. The linked observations, tool results, versions, and verification records supporting an outcome.

Fencing token. A monotonically increasing ownership number used to reject writes from stale workers.

Guardrail. A lifecycle control that allows, blocks, transforms, contains, escalates, or monitors behaviour.

Handoff. Transfer of responsibility to another agent or component with explicit state, evidence, authority, and completion contract.

Harness. Runtime that assembles context, invokes models, validates proposals, dispatches tools, stores state, applies policy, and records traces.

Heartbeat. Periodic signal that a worker holding a lease remains alive and making progress.

Idempotency. Property that repeated execution with the same key produces one intended business effect.

Information-flow control. Rules restricting how labelled data may move between sources, models, tools, memory, and outputs.

Lease. Temporary ownership of a task or resource that expires unless renewed.

Memory. Information retained to improve future model usefulness; it must not be the sole store of correctness-critical state.

MCP. Model Context Protocol for exposing tools, prompts, and resources to model clients.

Model judge. A model used to grade semantic properties of another system's output; it requires calibration and audit.

Pass@k. Probability or empirical rate that at least one of k attempts succeeds.

Pass-all-k. Probability or empirical rate that all k repeated attempts succeed; a measure of consistency.

Policy decision point. Component that evaluates identity, action, resource, context, and state to allow or deny authority.

Postcondition. A fact that must hold after an action for the action or task to be considered successful.

Prompt injection. Untrusted content manipulating model behaviour by being interpreted as instructions.

Provenance. Record of where information came from, when it was obtained, and how it was transformed.

Replay. Re-running code-owned orchestration against recorded history to reconstruct state. Replay can reproduce workflow transitions; it cannot make model calls or external operations deterministic.

Safe completion. Task success without violating defined safety, security, or policy constraints.

Semantic layer. Governed definitions for business metrics, dimensions, entities, grains, and allowed relationships.

Service-level objective (SLO). Measurable target for reliability or performance over a defined period.

Shadow evaluation. Running a candidate system on production inputs without applying its side effects.

State. Durable information required for correct continuation, recovery, ownership, approval, and completion.

Tool gateway. Deterministic broker that validates, authorises, executes, logs, and verifies tool calls.

Trace. Causal record connecting task, context, decisions, tools, policy, state transitions, evidence, and outcome.

Tripwire. A control that halts or changes execution immediately when a defined risk condition is detected.

Uncertain commit. A failure where the caller cannot tell whether an external side effect occurred; it requires reconciliation, not blind retry.

Value of information. Expected reduction in decision loss from obtaining additional evidence, compared with its cost and delay.

Verified completion. Task success supported by authoritative postconditions or an independent acceptance check.

Workflow. Explicit control graph whose states and legal transitions are owned by deterministic software.

Requirement-compliance matrix

This matrix records where the major review requirements are addressed.

Requirement	Primary location	Delivery
system boundary and harness	Parts 1 and 4	architecture, loop, invariants, pseudocode
deterministic-task decision framework	Parts 2 and 3; Appendix J	classification, three hybrid patterns, transition checks
prompt/specification engineering	Part 5	contracts, assembly, precedence, versioning, ablations
context engineering	Part 6	provenance, progressive disclosure, poisoning, tests
state and memory	Part 7	schemas, migration, artifacts, correctness boundary
tool design	Part 8; Appendix B	contracts, authority, idempotency, errors, compensation
MCP and A2A	Part 9	role split, gateway, delegation, freshness note
durable execution	Part 10	replay, leases, fencing, state machine, migration
verification and completion	Part 11	postconditions, independent verifier, evidence bundles
lifecycle guardrails	Part 12; Appendix C	nine lifecycle points, response taxonomy, fail modes
security and threat modelling	Part 13; Appendix D	threats, information flow, identity, incident controls
decision theory	Part 14	expected loss, value of information, abstention, stopping

Requirement	Primary location	Delivery
evaluation design	Part 15; Appendix E	task distribution, repeated trials, graders, release gates
observability and incidents	Part 16; Appendices F and H	event schema, SLOs, error budgets, response
model selection and economics	Part 17	routing, cascades, multi-agent, cost model, capacity
repository-scale engineering and harness autopsies	Part 18A-18B	framework-neutral design plus pinned Claude Code and Codex source audit
QA and verification	Part 19	queue, browser/API evidence, report and rollout
browser/computer use	Part 20A	stale observations, approvals, injection, verification
enterprise data analysis	Part 20B	semantic layer, query compiler, privacy, reconciliation
long-running research	Part 21A	source ledger, claim graph, stopping, citation audit
high-risk approved workflow	Part 21B	dual control, immutable proposal, transaction, recovery
reusable production templates	Appendices A-J	ten complete templates/checklists
stable references and evidence labels	front matter and bibliography	labelled claims, durable URLs, access dates

The guide now contains twenty-one substantive parts, six production case studies, ten reusable appendices, a glossary, a compliance matrix, a stable bibliography, and an embedded reproducibility package. It is intentionally a field manual rather than a catalogue of framework APIs.

Edition and maintenance record

Changelog

Edition 1.7.0 - 29 July 2026

- corrected the tested evaluation dependency reference and added companion-filename resolution checks;
- reclassified the OpenAI Agents SDK block as a versioned documentation mapping for 0.19.1 rather than source-contract checked;
- strengthened external receipt validation and source/PDF section and URL checks;
- renamed the installed-package inventory as an environment attestation rather than a reconstructive dependency lock;
- retained explicit limitations for upstream raw SHA-256, credentialed Claude defer/resume testing, PDF/UA and named external reviews.

Edition 1.6.0 - 29 July 2026

- corrected the duplicated Edition 1.5/1.4 changelog heading and added a general edition-history validator;
- removed stale Edition 1.4 metadata from every companion-package artefact;
- made publication checks bind canonical Markdown, deterministic code-block extraction, normalised rendered PDF text, TOC/bookmark destinations, embedded archive identity, and an external final build receipt;
- added exact build and post-processing scripts plus a build-environment manifest;
- corrected the companion-package invocation to `bash run_publication_checks.sh` and preserved executable mode in the ZIP;
- moved the visible file-attachment annotation to the rendered companion-package section;
- added a compact Part 18B chapter map without expanding the technical scope.

Edition 1.5.0 - 29 July 2026

- repaired the complete Part 18B numbering sequence and regenerated navigation;
- added source-audited Claude core-loop lifecycle and Codex tool scheduling/cancellation sections;
- triangulated the harness conclusions against Aider, mini-SWE-agent and OpenHands;
- bound manuscript code-block verification to the canonical Markdown and deterministic extraction output;
- standardised source-integrity records to carry Git blob SHA-1 and raw-content SHA-256 fields with explicit verification flags;
- added an explicit source-audit cutoff;
- narrowed deferred approval to a source-contract-checked integration design pending a credentialed cross-process test.

Edition 1.4.0 - 29 July 2026

- repaired the PDF text layer by disabling common font ligatures and added Poppler-/PyMuPDF extraction tests;
- replaced manifest-format checking with a verifier that fetches immutable source files and recomputes Git blob SHAs;
- added Claude root exports, client, internal client and deferred-result parser source contracts;
- replaced clustered-row Wilson intervals with percentile task-cluster bootstrap marginal intervals;
- renamed the autonomous safety metric to match its actual denominator and numerator;
- made code-block verification self-contained and mandatory through a machine-readable extraction;
- removed bytecode artefacts, fixed the visible attachment annotation and corrected the MCP date;
- added the contract-level delta from recovered Claude Code 2.1.88 to official SDK 0.2.128 / bundled CLI 2.1.220.

Edition 1.3.0 - 29 July 2026

- regenerated the table of contents from a clean multi-pass build and added automated destination checks against rendered PDF pages;
- corrected and executed the percentile task-cluster bootstrap example from the embedded companion package;
- added a one-sided task-cluster safety non-inferiority bound and clarified the McNemar composite endpoint;

- replaced the Claude deny-and-interrupt approval path with the pinned SDK's deferred-tool lifecycle and explicit durable resume contract;
- embedded and separately distributed the complete reproducibility archive with a stable SHA-256 and pinned environment;
- expanded the source-contract manifest with symbols, source files, blob hashes and explicit syntax/import/runtime verification flags;
- labelled Part 18B findings as observed mechanism, implementation-author comment or manuscript inference, and added a not-audited boundary;
- added failure vignettes to Parts 4, 6, 8, 12 and 15;
- tightened authority, workflow-path and distributed-systems language;
- produced a linearised PDF and retained an explicit accessibility limitation rather than claiming unverified PDF/UA conformance.

Edition 1.2.0 - 29 July 2026

- corrected the ADK mapping to `google.adk.workflow` and edge-defined `Workflow` at pinned ADK 2.5.0 source;
- replaced bare Claude hook arrays with `HookMatcher` registrations pinned to Claude Agent SDK 0.2.128 source;
- removed provider-block runtime-test claims and added source-contract verification status;
- replaced the non-auditable evaluation with a complete synthetic dataset, task-cluster bootstrap, task-level McNemar analysis, grader audit, risk-coverage results and recovery set;
- added a source-audited production harness autopsy covering Claude Code compaction, child isolation, authority attenuation, prompt-cache construction and capability discovery;
- added a pinned Codex autopsy covering App Server protocol, thread/turn/item state, phase-aware compaction, backpressure and authority configuration;
- corrected authority-boundary terminology, distributed-systems wording and section numbering;
- pinned new source references to immutable repository commits and exposed companion verification artifacts.

Edition 1.1.0 - 29 July 2026

- replaced undocumented Claude approval event with the version-pinned `can_use_tool` contract;
- replaced legacy Google ADK `SequentialAgent` mapping with ADK 2.5.0 graph `Workflow`;
- classified framework code blocks and added compatibility metadata;
- corrected repeated-trial analysis to task-cluster resampling and task-level paired endpoints;
- renamed severity-weighted safety statistics and expanded escalation and coverage metrics;
- corrected value-of-information treatment;
- added a complete guardrail execution walkthrough and identity/delegation mechanics;
- increased body and code size for print readability.

Accessibility and format note

The PDF includes searchable text, bookmarks, document metadata, embedded fonts, descriptive table headings, and a linearised object layout for web viewing. The build pipeline tests extraction with both Poppler and PyMuPDF for common ligature-sensitive words. This edition is **not tagged and is not certified as PDF/UA conformant**. A tagged XeLaTeX build was attempted, but the available LaTeX tagging stack did not complete reliably across the manuscript's long tables; the release therefore does not claim accessibility work that was not successfully validated. Specialist remediation remains required for logical reading order, alternative text, complex-table semantics, annotations and assistive-technology behaviour.

Bibliography

All web documentation was accessed on **29 July 2026** unless another access date is stated. Documentation links are version-sensitive; production systems should pin the exact SDK or protocol version they use.

[R1] OpenAI. *OpenAI Agents SDK - Python documentation.* Software documentation. URL: https://openai.github.io/openai-agents-python/. Describes agents, runners, hand-offs, guardrails, sessions, tracing, tools, and related runtime primitives. Accessed 29 July 2026.

[R2] OpenAI. *Tracing - OpenAI Agents SDK.* Software documentation. URL: https://openai.github.io/openai-agents-python/tracing/. Describes tracing of generations, tool calls, hand-offs, guardrails, and custom spans. Accessed 29 July 2026.

[R3] OpenAI. *Guardrails - OpenAI Agents SDK.* Software documentation. URL: https://openai.github.io/openai-agents-python/guardrails/. Documents input, output, and tool guardrail concepts. Accessed 29 July 2026.

[R4] OpenAI. *Tools - OpenAI Agents SDK.* Software documentation. URL: https://openai.github.io/openai-agents-python/tools/. Documents function tools, hosted tools, agents-as-tools, schemas, and tool controls. Accessed 29 July 2026.

[R5] OpenAI. *Sessions - OpenAI Agents SDK.* Software documentation. URL: https://openai.github.io/openai-agents-python/sessions/. Describes persistent conversation/session history in the SDK. Accessed 29 July 2026.

[R6] OpenAI. *Sandbox guide - OpenAI Agents SDK.* Software documentation. URL: https://openai.github.io/openai-agents-python/sandbox/guide/. Describes isolated workspaces and sandbox-agent patterns. Accessed 29 July 2026.

[R7] OpenAI. *New tools for building agents.* Product and engineering announcement, 11 March 2025. URL: https://openai.com/index/new-tools-for-building-agents/. Introduces the Responses API, built-in tools, and Agents SDK.

[R8] Anthropic. *Claude Agent SDK overview.* Software documentation. URL: https://code.claude.com/docs/en/agent-sdk/overview. Describes the agent loop, context management, tools, hooks, sessions, permissions, and telemetry. Accessed 29 July 2026.

[R9] Anthropic. *Building effective agents.* Engineering article, 19 December 2024. URL: https://www.anthropic.com/research/building-effective-agents. Distinguishes workflows from agents and documents common orchestration patterns.

[R10] Anthropic. *Effective harnesses for long-running agents.* Engineering article, 26 November 2025. URL: https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents. Documents cross-context progress artifacts, incremental work, and verification failures.

[R11] Anthropic. *Harness design for long-running application development.* Engineering article, 24 March 2026. URL: https://www.anthropic.com/engineering/harness-design-long-running-apps. Extends long-running harness design for application-building workloads.

[R12] Anthropic. *Managed Agents permission policies.* Software documentation. URL: https://platform.claude.com/docs/en/managed-agents/permission-policies. Describes hosted-agent permission controls and policy configuration. Accessed 29 July 2026.

[R13] Anthropic. *MCP connector.* API documentation. URL: https://platform.claude.com/docs/en/agents-and-tools/mcp-connector. Describes connecting remote MCP servers and configuring tool access. Accessed 29 July 2026.

[R14] Anthropic. *Code execution with MCP: building more efficient AI agents.* Engineering article, 4 November 2025. URL: https://www.anthropic.com/engineering/code-execution-

with-mcp. Explains on-demand tool loading and filtering intermediate results outside model context.

[R15] Google. *Agent Development Kit documentation.* Software documentation. URL: https://adk-labs.github.io/adk-docs/. Documents agents, workflow agents, tools, sessions, evaluation, observability, MCP, and A2A. Accessed 29 July 2026.

[R16] Google Cloud. *Build agents with Agent Development Kit.* Product documentation. URL: https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk. Describes ADK within Google Cloud's agent platform. Accessed 29 July 2026.

[R17] Model Context Protocol project. *The 2026-07-28 MCP Specification Release Candidate.* Project announcement, 21 May 2026. official specification page . Describes proposed stateless-core, extension, authorisation, task, and deprecation changes.

[R18] Model Context Protocol project. *Beta SDKs for the 2026-07-28 specification.* Project announcement, 29 June 2026. beta-SDK announcement . Records beta implementation status; archive checked at release archive .

[R19] Agent2Agent Protocol project, Linux Foundation. *A2A Protocol v1.0.* Protocol documentation, 2026. v1.0 specification . Defines discovery, tasks, messages, artifacts, asynchronous interaction, and interoperability; latest project entry at latest project page . Accessed 29 July 2026.

[R20] Microsoft. *Durable Task programming model overview.* Software documentation. URL: https://learn.microsoft.com/en-us/azure/durable-task/common/programming-model-overview. Explains event-sourced orchestration, replay, activities, and determinism. Accessed 29 July 2026.

[R21] Temporal Technologies. *Workflows.* Software documentation. URL: https://docs.temporal.io/workflows. Describes durable workflow execution and recovery semantics. Accessed 29 July 2026.

[R22] Temporal Technologies. *Temporal SDKs and replay.* Software documentation. URL: https://docs.temporal.io/encyclopedia/temporal-sdks. Explains workflow code, activities, event history, and replay. Accessed 29 July 2026.

[R23] Temporal Technologies. *Workflow versioning.* Software documentation. URL: https://docs.temporal.io/develop/dotnet/workflows/versioning. Documents safe changes to long-running workflow definitions. Accessed 29 July 2026.

[R24] Dapr project. *Dapr Agents introduction.* Software documentation. URL: https://docs.dapr.io/developing-ai/dapr-agents/dapr-agents-introduction/. Describes durable agent workflows and runtime integration. Accessed 29 July 2026.

[R25] Dapr project. *Dapr Agents patterns and hooks.* Software documentation. patterns documentation and hooks documentation . Describes workflow, human-interaction, and lifecycle interception patterns. Accessed 29 July 2026.

[R26] OWASP GenAI Security Project. *OWASP Top 10 for LLM Applications 2025.* Security guidance. URL: https://genai.owasp.org/llm-top-10/. Taxonomy of major security risks in LLM applications. Accessed 29 July 2026.

[R27] OWASP GenAI Security Project. *LLM01:2025 Prompt Injection.* Security guidance. URL: https://genai.owasp.org/llmrisk/llm01-prompt-injection/. Defines direct and indirect prompt-injection risk and mitigations. Accessed 29 July 2026.

[R28] OWASP GenAI Security Project. *LLM06:2025 Excessive Agency.* Security guidance. URL: https://genai.owasp.org/llmrisk/llm062025-excessive-agency/. Identifies excessive functionality, permissions, and autonomy as root causes. Accessed 29 July 2026.

[R29] Microsoft. *Defend against indirect prompt injection attacks.* Zero Trust security guidance. URL: https://learn.microsoft.com/en-us/security/zero-trust/sfi/defend-indirect-prompt-

injection. Recommends defence in depth for untrusted content and agent tools. Accessed 29 July 2026.

[R30] Microsoft. *Information flow control: moving toward secure autonomous agents.* Engineering article. URL: https://commandline.microsoft.com/information-flow-control-moving-toward-secure-autonomous-agents/. Describes tracking and restricting the flow of untrusted or sensitive information. Accessed 29 July 2026.

[R31] Microsoft. *Manage agentic memory safety.* Zero Trust security guidance. URL: https://learn.microsoft.com/en-us/security/zero-trust/sfi/manage-agentic-memory-safety. Treats persistent memory as a distinct poisoning and disclosure surface. Accessed 29 July 2026.

[R32] Mateusz Dziemian, Maxwell Lin, Xiaohan Fu, et al. *How Vulnerable Are AI Agents to Indirect Prompt Injections? Insights from a Large-Scale Public Competition.* arXiv:2603.15714, 2026. DOI: 10.48550/arXiv.2603.15714 . Large public red-team study across tool-calling, coding, and computer-use agents.

[R33] Microsoft. *Secure agentic systems.* Zero Trust security guidance. URL: https://learn.microsoft.com/en-us/security/zero-trust/sfi/secure-agentic-systems. Covers identities, least privilege, boundaries, monitoring, and governance for agents. Accessed 29 July 2026.

[R34] OpenAI. *ChatGPT Agent System Card.* System card, 17 July 2025. URLs: https://deployomentsafety.openai.com/chatgpt-agent and https://cdn.openai.com/pdf/839e66fc-602c-48bf-81d3-b21eacc3459d/chatgpt_agent_system_card.pdf. Documents computer-use risks, mitigations, confirmations, and monitoring.

[R35] Noam Michael, Daniel BenShushan, Jacob Bien, and Don A. Moore. *Confidence Calibration in Large Language Models.* arXiv:2605.23909, 2026. DOI: 10.48550/arXiv.2605.23909 . Preregistered empirical study of confidence and accuracy across task difficulty.

[R36] Changdae Oh, Seongheon Park, To Eun Kim, et al. *Uncertainty Quantification in LLM Agents: Foundations, Emerging Challenges, and Opportunities.* ACL 2026 Main Conference; arXiv:2602.05073. DOI: 10.48550/arXiv.2602.05073 . Agent-specific uncertainty formulation and research agenda.

[R37] Glenn Zhang, Treasure Mayowa, Jason Fan, et al. *Direct Confidence Alignment: Aligning Verbalized Confidence with Internal Confidence in Large Language Models.* ACL 2025 Student Research Workshop; arXiv:2512.11998. DOI: 10.48550/arXiv.2512.11998 . Shows that verbalised and internal confidence can diverge and that alignment effects are architecture-dependent.

[R38] Tejas Srinivasan, Jack Hessel, Tanmay Gupta, et al. *Selective "Selective Prediction": Reducing Unnecessary Abstention in Vision-Language Reasoning.* Findings of ACL 2024; arXiv:2402.15610. DOI: 10.48550/arXiv.2402.15610 . Demonstrates evidence acquisition to reduce over-abstention without increasing error in the tested setting.

[R39] Ziyang Guo, Yifan Wu, Jason Hartline, and Jessica Hullman. *Explaining and Improving Information Complementarities in Multi-Agent Decision-Making.* arXiv:2502.06152, 2025. DOI: 10.48550/arXiv.2502.06152 . Decision-theoretic treatment of information value in human-AI workflows.

[R40] Anthropic. *Demystifying evals for AI agents.* Engineering article, 9 January 2026. URL: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents. Covers task design, repeated trials, pass metrics, graders, transcripts, and eval saturation.

[R41] Anthropic. *Quantifying infrastructure noise in agentic coding evals.* Engineering article, 5 February 2026. URL: https://www.anthropic.com/engineering/infrastructure-noise. Examines how environment and infrastructure variation distort coding-agent measurements.

[R42] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024; arXiv:2310.06770. DOI: 10.48550/arXiv.2310.06770 . Introduces execution-based repository issue resolution.

[R43] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. *SWE-Bench+: Enhanced Coding Benchmark for LLMs.* arXiv:2410.06992, 2024. DOI: 10.48550/arXiv.2410.06992 . Empirical audit of solution leakage and weak tests in SWE-bench variants.

[R44] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation (EvalPlus).* NeurIPS 2023; arXiv:2305.01210. DOI: 10.48550/arXiv.2305.01210 . Demonstrates the importance of expanded execution tests.

[R45] Linghao Zhang, Shilin He, Chaoyun Zhang, et al. *SWE-bench Goes Live!* arXiv:2505.23419, 2025. DOI: 10.48550/arXiv.2505.23419 . Live-updatable benchmark with recent issues, broader repositories, and reproducible environments.

[R46] SWE-bench project. *SWE-bench Verified.* Benchmark documentation. URL: https://www.swebench.com/SWE-bench/guides/verified/. Documents a human-validated subset and its curation criteria. Accessed 29 July 2026.

[R47] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. *WebArena: A Realistic Web Environment for Building Autonomous Agents.* ICLR 2024; arXiv:2307.13854. DOI: 10.48550/arXiv.2307.13854 . Reports low end-to-end baseline success relative to humans in realistic websites.

[R48] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments.* NeurIPS 2024 Datasets and Benchmarks; arXiv:2404.07972. DOI: 10.48550/arXiv.2404.07972 . Evaluates open-ended desktop-computer tasks and reports a large human-system gap in the original benchmark.

[R49] Frank F. Xu, Yufan Song, Boxuan Li, et al. *TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks.* NeurIPS 2025 Datasets and Benchmarks; arXiv:2412.14161. DOI: 10.48550/arXiv.2412.14161 . Evaluates workplace-style digital tasks across tools and roles.

[R50] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, et al. *OSWorld 2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks.* arXiv:2606.29537, version 2, 2026. DOI: 10.48550/arXiv.2606.29537 . Introduces 108 long workflows, binary and partial completion, and separate safety reporting.

[R51] Anthropic. *How we built our multi-agent research system.* Engineering article, 13 June 2025. URL: https://www.anthropic.com/engineering/multi-agent-research-system. Production report on parallel research subagents, coordination, token use, and evaluation.

[R52] Anthropic. *Computer use tool.* API documentation. URL: https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/computer-use-tool. Documents the application-managed computer-use loop, environment mediation, and safety considerations. Accessed 29 July 2026.

[R53] Tilmann Gneiting and Adrian E. Raftery. *Strictly Proper Scoring Rules, Prediction, and Estimation.* Journal of the American Statistical Association, 102(477), 2007, pp. 359-378. DOI: 10.1198/016214506000001437 . Foundation for Brier and other proper scoring rules.

[R54] A. Philip Dawid. *The Well-Calibrated Bayesian.* Journal of the American Statistical Association, 77(379), 1982, pp. 605-610. DOI: 10.1080/01621459.1982.10477856 . Classical formulation of probabilistic calibration.

[R55] James A. Hanley and Barbara J. McNeil. *The Meaning and Use of the Area under a Receiver Operating Characteristic Curve.* Radiology, 143(1), 1982, pp. 29-36. DOI: 10.1148/radiology.143.1.7063747 . Interpretation of discrimination metrics; calibration must still be evaluated separately.

[R56] Robert L. Winkler. *Scoring Rules and the Evaluation of Probability Assessors.* Journal of the American Statistical Association, 64(327), 1969, pp. 1073-1078. DOI: 10.1080/01621459.1969.10501069 . Proper evaluation of probabilistic judgements.

[R57] Edwin B. Wilson. *Probable Inference, the Law of Succession, and Statistical Inference.* Journal of the American Statistical Association, 22(158), 1927, pp. 209-212. DOI: 10.1080/01621459.1927.10502953 . Classic interval for independent binomial proportions; not used for the clustered repeated-trial intervals in the worked study.

[R58] Robert G. Newcombe. *Interval Estimation for the Difference Between Independent Proportions: Comparison of Eleven Methods.* Statistics in Medicine, 17(8), 1998, pp. 873-890. DOI: 10.1002/(SICI)1097-0258(19980430)17:8%3C873::AID-SIM779%3E3.0.CO;2-I . Practical confidence intervals for proportion differences; paired designs require paired methods or bootstrap.

[R59] Quinn McNemar. *Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages.* Psychometrika, 12, 1947, pp. 153-157. DOI: 10.1007/BF02295996 . Paired binary comparison used in the worked evaluation.

[R60] Jacob Cohen. *A Coefficient of Agreement for Nominal Scales.* Educational and Psychological Measurement, 20(1), 1960, pp. 37-46. DOI: 10.1177/001316446002000104 . Cohen's kappa for agreement beyond chance.

[R61] NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1).* Standard/guidance, July 2024. 10.6028/NIST.AI.600-1 . Risk governance, measurement, content provenance, and incident considerations for generative AI.

[R62] MITRE. *ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems.* Threat-knowledge base. URL: https://atlas.mitre.org/. Structured adversarial tactics and techniques relevant to AI systems. Accessed 29 July 2026.

[R63] Google. *Site Reliability Engineering: How Google Runs Production Systems.* O'Reilly Media, 2016. URL: https://sre.google/sre-book/table-of-contents/. Error budgets, monitoring, incident response, and production reliability foundations.

[R64] LangChain. *LangGraph documentation: persistence, durable execution, interrupts, and human-in-the-loop.* Software documentation. URL: https://docs.langchain.com/oss/python/langgraph/. Low-level state-graph runtime; application semantics and external transactions remain user-owned. Accessed 29 July 2026.

[R65] Pat Helland. *Life beyond Distributed Transactions: An Apostate's Opinion.* CIDR 2007. PDF: http://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf. Explicit entities, messages, idempotency, and compensating work across service boundaries.

[R66] Hector Garcia-Molina and Kenneth Salem. *Sagas.* Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, pp. 249-259. DOI: 10.1145/38713.38742 . Foundational model for long-lived transactions and compensation.

[R67] Jeffrey Dean and Luiz Andre Barroso. *The Tail at Scale.* Communications of the ACM, 56(2), 2013, pp. 74-80. DOI: 10.1145/2408776.2408794 . Tail-latency mechanisms and the operational importance of p95/p99 behaviour.

[R68] Martin Kleppmann. *Designing Data-Intensive Applications.* O'Reilly Media, 2017. ISBN 978-1-4493-7332-0. Distributed state, logs, transactions, replication, and stream-processing foundations used throughout this guide.

[R69] Leslie Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System.* Communications of the ACM, 21(7), 1978, pp. 558-565. DOI: 10.1145/359545.359563 . Causal ordering and event-history foundations.

[R70] OpenTelemetry Authors. *OpenTelemetry Specification.* Open standard. URL: https://opentelemetry.io/docs/specs/otel/. Vendor-neutral traces, metrics, logs, context propagation, and semantic conventions. Accessed 29 July 2026.

[R71] Soheil Khodayari, Xuenan Zhang, Bhupendra Acharya, and Giancarlo Pellegrino. *Indirect Prompt Injection in the Wild: An Empirical Study of Prevalence, Techniques, and Objectives.* arXiv:2604.27202, 2026. DOI: 10.48550/arXiv.2604.27202 . Large-scale web measurement and controlled model experiments showing non-zero compliance and representation effects.

[R72] Matthew ffrench-Constant, Daniel Yang, Xinmeng Huang, and Sanyam Kapoor. *ConfidenceBench: Evaluating Confidence Calibration in Large Language Models.* arXiv:2607.20526, 2026. DOI: 10.48550/arXiv.2607.20526 . Recent benchmark using Brier score and repeated runs to separate calibration from accuracy.

[R73] Google LLC. *Google ADK Python 2.5.0 source contract: package version, workflow exports, graph construction, and `LlmAgent` tool callbacks.* Commit `6bab08fc803d`, 2026. URLs: https://github.com/google/adk-python/blob/6bab08fc803d26853417c4d6e71704b1a72e035e/src/google/adk/version.py, https://github.com/google/adk-python/blob/6bab08fc803d26853417c4d6e71704b1a72e035e/src/google/adk/workflow/__init__.py, https://github.com/google/adk-python/blob/6bab08fc803d26853417c4d6e71704b1a72e035e/src/google/adk/workflow/_graph.py, https://github.com/google/adk-python/blob/6bab08fc803d26853417c4d6e71704b1a72e035e/src/google/adk/workflow/_workflow.py, and https://github.com/google/adk-python/blob/6bab08fc803d26853417c4d6e71704b1a72e035e/src/google/adk/agents/llm_agent.py. Pinned source for the corrected ADK mapping. Accessed 29 July 2026.

[R74] Anthropic. *Claude Agent SDK Python 0.2.128 type and lifecycle contract.* Commit `f8b9ec923982`, 2026. URL: https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/src/claude_agent_sdk/types.py. Defines `HookMatcher`, hook callback shapes, `can_use_tool`, permission results, shadowing warnings, deferred tool use, session stores and option fields. Accessed 29 July 2026.

[R75] OpenAI. *OpenAI Agents SDK for Python 0.19.1 release and versioned human-in-the-loop documentation.* URLs: https://github.com/openai/openai-agents-python/releases/tag/v0.19.1 and https://openai.github.io/openai-agents-python/human_in_the_loop/. Used only for a versioned documentation mapping; the block is neither source-contract checked nor labelled runtime-tested. Accessed 29 July 2026.

[R76] ChinaSiro. *Claude Code source-map restoration README for `@anthropic-ai/claude-code` 2.1.88.* Unofficial research snapshot, commit `a8a678cb6244`. URL: https://github.com/ChinaSiro/claude-code-sourcemap/blob/a8a678cb6244e6770e1e421767ff0987a1d95549/README.md. The repository states that it is reconstructed from public package artifacts, is not an official Anthropic repository, and may not match the original internal structure. Accessed 29 July 2026.

[R77] ChinaSiro. *Observed Claude Code 2.1.88 compaction implementation.* Unofficial reconstructed source, commit `a8a678cb6244`. URL: https://github.com/ChinaSiro/claude-code-sourcemap/blob/a8a678cb6244e6770e1e421767ff0987a1d95549/restored-src/src/services/compact/compact.ts. Source-observed evidence for payload stripping, API-round truncation, file and child-state reconstruction, plan/skill/tool/MCP reinjection, boundary metadata and cache telemetry. Accessed 29 July 2026.

[R78] ChinaSiro. *Observed Claude Code 2.1.88 subagent runner.* Unofficial reconstructed source, commit `a8a678cb6244`. URL: https://github.com/ChinaSiro/claude-code-sourcemap/blob/a8a678cb6244e6770e1e421767ff0987a1d95549/restored-src/src/tools/AgentTool/runAgent.ts. Source-observed evidence for context projection, file-state handling, permission scoping, child identity, transcripts, worktrees, liveness and MCP lifecycle. Accessed 29 July 2026.

[R79] ChinaSiro. *Observed Claude Code 2.1.88 forked-agent context implementation.* Unofficial reconstructed source, commit `a8a678cb6244`. URL: https://github.com/ChinaSiro/claude-code-sourcemap/blob/a8a678cb6244e6770e1e421767ff0987a1d95549/restored-src/src/utils/forkedAgent.ts. Source-observed evidence for cache-critical request fields, cloned mutable state, explicit callback sharing, child cancellation, transcript recording and cleanup. Accessed 29 July 2026.

[R80] OpenAI. *Codex App Server protocol documentation.* Official repository, commit `fe01054a28fa`. URL: https://github.com/openai/codex/blob/fe01054a28fa4bd04716d9ceadb410f2443a50ce/codex-rs/app-server/README.md. Documents the bidirectional JSON-RPC interface, capability handshake, generated schemas, thread/turn/item primitives, lifecycle events, interruption, resumption, forking, approvals, bounded queues and overload errors. Accessed 29 July 2026.

[R81] OpenAI. *Codex local compaction implementation.* Official repository, commit `fe01054a28fa`. URL: https://github.com/openai/codex/blob/fe01054a28fa4bd04716d9ceadb410f2443a50ce/codex-rs/core/src/compact.rs. Source for phase-aware initial-context injection, hooks, retry/session behaviour, checkpoint metadata, replacement history and compaction analytics. Accessed 29 July 2026.

[R82] OpenAI. *Codex compact task selection.* Official repository, commit `fe01054a28fa`. URL: https://github.com/openai/codex/blob/fe01054a28fa4bd04716d9ceadb410f2443a50ce/codex-rs/core/src/tasks/compact.rs. Source for local, remote and remote-v2 compaction selection and feature gating. Accessed 29 July 2026.

[R83] Anthropic. *Claude Agent SDK Python 0.2.128 public root exports.* Commit `f8b9ec923982`. URL: https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/src/claude_agent_sdk/__init__.py. Confirms root-level exports for `ClaudeSDKClient`, `HookMatcher`, `DeferredToolUse`, task lifecycle types, session-store types and context-usage types. Accessed 29 July 2026.

[R84] Anthropic. *Claude Agent SDK Python 0.2.128 client and internal client contracts.* Commit `f8b9ec923982`. URLs: https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/src/claude_agent_sdk/client.py and https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/src/claude_agent_sdk/_internal/client.py. Confirms stateful client, resume, hook conversion and session-store materialisation surfaces. Accessed 29 July 2026.

[R85] Anthropic. *Claude Agent SDK Python 0.2.128 message parser.* Commit `f8b9ec923982`. URL: https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/src/claude_agent_sdk/_internal/message_parser.py. Confirms typed parsing of deferred tool use, task lifecycle, hook events and result messages. Accessed 29 July 2026.

[R86] Anthropic. *Claude Agent SDK Python changelog through 0.2.128.* Commit `f8b9ec923982`. URL: https://github.com/anthropics/claude-agent-sdk-python/blob/f8b9ec923982082a02c485924e0f60367949c3a1/CHANGELOG.md. Records bundled CLI 2.1.220 and supported lifecycle changes including deferred tool use, task terminal states, terminal reasons, session stores, context accounting and callback-shadowing warnings. Accessed 29 July 2026.

[R87] ChinaSiro. *Observed Claude Code 2.1.88 query and tool-execution loop.* Unofficial reconstructed source, commit `a8a678cb6244`. URL: https://github.com/ChinaSiro/claude-code-sourcemap/blob/a8a678cb6244e6770e1e421767ff0987a1d95549/restored-src/src/query.ts. Source-observed evidence for streaming tool collection, result pairing, fallback cleanup, interruption handling, continuation and terminal reasons. Accessed 29 July 2026.

[R88] OpenAI. *Codex tool router.* Official repository, commit `cef3910ea4d0`. URL: https://github.com/openai/codex/blob/cef3910ea4d09617e50a94e40bf25a6cb2e4e765/codex-rs/core/src/tools/router.rs. Source for model-visible and deferred capabilities, per-tool

parallel/cancellation metadata, retained step context and tool-call source. Accessed 29 July 2026.

[R89] OpenAI. *Codex parallel tool runtime.* Official repository, commit `cef3910ea4d0`. URL: https://github.com/openai/codex/blob/cef3910ea4d09617e50a94e40bf25a6cb2e4e765/codex-rs/core/src/tools/parallel.rs. Source for read/write execution gating, scoped dependency waits, cancellation ownership, teardown and explicit aborted results. Accessed 29 July 2026.

[R90] OpenAI. *Codex tool invocation and output contracts.* Official repository, commit `cef3910ea4d0`. URL: https://github.com/openai/codex/blob/cef3910ea4d09617e50a94e40bf25a6cb2e4e765/codex-rs/core/src/tools/context.rs. Source for invocation provenance, cancellation token, output normalisation and typed aborted output. Accessed 29 July 2026.

[R91] OpenAI. *Codex tool lifecycle notifications.* Official repository, commit `cef3910ea4d0`. URL: https://github.com/openai/codex/blob/cef3910ea4d09617e50a94e40bf25a6cb2e4e765/codex-rs/core/src/tools/lifecycle.rs. Source for start/finish/aborted events carrying call identity, source and outcome. Accessed 29 July 2026.

[R92] Aider-AI. *Aider repository map implementation.* Official repository, commit `5dc9490bb35f`. URL: https://github.com/Aider-AI/aider/blob/5dc9490bb35f9729ef2c95d00a19ccd30c26339c/aider/repomap.py. Source for tree-sitter definitions/references, graph ranking, personalised PageRank, caching and token-budgeted rendering. Accessed 29 July 2026.

[R93] SWE-agent. *mini-SWE-agent default loop.* Official repository, commit `a83fcae82d2a`. URL: https://github.com/SWE-agent/mini-swe-agent/blob/a83fcae82d2a08f0ee0c688f9d137b3566c097f8/src/minisweagent/agents/default.py. Source for the model-action-environment loop, step/cost/time/format limits and trajectory persistence. Accessed 29 July 2026.

[R94] OpenHands. *Agent Canvas architecture.* Official repository, commit `5086cbbed756`. URL: https://github.com/OpenHands/OpenHands/blob/5086cbbed756d0995bfc99a9ff663b2719b4d69a/docs/architecture.md. Source for separation of UI/control, Agent Server, sandbox, local/remote/hosted backends and automation services. Accessed 29 July 2026.